
Documentação de Projeto

para o sistema

Gabinete Virtual

Versão 1.6

Projeto de sistema elaborado pelo aluno Bernardo Cavanellas Biondini e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo dos professores Danilo de Quadros Maia, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa, orientação acadêmica do professor Cleiton Silva Tavares e orientação de TCC II do professor Marco Rodrigo Costa.

14/06/2026

Tabela de Conteúdo

Tabela de Conteúdo	2
Histórico de Revisões	3
1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	2
2.2 Modelos de Usuários	3
Persona 1 - Chiara Biondini	4
Persona 2 - Raquel Assis	5
Persona 3 - Fernanda	6
Persona 4 - Equipe de Atendimento	7
Persona 5 - Equipe Geral (Funcionários)	9
2.3 Modelos de Casos de Uso e Histórias de Usuário	10
2.3.1 Diagrama de Casos de Uso	10
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	16
3. Modelos de Projeto	35
3.1 Diagrama de Classes	35
3.2 Diagramas de Sequência	39
3.3 Diagramas de Comunicação	66
3.4 Arquitetura	74
3.5 Diagramas de Atividade	81
3.6 Diagrama de Componentes e Implantação.	84
4. Projeto de Interface com Usuário	86
4.1 Esboço das Interfaces	86
5. Glossário e Modelos de Dados	108
6. Casos de Teste	115
6.1 Testes Automatizados de Backend e Chatbot	116
6.1.1 Backend: autenticação, permissões e módulos administrativos	116
6.1.2 Backend: agenda, alertas e processamento assíncrono	117
6.1.3 Chatbot	119
Além dos cenários resumidos nas tabelas anteriores, o backend também possui cobertura complementar para módulos institucionais e legislativos por meio de suítes como AmendmentApiTest, ProjectLawApiTest, CmsApiTest, DashboardApiTest, PublicApiTest e AccessProfileApiTest. Essas suítes ampliam a validação dos fluxos administrativos, públicos e de autorização da aplicação.	121
Essa escolha foi coerente com a natureza do projeto, pois grande parte do comportamento esperado no frontend depende diretamente das permissões devolvidas pela API, do estado persistido no banco e da chegada de eventos assíncronos, o que torna mais relevante a validação integrada do que uma suíte isolada de componentes.	

122

Também foram exercitados cenários de borda, como ausência de instituição na demanda, ocultação padrão de demandas descartadas, filtragem restrita para usuários sem permissão de gestão e rejeição de mensagens consideradas inválidas pelo fluxo do chatbot. 122

Com isso, a cobertura adotada priorizou os fluxos mais críticos do domínio, validando tanto o caminho de sucesso quanto regras de acesso, descarte automatizado, conflitos de agenda e entrega assíncrona de notificações. 122

6.2 Testes de Integração e de Sistema 122

6.2.1 Visão Geral 123

6.2.2 Interfaces de Integração Validadas 123

6.2.3 Estratégia Adotada 124

Esses cenários representam bem a lógica adotada no projeto: backend, chatbot e frontend foram validados como partes de um mesmo ecossistema, com ênfase nos fluxos reais de uso e nas consequências assíncronas geradas pelas operações principais. 125

7. Plano de Desenvolvimento 125

8. Cronograma e Processo de Implementação 127

9. Post-Mortem 129

Experiências Positivas 130

Experiências Negativas 130

Lições Aprendidas 131

Repositório do Trabalho 131

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Iniciação do Documento	25/09/2025	N/A	1
Histórias de Usuário	04/10/2025	Adição das histórias de usuário	1.1.1
Diagrama de Casos de Uso	05/10/2025	Adição do Diagrama de Casos de Uso	1.1.2
Finalização Seção 2	11/10/2025	Ajustes e término da seção 2 (sem correções)	1.1.3

Entrega A4	12/10/2025	Adiciona diagramas de classe, de sequência e de comunicação	1.2
Entrega A5	02/11/2025	Seções 3.4, 3.5, 3.6 e 5. Realiza ajustes da correção A4	1.3
Correções A5	14/11/2025	Adição de descrição nas tabelas e imagens e contextualização no texto.	1.3.2
Atualização dos Wireframes	17/11/2025	Atualização dos Wireframes	1.3.3
Seções 6 e 7	30/11/2025	Atividade A7, Seções 6 e 7	1.4
Entrega final	14/06/2026	Alteração de alguns diagramas, alteração na seção de testes. Seção post motem	1.6

1. Introdução

O sistema Gabinete Virtual é uma plataforma web desenvolvida para otimizar a gestão de demandas, emendas parlamentares e atividades administrativas de um gabinete político. Seu objetivo é integrar, em um único ambiente, informações que tradicionalmente ficam dispersas em planilhas, mensagens e documentos, oferecendo uma visão consolidada e estratégica das ações do mandato.

O sistema foi projetado com uma arquitetura cliente-servidor, composta por um *frontend* em React.js e um *backend* em Laravel/PHP, comunicando-se via API REST e utilizando um banco de dados relacional MySQL. Essa combinação garante escalabilidade, modularidade e facilidade de manutenção. Além disso, a aplicação contempla níveis de acesso específicos para diferentes perfis (deputada, chefe de gabinete, gestora de demandas, equipe de atendimento e cidadãos) promovendo transparência, eficiência operacional e controle das informações públicas.

A proposta central é permitir que o gabinete monitore suas ações de forma inteligente, automatizando tarefas repetitivas e proporcionando relatórios e *dashboards* que auxiliem na tomada de decisão política e administrativa.

Integrado ao trabalho está o desenvolvimento de um *Chatbot* para atendimento ao público que foi projetado em Python e servirá como um apoio aos eleitores e à sociedade.

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema Gabinete Virtual. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

2. Modelos de Usuário e Requisitos

Esta seção descreve os atores, perfis e necessidades que interagem com o sistema, além de apresentar os modelos de casos de uso, histórias de usuário e demais representações que orientam o desenvolvimento do projeto.

O objetivo é compreender quem são os usuários, o que esperam do sistema e como suas interações moldam o design e os requisitos funcionais do Gabinete Virtual.

2.1 Descrição de Atores

Nesta subseção é apresentada descrição de cada um dos atores que interagem com o sistema. Os atores definidos na Tabela 1 a seguir representam os diferentes perfis de usuários que interagem com o Gabinete Virtual. Cada ator desempenha funções específicas no contexto de atendimento à população, gestão administrativa e comunicação institucional.

Perfil	Descrição	Objetivos	Nível de acesso
Deputada	Usuária principal, tomadora de decisão política.	Consultar informações estratégicas sobre demandas, emendas, cidades e compromissos.	Leituras e relatórios consolidados
Chefe de Gabinete	Responsável pela coordenação geral da equipe e da agenda.	Organizar compromissos, acompanhar demandas e priorizar atividades.	Leitura, edição e controle sobre registros.
Gestora	Responsável por designar tarefas e acompanhar execuções.	Atribuir responsáveis, monitorar prazos e atualizar status das atividades.	Edição e controle de demandas e eventos.
Gestora de Demandas	Responsável por designar tarefas e acompanhar execuções.	Registrar novas demandas, atualizar informações e alimentar relatórios.	Acesso restrito às funções operacionais.
Atendimento	Responsável pelo contato externo.	Acompanhar demandas, verificar	Acesso restrito às funções operacionais

		informações de lideranças.	
Funcionários	Executor das tarefas administrativas e operacionais		Acesso restrito às funções operacionais
Cidadão (Consulta Pública)	Usuário externo interessado em informações públicas.	Acessar dados do <i>site</i> oficial, como notícias, projetos e emendas. Comunicar com a deputada, solicitar atendimento (<i>chatbot</i>).	Apenas leitura (público).

Tabela 1 - Descrição dos Atores do Sistema Gabinete Virtual

2.2 Modelos de Usuários

Esta subseção apresenta os modelos de usuários desenvolvidos para o Sistema de Gestão de Demandas de Gabinete, a partir da perspectiva do *designer*. Foram utilizados modelos de personas para guiar as decisões de *design* e usabilidade. As Tabelas 2 a 6 apresentam os modelos de usuários adotados no projeto, sendo cada uma explicada individualmente a seguir.

A Tabela 2 apresenta a persona Deputada, considerada o nível máximo de acesso dentro do Gabinete Virtual. Essa persona é responsável pela tomada de decisões estratégicas, consulta de indicadores, acompanhamento de emendas, projetos de lei e demandas gerais. A Deputada utiliza o sistema principalmente para obter visões consolidadas, acessar relatórios completos e validar informações cadastradas pela equipe. Sua necessidade central é ter uma interface simples, rápida e capaz de fornecer dados confiáveis em tempo real, permitindo decisões políticas baseadas em evidências.

Persona 1 - Chiara Biondini	
Cargo	Deputada Estadual
Idade	23 anos
Formação	Administração
Perfil Tecnológico	Básico – Utiliza o sistema principalmente para consultas em dispositivos móveis. Prefere interfaces intuitivas e informações resumidas.
Responsabilidades	Tomada de decisões estratégicas, consulta de informações sobre municípios, demandas e emendas. Representação política e prestação de contas às comunidades.
Motivações	Ter informações rápidas e confiáveis antes de visitas e reuniões políticas; Demonstrar transparência e eficiência na gestão do mandato; Tomar decisões estratégicas baseadas em dados consolidados.
Frustrações	Dificuldade em acessar relatórios resumidos e navegar em sistemas complexos; Falta de informações consolidadas durante visitas aos municípios; Dependência de terceiros para obter informações estratégicas.
Necessidades de Design	Interface simplificada e intuitiva; Visualização de gráficos e relatórios prontos; Acesso rápido via dispositivos móveis; <i>Dashboard</i> com informações-chave; Relatórios executivos sintéticos.
Citação Representativa	“Quero abrir o sistema e entender rapidamente o que está acontecendo nas cidades que visito.”

Tabela 2 - Descrição Persona 1 do Gabinete Virtual

A Tabela 3 descreve a persona Chefe de Gabinete, que atua como figura intermediária entre a Deputada e os demais setores. Esse usuário precisa de acesso ampliado para gerenciar a agenda, supervisionar demandas, acompanhar emendas e auxiliar na priorização de ações. A persona usa o sistema para organizar processos, revisar cadastros e orientar fluxos internos, garantindo que os encaminhamentos estejam alinhados à estratégia do mandato. Seu foco está na coordenação operacional.

Persona 2 - Raquel Assis	
Cargo	Chefe de Gabinete
Idade	38 anos
Formação	Administração Pública
Perfil Tecnológico	Intermediário – Usa sistemas administrativos, planilhas e ferramentas online de agenda. Familiarizada com gestão de equipes remotas.
Responsabilidades	Coordenação da equipe do gabinete; Organização da agenda da deputada; Supervisão do andamento das demandas; Gestão de conflitos internos; Comunicação entre equipe e deputada.
Motivações	Ter uma visão consolidada das atividades do gabinete; Acompanhar o progresso das demandas em tempo real; Garantir que a equipe trabalhe de forma sincronizada; Evitar conflitos de agenda e perda de prazos.
Frustrações	Informações dispersas em planilhas e mensagens; Falta de notificações automáticas; Retrabalho constante; Dificuldade em priorizar tarefas urgentes; Ausência de visão consolidada do gabinete.

Persona 2 - Raquel Assis	
Necessidades de Design	Painel de controle com visão geral; Alertas visuais de prazos e prioridades; Interface limpa e organizada; Filtros por status e responsável; Sistema de notificações eficiente; Calendário integrado com detecção de conflitos.
Citação Representativa	“Preciso ter uma visão rápida de tudo o que está acontecendo no gabinete, sem perder prazos.”

Tabela 3 - Descrição Persona 2 do Gabinete Virtual

Na Tabela 4, é apresentada a persona Gestora de Demandas, responsável pelo gerenciamento direto das solicitações recebidas pelo gabinete. Esse usuário utiliza o sistema para abrir, classificar, atribuir responsáveis, monitorar prazos e atualizar status das demandas. Além disso, acessa relatórios específicos para análise de volume, prazos médios e gargalos. Sua interface exige eficiência, clareza e controle rigoroso dos registros. Essa persona é fundamental para o processamento interno do gabinete.

Persona 3 - Fernanda	
Cargo	Gestora de Demandas
Idade	32 anos
Formação	Gestão Pública
Perfil Tecnológico	Avançado – Usa softwares de produtividade, dashboards e ferramentas de comunicação interna. Confortável com sistemas complexos.
Responsabilidades	Designar responsáveis para cada demanda; Acompanhar o andamento de tarefas; Gerar relatórios de status; Garantir a execução das demandas dentro dos prazos; Analisar métricas de desempenho da equipe.

Persona 3 - Fernanda	
Motivações	Otimizar o trabalho da equipe; Garantir a execução eficiente das demandas; Ter métricas claras de desempenho; Identificar gargalos e oportunidades de melhoria.
Frustrações	Falta de histórico centralizado; Dificuldade para gerar relatórios precisos; Ausência de indicadores de desempenho da equipe; Perda de contexto sobre demandas antigas; Dificuldade em rastrear responsabilidades.
Necessidades de Design	Filtros eficientes e busca avançada; Campos padronizados; Botões de atualização rápida; Feedback visual claro; Histórico de alterações detalhado; Relatórios customizáveis; Indicadores visuais de performance.
Citação Representativa	“Quero um sistema que me mostre o que está pendente e quem é o responsável, de forma visual e objetiva.”

Tabela 4 - Descrição Persona 3 do Gabinete Virtual

A Tabela 5 documenta a persona Atendimento, responsável pelo registro inicial de demandas e pelo relacionamento direto com cidadãos, instituições e lideranças. Esse usuário cadastra informações, anexa documentos, atualiza registros e repassa demandas para análise. Sua interação demanda telas simples, diretas e otimizadas para rapidez, já que frequentemente trabalha sob fluxo contínuo de solicitações. O papel é essencial para a etapa de entrada de dados no sistema.

Persona 4 - Equipe de Atendimento	
Cargo	Atendentes do Gabinete
Idade	26–35 anos

Persona 4 - Equipe de Atendimento	
Formação	Ciências Políticas, Administração, Comunicação Social
Perfil Tecnológico	Intermediário – Acostumados com planilhas e sistemas administrativos simples. Necessitam de orientação clara sobre processos.
Responsabilidades	Registrar e atualizar demandas; Cadastrar instituições e lideranças; Registrar pessoas atendidas; Definir prazos iniciais; Anexar documentos (ofícios); Enviar informações aos gestores.
Motivações	Executar as tarefas com clareza; Evitar retrabalho ou dúvidas sobre o fluxo correto; Contribuir para o bom funcionamento do gabinete; Ter reconhecimento pelo trabalho bem feito.
Frustrações	Falta de padronização no preenchimento; Erros por falta de feedback do sistema; Dificuldade em localizar informações cadastradas anteriormente; Incerteza sobre campos obrigatórios; Perda de tempo em tarefas repetitivas.
Necessidades de Design	Campos com preenchimento guiado; Validações em tempo real; Mensagens de confirmação claras; Histórico de alterações acessível; Tooltips explicativos; Formulários intuitivos; Autocomplete e sugestões.
Citação Representativa	“Quero saber exatamente o que devo preencher e ver que a informação foi salva corretamente.”

Tabela 5 - Descrição Persona 4 do Gabinete Virtual

Por fim, a Tabela 6 apresenta a persona Funcionários, composta por membros da equipe que apoiam atividades administrativas gerais. Esses usuários possuem permissões mais restritas, atuando em cadastros específicos, atualização de informações e suporte a rotinas internas. Não acessam relatórios estratégicos nem funcionalidades sensíveis. O sistema deve oferecer segurança e delimitação clara de permissões, garantindo que esse grupo interaja apenas com os módulos correspondentes ao seu nível de responsabilidade.

Persona 5 - Equipe Geral (Funcionários)	
Cargo	Funcionários do Gabinete
Idade	28-50 anos
Formação	Diversas áreas
Perfil Tecnológico	Básico a Intermediário – Usuários ocasionais do sistema, principalmente para consultas e cadastros básicos.
Responsabilidades	Cadastrar emendas parlamentares; Atualizar status de projetos; Consultar informações sobre cidades, lideranças, e instituições; Manter dados atualizados; Cadastrar e atualizar conteúdo do <i>site</i> .
Motivações	Contribuir com a organização geral do gabinete; Ter acesso fácil a informações necessárias para o trabalho diário; Executar tarefas de forma eficiente; Manter dados precisos e atualizados.
Frustrações	Interfaces complexas; Falta de treinamento adequado; Dificuldade em encontrar as funcionalidades necessárias; Menus confusos; Falta de documentação clara.
Necessidades de Design	Navegação intuitiva; Menus claros e organizados; Busca eficiente; Acesso direto

Persona 5 - Equipe Geral (Funcionários)	
	às funcionalidades mais utilizadas; Ajuda contextual; Manual do usuário acessível; Organização lógica das informações.
Citação Representativa	“Preciso de um sistema simples que me permita encontrar e atualizar informações rapidamente.”

Tabela 6 - Descrição Persona 5 do Gabinete Virtual

2.3 Modelos de Casos de Uso e Histórias de Usuário

Os requisitos funcionais foram organizados em casos de uso, representando as interações de cada ator com o sistema. Os casos de uso abrangem: Gestão de cadastros (instituições, lideranças, cidades), Gestão de conteúdo institucional, Gestão de projetos de lei, Gestão de emendas, Gestão de demandas, Gestão da agenda, Relatórios segmentados

2.3.1 Diagrama de Casos de Uso

A Figura 1 apresenta o diagrama de casos de uso do sistema Gabinete Virtual, reunindo em uma única visão os principais atores, módulos e interações da solução. Os perfis internos do sistema, Deputada, Chefe de Gabinete, Gestora de Demandas, Atendimento e Funcionários, aparecem à esquerda, enquanto os elementos externos Cidadão, WhatsApp Business API e Servidor *WebSocket* complementam o cenário de uso da plataforma. No interior do sistema, estão organizados os casos de uso que representam as funcionalidades centrais do gabinete.

No centro do diagrama está o UC01 – Autenticar no Sistema + Validar Permissão, modelado como um caso de uso transversal. Essa escolha se justifica pelo fato de que todas as funcionalidades acessadas pelo painel administrativo dependem desse fluxo padrão, que envolve autenticação, identificação do perfil do usuário e validação das permissões específicas de cada recurso. Por isso,

os demais casos de uso internos mantêm relação <<include>> com o UC01, conforme indicado na nota explicativa associada ao diagrama.

A partir desse núcleo, o sistema é estruturado em áreas funcionais principais. O UC02 – Gerenciar Cadastros reúne operações de cadastro e consulta de Instituições, Lideranças e Cidades, agrupadas em um único caso de uso devido à semelhança entre essas rotinas administrativas. O UC03 – Gerenciar Conteúdo do *Site* contempla as funcionalidades relacionadas ao CMS, como publicação e edição de notícias, atualização de páginas institucionais e cadastro de projetos exibidos publicamente.

Os módulos parlamentares aparecem no UC04 – Gerenciar Projetos de Lei e no UC05 – Gerenciar Emendas. Em ambos os casos, o diagrama representa operações de cadastro, atualização e consulta, refletindo o acompanhamento contínuo dessas informações no contexto do gabinete. Já o UC06 – Gerenciar Demandas consolida o ciclo de atendimento das demandas, incluindo sua abertura, consulta, atualização de dados, alteração de status, definição de responsável, ajuste de prazo, manipulação de ofício, registro de histórico e visualização do motivo de descarte quando aplicável.

O UC07 – Gerenciar Agenda reúne as operações relacionadas aos eventos, como criação, atualização, visualização do calendário, filtragem por período e verificação de conflitos. Diferentemente da versão anterior do modelo, a criação manual de alertas não é tratada como um caso de uso independente, uma vez que os lembretes passaram a ser entendidos como comportamento derivado da gestão dos eventos e do processamento automático do sistema.

Além dos módulos centrais, o diagrama passa a explicitar o UC08 – Receber Alertas em Tempo Real, responsável por representar a entrega de *pop-ups* e lembretes no *frontend* por meio da infraestrutura de tempo real, e o UC09 – Atender Cidadão via *Chatbot*, que descreve o fluxo de interação do cidadão com o sistema por meio do WhatsApp. Nesse caso, estão compreendidas ações como identificação do cidadão pelo telefone, cadastro quando necessário, definição da preferência por receber atualizações, busca de cidade e instituições relacionadas, envio de demanda via *chatbot*, validações automáticas da mensagem e envio posterior de atualizações sobre a demanda.

Por fim, os casos UC10a, UC10b, UC10c e UC10d representam os relatórios do sistema, cada um com regras próprias de acesso. Os relatórios de Projetos de Lei e Emendas são acessíveis à Deputada e ao Chefe de Gabinete; o relatório de Demandas é disponibilizado à Deputada, ao Chefe de Gabinete, à Gestora de Demandas e ao Atendimento; e o Relatório Consolidado Geral permanece

restrito à Deputada. As notas posicionadas ao lado dos casos de uso reforçam o escopo de cada módulo e ajudam a esclarecer quais operações estão agrupadas em cada caso.

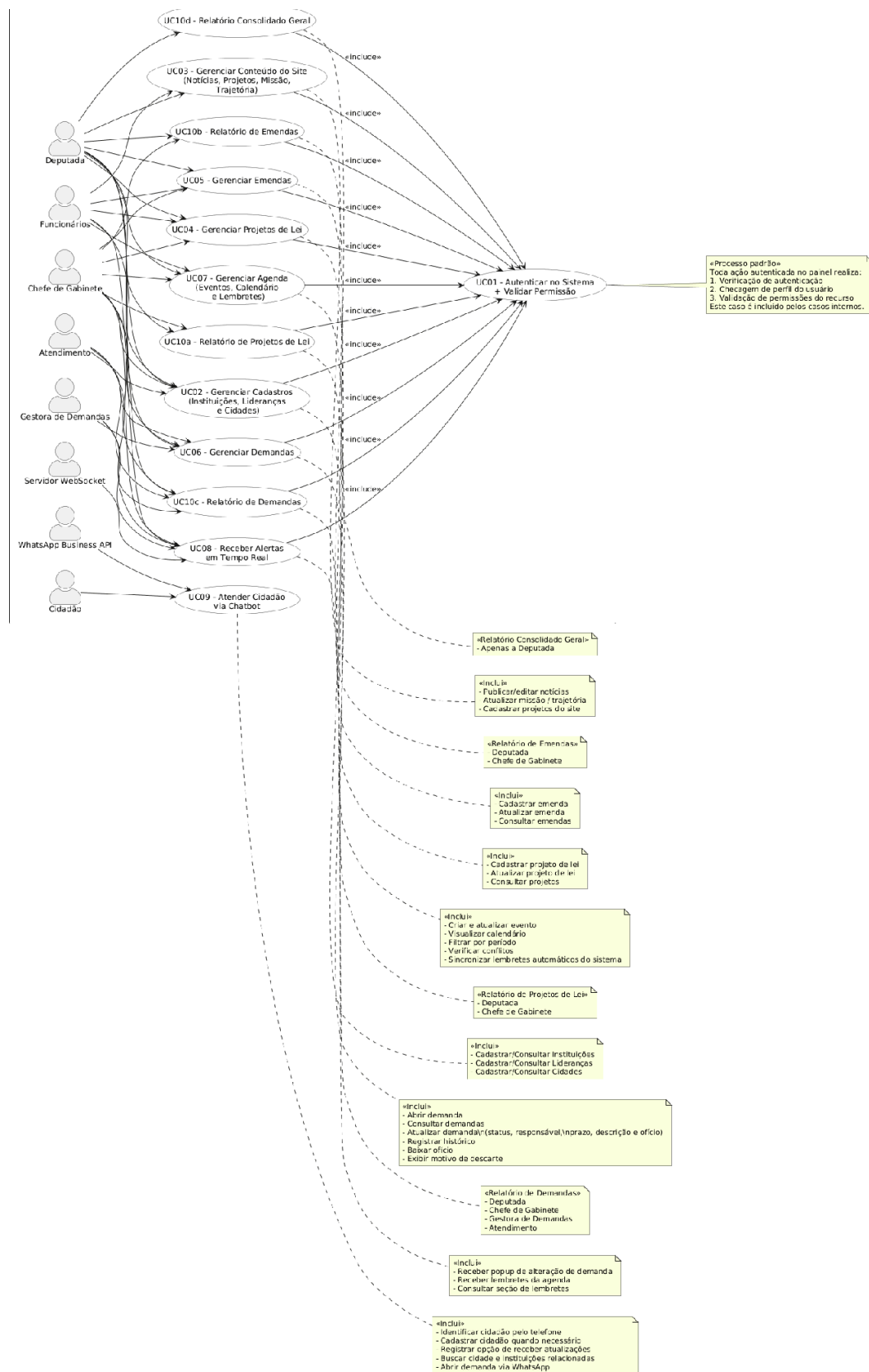


Figura 1 - Diagrama de Casos de Uso do Gabinete Virtual

A Figura 2 apresenta o diagrama de casos de uso do *Chatbot* do Gabinete Virtual, responsável por intermediar a comunicação entre o cidadão e o gabinete parlamentar. Por meio do *chatbot*, o cidadão pode ser identificado automaticamente pelo telefone, complementar seu cadastro quando necessário, informar sua preferência por receber atualizações, enviar demandas relacionadas à sua cidade e acompanhar alterações posteriores nessas solicitações por meio de notificações. O diagrama também evidencia as validações automáticas aplicadas antes do envio da demanda ao *backend* e destaca o caráter opcional do recebimento de conteúdos institucionais.

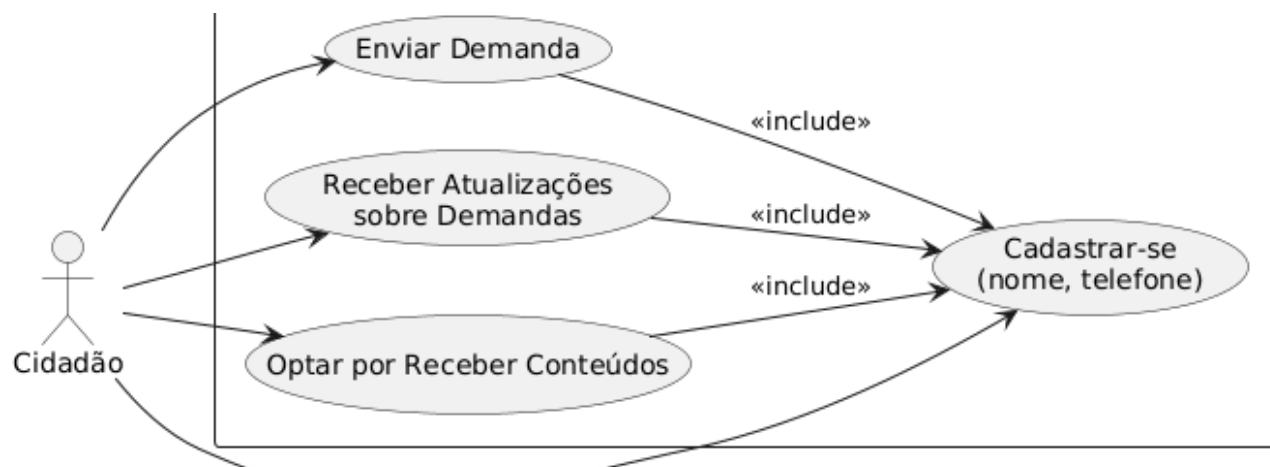


Figura 2 - Diagrama de Casos de Uso do *Chatbot* do Gabinete Virtual

2.3.2 Histórias de Usuário

As histórias de usuário a seguir descrevem as principais interações entre os atores do sistema Gabinete Virtual e suas funcionalidades, contemplando tanto o atendimento ao cidadão por meio do *chatbot* quanto as atividades internas realizadas pela equipe do gabinete. As histórias foram elaboradas de forma orientada ao valor entregue a cada perfil de usuário, mantendo consistência com os casos de uso apresentados e com as necessidades identificadas no Documento de Visão.

Histórias de Usuário – *Chatbot* do Gabinete Virtual (Cidadão)

US01 – Como cidadão, quero me cadastrar no *chatbot* informando nome, para que minhas interações sejam identificadas de forma segura.

US02 – Como cidadão cadastrado, quero enviar demandas relacionadas à minha cidade pelo *chatbot*, para que elas sejam registradas e encaminhadas ao gabinete.

US03 – Como cidadão, quero receber atualizações automáticas sobre minhas demandas, para ser informado sempre que houver mudanças de *status*.

US04 – Como cidadão, quero optar por receber conteúdos institucionais pelo *chatbot*, para poder receber notificações.

Histórias de Usuário – Sistema Gabinete Virtual (Equipe Interna)

US05 – Como atendente do gabinete, quero visualizar as demandas enviadas pelo *chatbot*, para iniciar o atendimento ao cidadão.

US06 – Como gestora de demandas, quero atribuir responsáveis às demandas recebidas, para garantir que cada solicitação tenha um acompanhamento adequado.

US07 – Como gestora de demandas, quero atualizar o status das demandas, para refletir o andamento real do atendimento ao cidadão.

US08 – Como atendente, quero registrar informações sobre o atendimento realizado ao cidadão, para manter o histórico da demanda atualizado.

US09 – Como gestora de demandas, quero gerar relatórios de demandas por período, status ou município, para apoiar a tomada de decisão.

US10 – Como funcionária do gabinete, quero cadastrar e manter atualizadas informações de cidades, instituições e lideranças, para garantir consistência nos registros do sistema.

US11 – Como chefe de gabinete, quero visualizar relatórios consolidados de demandas, projetos e emendas, para acompanhar a atuação geral do mandato.

US12 – Como deputada, quero visualizar relatórios consolidados gerais, para avaliar o impacto das ações do gabinete.

US13 – Como funcionária do gabinete, quero cadastrar e gerenciar conteúdos do *site* institucional, como notícias, projetos e trajetória política, para manter a comunicação com a sociedade atualizada.

US14 – Como deputada, quero gerenciar minha agenda de eventos, com verificação de conflitos e alertas, para organizar meus compromissos de forma eficiente.

US15 – Como chefe de gabinete, quero consultar relatórios de emendas parlamentares e projetos de lei, para acompanhar sua execução e andamento.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Nesta seção, são apresentados os Diagramas de Sequência do Sistema (DSS) para as principais operações que modificam o estado do sistema, acompanhados de seus respectivos contratos de operação.

Como mostra a Figura 3, o usuário informa suas credenciais, e o sistema aciona o serviço de autenticação para validar e verificar se o usuário está ativo. Se as credenciais forem válidas, um *token* é gerado e retornado ao cliente, permitindo que ele acesse as funcionalidades conforme o seu perfil. Caso contrário, uma mensagem de erro é emitida. O diagrama contempla os cenários de autenticação bem-sucedida e falha por credenciais incorretas ou usuário inativo. A tabela 7 apresenta o contrato de operação para o Diagrama.

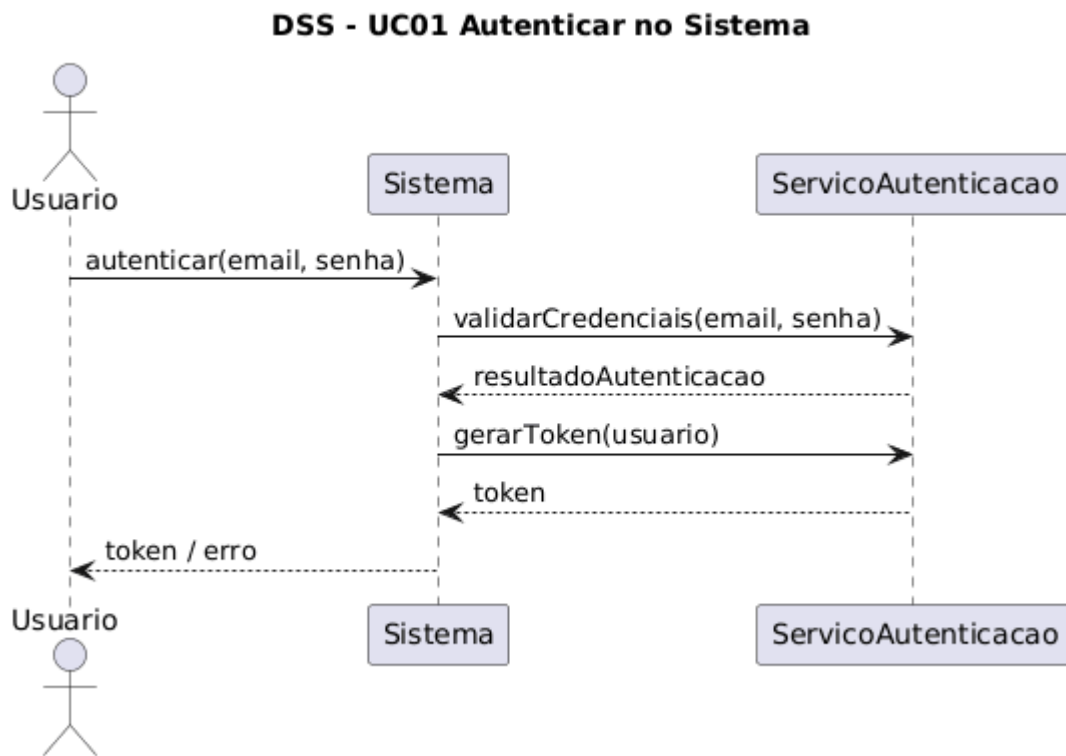


Figura 3 - DSS: Autenticar no Sistema

Operação	autenticar(email, senha)
Referências Cruzadas	UC01 – Autenticar no Sistema Classes: Usuario, PerfilAcesso
Pré-condições	Usuário cadastrado e ativo Credenciais informadas

Pós-condições	<i>Token</i> geradoÚltimo login atualizado
---------------	---

Tabela 7 - Contrato de Operação: Autenticar no Sistema

A Figura 4 demonstra o fluxo onde o usuário solicita o cadastro de uma instituição. O sistema primeiro valida autenticação e permissão específica para essa ação. Em seguida, os dados são enviados ao repositório, que persiste a instituição e devolve o objeto criado. O caso contempla os cenários de sucesso, além de falhas por campos inválidos ou ausência de permissão. O Contrato de Operação referente está disponível na Tabela 8.

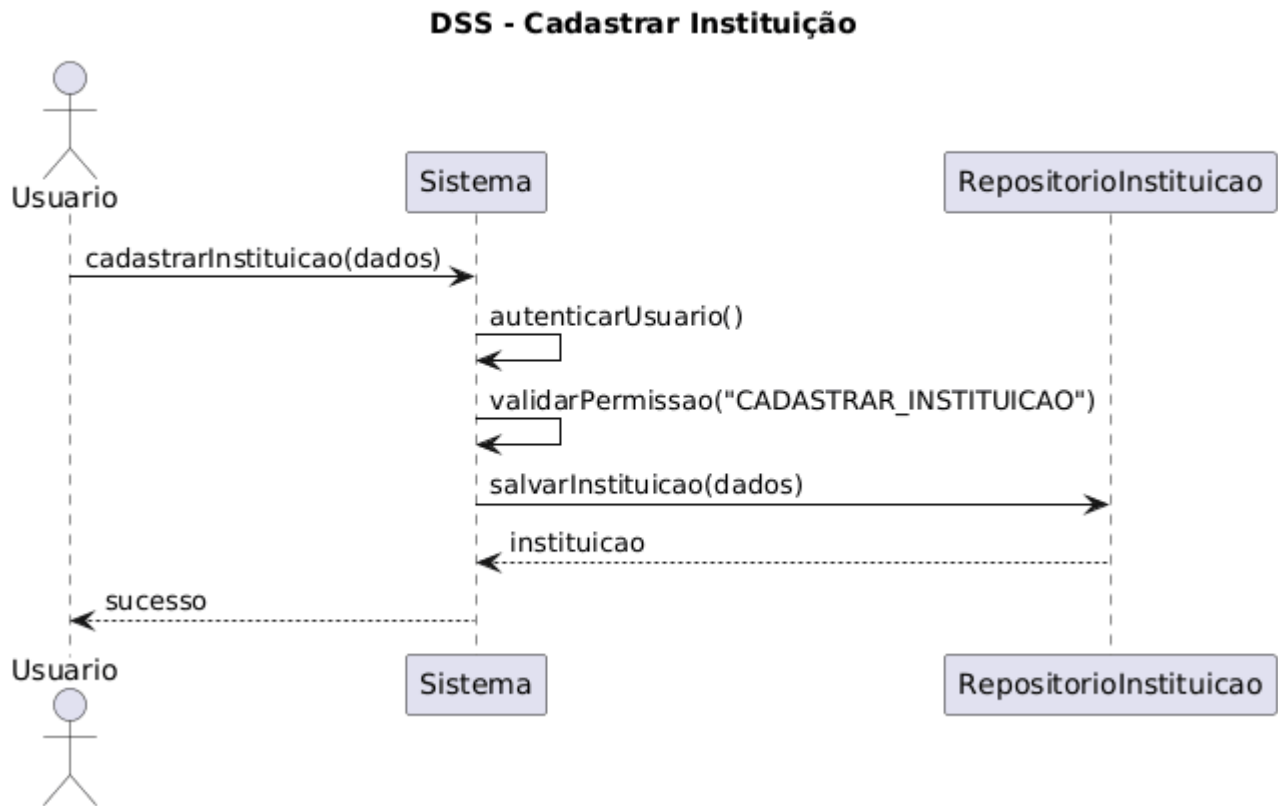


Figura 4 - DSS: Cadastrar Instituição

Operação	cadastrarInstituicao(dados)
Referências Cruzadas	UC03 – Cadastrar Instituição

Pré-condições	Usuário autenticado Permissão CADASTRAR_INSTITUICAO
Pós-condições	Instituição salva no banco

Tabela 8 - Contrato de Operação: Cadastrar Instituição

Como mostra a Figura 5 e o contrato de operação da Tabela 9, o ator submete os dados de uma nova liderança. Após validar identidade e permissão, o sistema verifica também se a instituição associada existe. Com todas as condições atendidas, a liderança é registrada e o sistema retorna os dados persistidos. O DSS evidencia o fluxo principal e possíveis falhas de validação relacionadas ao vínculo com a instituição.

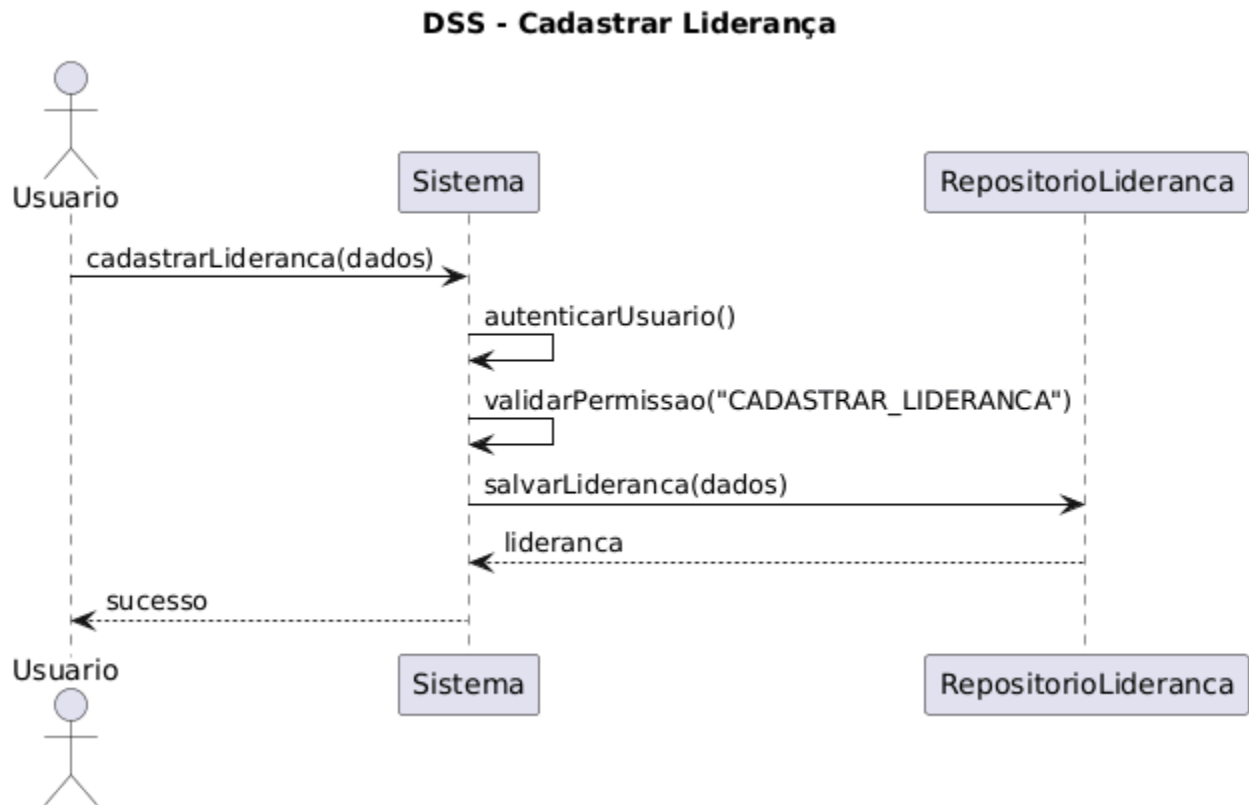


Figura 5 - DSS: Cadastrar Liderança

Operação	cadastrarLideranca(dados)
Referências Cruzadas	UC03 – Cadastrar Liderança
Pré-condições	Usuário autenticado Permissão CADASTRAR_LIDERANCA

Pós-condições	Liderança cadastrada
---------------	----------------------

Tabela 9 - Contrato de Operação: Cadastrar Liderança

A Figura 6 demonstra o cadastro de uma cidade e a Tabela 10 o contrato de operação referente. O sistema autentica o usuário, verifica permissões e persiste os dados no banco. O diagrama apresenta o fluxo de criação e o cenário onde a operação falha devido a dados incompletos ou ausência de permissão adequada.

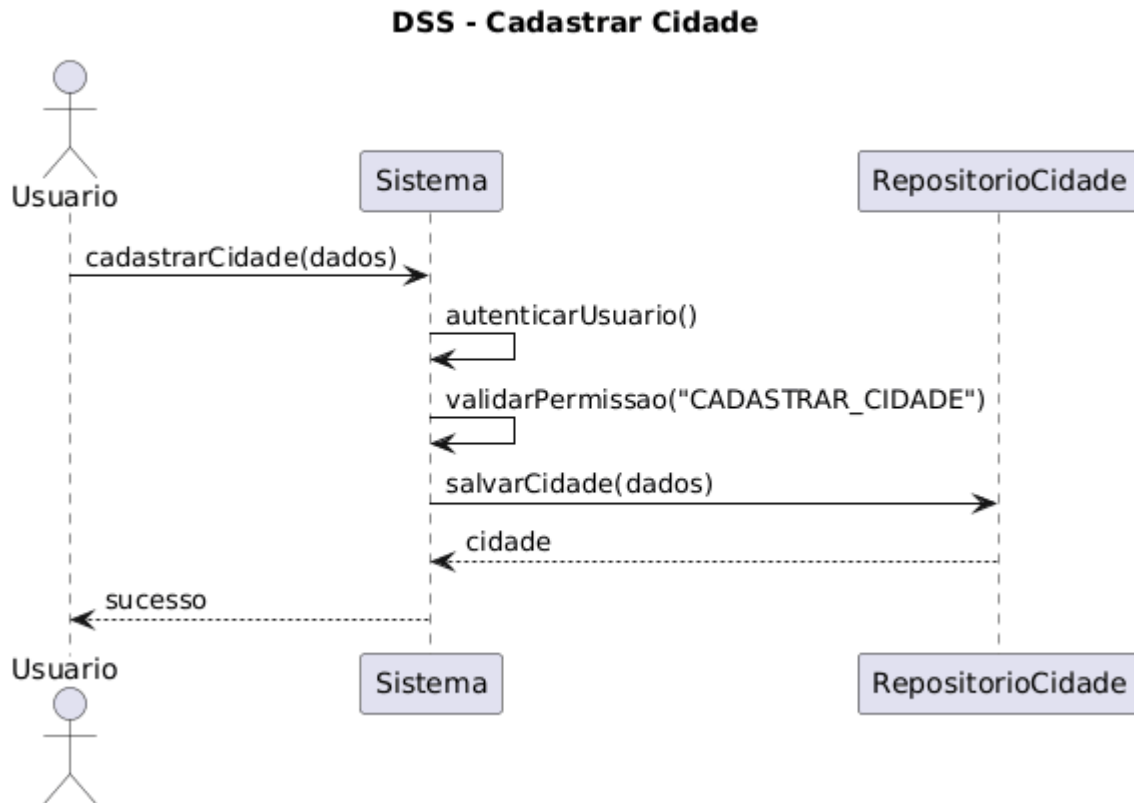


Figura 6 - DSS: Cadastrar Cidade

Operação	cadastrarCidade(dados)
Referências Cruzadas	UC03 – Cadastrar Cidade
Pré-condições	Usuário autenticado Permissão CADASTRAR_CIDADE
Pós-condições	Cidade registrada

Tabela 10 - Contrato de Operação: Cadastrar Cidade

A Figura 7 descreve como o ator registra uma notícia para publicação. O sistema autentica o usuário, verifica permissões e então persiste a notícia com autor associado. O DSS contempla o

cenário de sucesso e condições de erro, como ausência de título ou conteúdo, e a Tabela 11 contempla o contrato de operação referente a esse cenário.

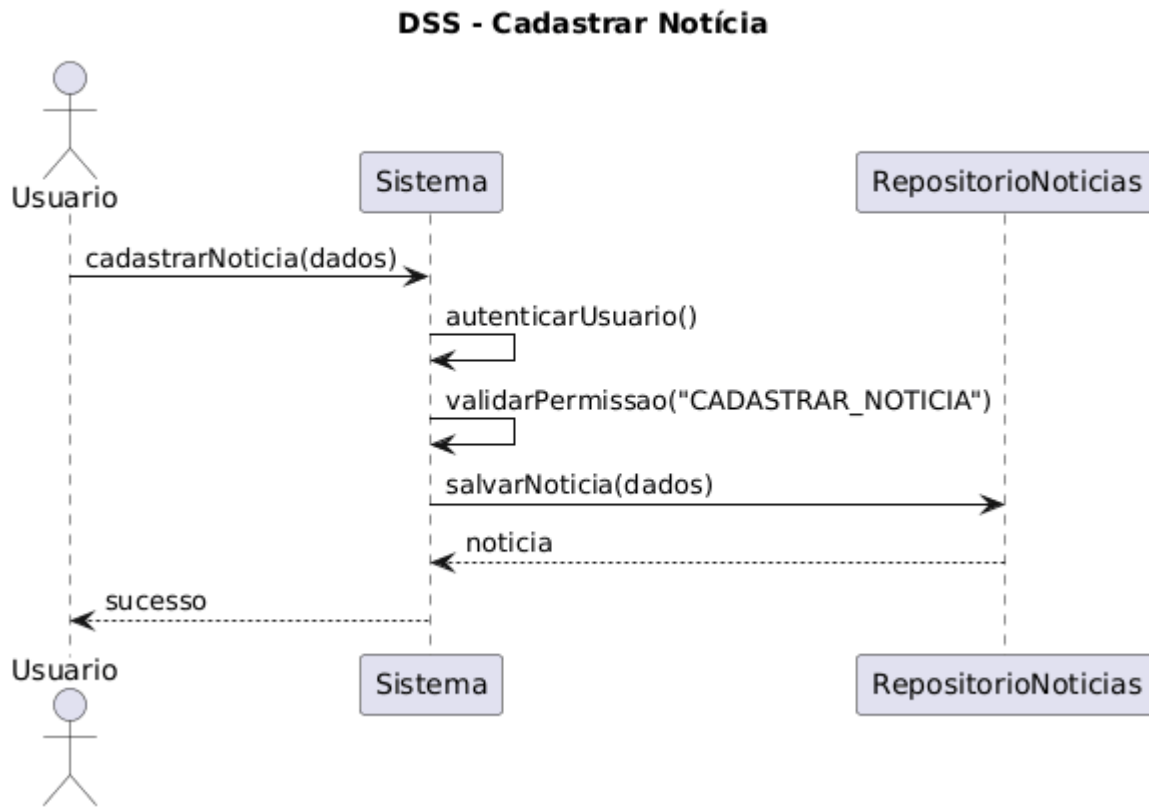


Figura 7 - DSS: Cadastrar Notícia

Operação	cadastrarNoticia(dados)
Referências Cruzadas	UC11 – Cadastrar Notícias
Pré-condições	Usuário autenticado Permissão CADASTRAR_NOTICIA
Pós-condições	Notícia criada

Tabela 11 - Contrato de Operação: Cadastrar Notícia

Como mostra a Figura 8, o usuário altera o texto da seção “Missão” do *site*. Após validação de permissão, o sistema atualiza o conteúdo no CMS. O diagrama contempla tanto o fluxo bem-sucedido quanto o caso em que o texto enviado está vazio ou a permissão não é concedida. Além disso, abaixo é possível observar a Tabela 12 que define o Contrato de Operação referente ao diagrama.

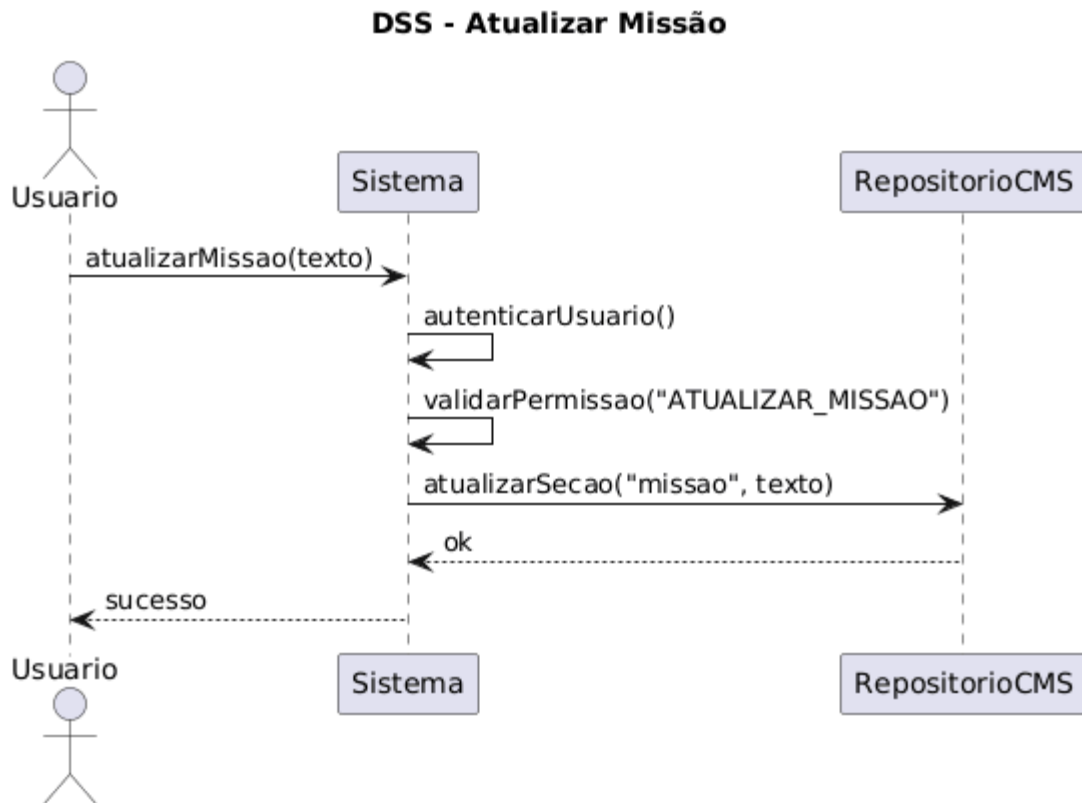


Figura 8 - DSS: Atualizar Missão

Operação	atualizarMissao(texto)
Referências Cruzadas	UC13 – Atualizar Missão
Pré-condições	Usuário autenticado Permissão ATUALIZAR_MISSAO
Pós-condições	Texto da missão atualizado

Tabela 12 - Contrato de Operação: Atualizar Missão

A Figura 9 demonstra o fluxo de atualização da seção “Trajetória”. Assim como na missão, a ação requer permissão específica. O conteúdo é sobrescrito e torna-se imediatamente disponível para o *site*. O diagrama evidencia validações e possíveis erros de conteúdo inválido.

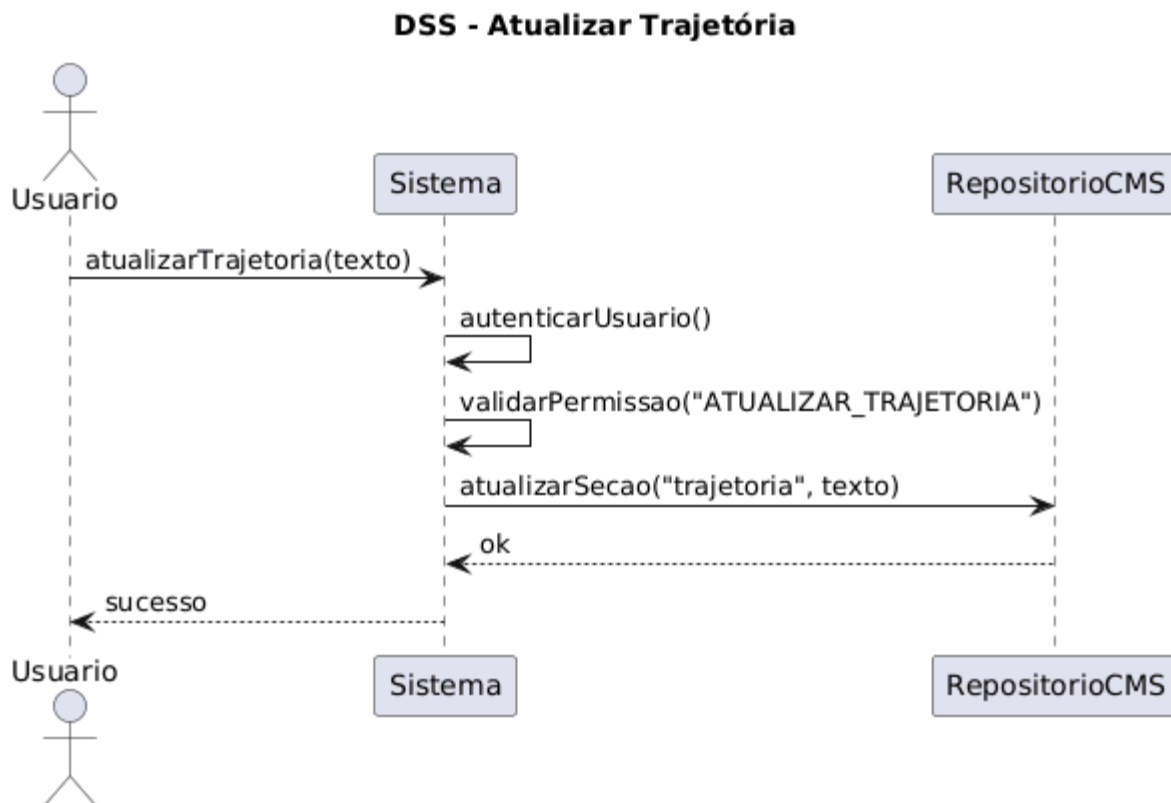
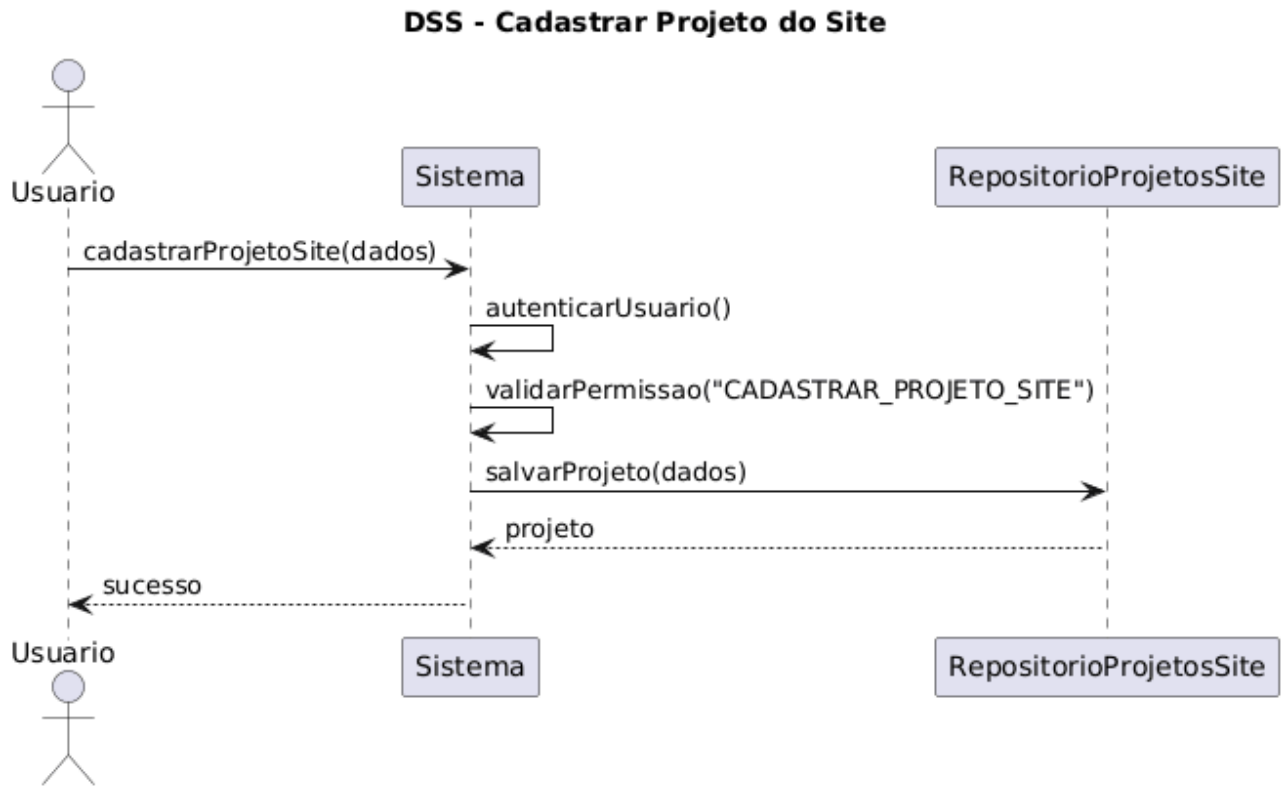


Figura 9 - DSS: Atualizar Trajetória

Operação	atualizarTrajetoria(texto)
Referências Cruzadas	UC14 – Atualizar Trajetória
Pré-condições	Usuário autenticado Permissão ATUALIZAR_TRAJETORIA
Pós-condições	Trajetória atualizada

Tabela 13 - Contrato de Operação: Atualizar Trajetória

Na Figura 10 que antecede a Tabela 14 referente ao Contrato de Operação de Cadastro de Projeto, mostra o fluxo para essa operação. O usuário solicita o cadastro de um projeto institucional para exibição no *site*. O sistema valida permissões, persiste os dados e retorna o projeto cadastrado. O DSS mostra o fluxo principal e possíveis falhas por dados incompletos.

Figura 10 - DSS: Cadastrar Projeto do *Site*

Operação	cadastrarProjetoSite(dados)
Referências Cruzadas	UC15 – Cadastrar Projetos
Pré-condições	Usuário autenticado Permissão CADASTRAR_PROJETO_SITE
Pós-condições	Projeto salvo

Tabela 14 - Contrato de Operação: Cadastrar Projeto *Site*

Como mostra a Figura 11, o usuário registra um novo Projeto de Lei. O sistema verifica autenticação, permissões e integridade dos dados (como número e descrição). Após salvamento, o PL fica disponível para consulta e relatórios. O DSS contempla erros por duplicidade ou dados inconsistentes. A Tabela 15 representa o Contrato de Operação deste diagrama.

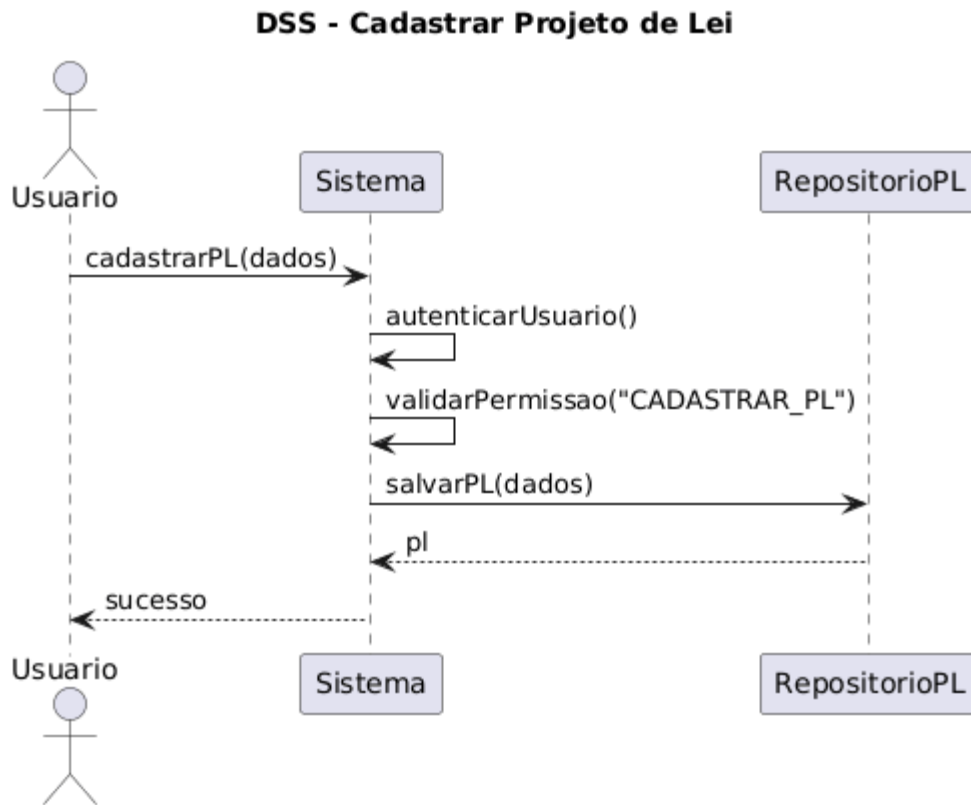


Figura 11 - DSS: Cadastrar Projeto de Lei

Operação	cadastrarPL(dados)
Referências Cruzadas	UC17 – Cadastrar PL
Pré-condições	Usuário autenticado Permissão CADASTRAR_PL
Pós-condições	PL cadastrado

Tabela 15 - Contrato de Operação: Cadastrar Projeto de Lei

A Figura 12 demonstra a alteração de dados de um de um PL já cadastrado. O sistema valida se o usuário tem permissão e se as novas informações são válidas. Em seguida, atualiza o registro. O DSS ilustra fluxo de sucesso e falhas por status inválido ou ausência de permissão. O Contrato de Operação equivalente é exibido na Tabela 16.

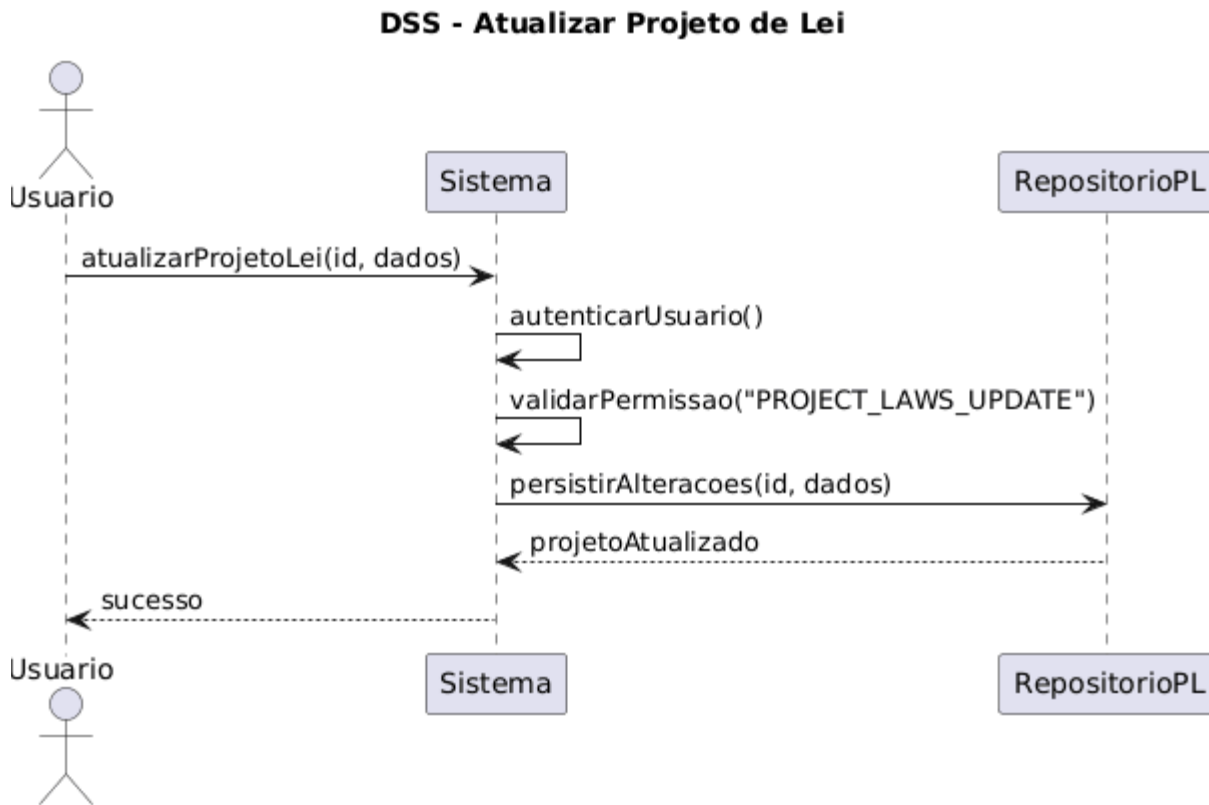


Figura 12 - DSS: Atualizar Projeto de Lei

Operação	atualizarPL(id, novosDados)
Referências Cruzadas	UC18 – Atualizar
Pré-condições	Usuário autenticado Permissão ATUALIZAR_PL
Pós-condições	Projeto de Lei atualizado

Tabela 16 - Contrato de Operação: Atualizar Projeto de Lei

Na Figura 13, o ator cadastra uma nova emenda parlamentar. Depois da validação, os dados são persistidos e relacionados ao município. O DSS mostra o fluxo positivo e rejeições por campos obrigatórios faltantes ou vínculo com cidade inexistente, a Tabela 17 exhibe o contrato de operação equivalente ao diagrama.

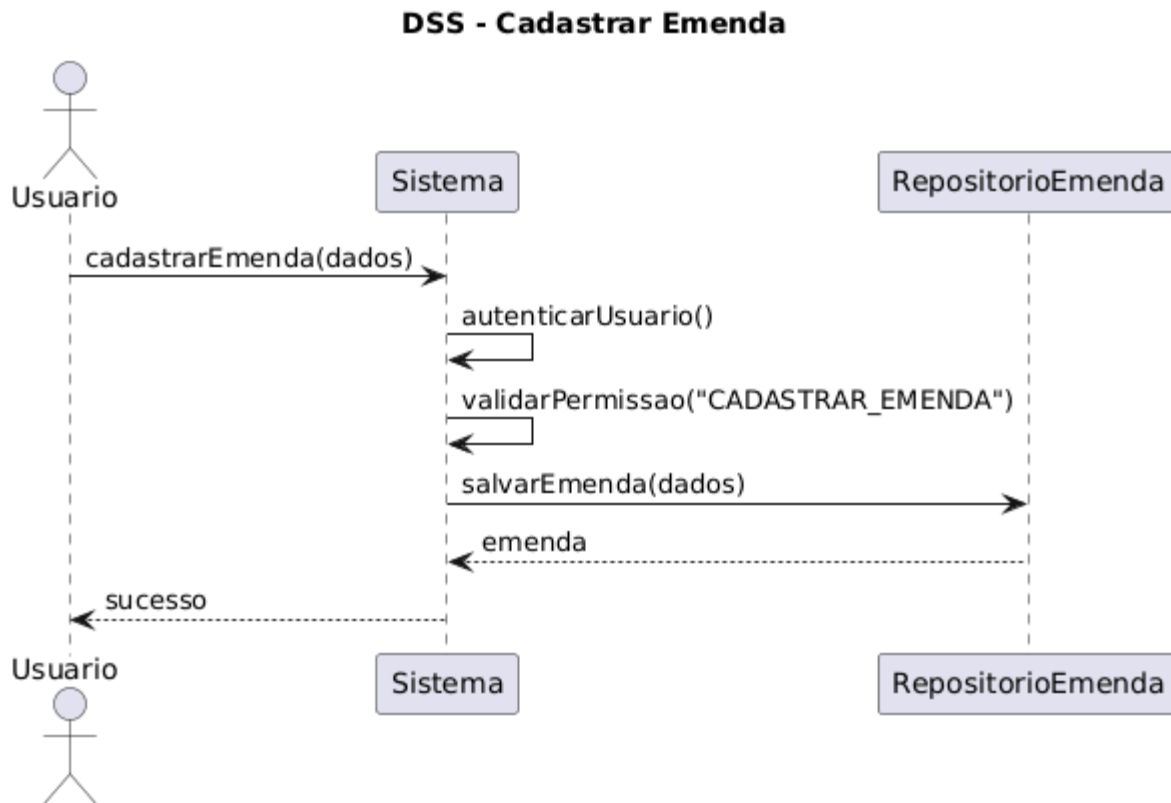


Figura 13 - DSS: Cadastrar Emenda

Operação	cadastrarEmenda(dados)
Referências Cruzadas	UC21 – Cadastrar Emenda
Pré-condições	Usuário autenticado Permissão CADASTRAR_EMENDA
Pós-condições	Emenda cadastrada

Tabela 17 - Contrato de Operação: Cadastrar Emenda

A Figura 14 apresenta a atualização de uma emenda e a Tabela 18 o contrato de operação desse fluxo. O sistema exige permissão específica, valida as informações registra a alteração. O DSS contempla fluxo normal e erro por transição inválida ou permissão insuficiente.

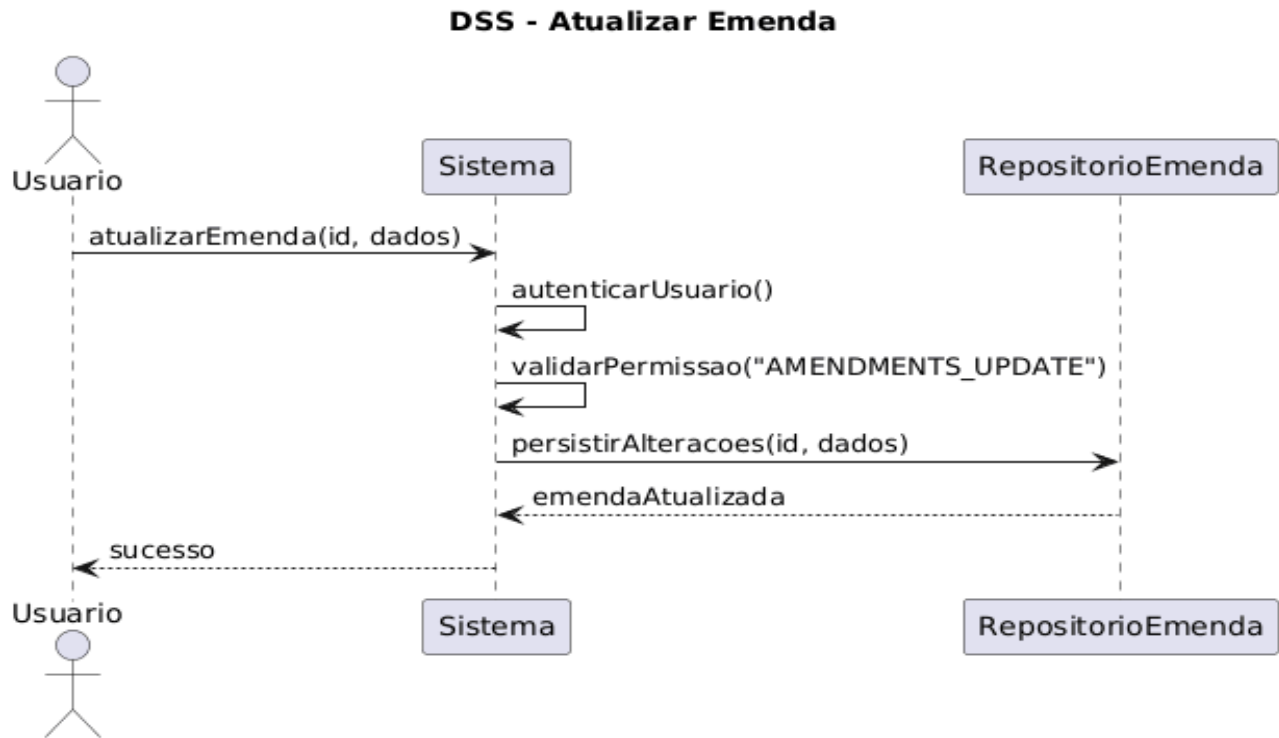


Figura 14 - DSS: Atualizar Emenda

Operação	atualizarEmenda(id, novosDados)
Referências Cruzadas	UC22 – Atualizar Emenda
Pré-condições	Usuário autenticado Permissão ATUALIZAR_EMENDA
Pós-condições	Status atualizado

Tabela 18 - Contrato de Operação: Atualizar Emenda

Como mostra a Figura 15, uma demanda é aberta pelo usuário. O sistema valida permissões, cria a demanda com status inicial e registra automaticamente um histórico de criação. O diagrama considera falhas como campos obrigatórios vazios e abaixo está a Tabela 19 referente ao Contrato de Operação deste fluxo..

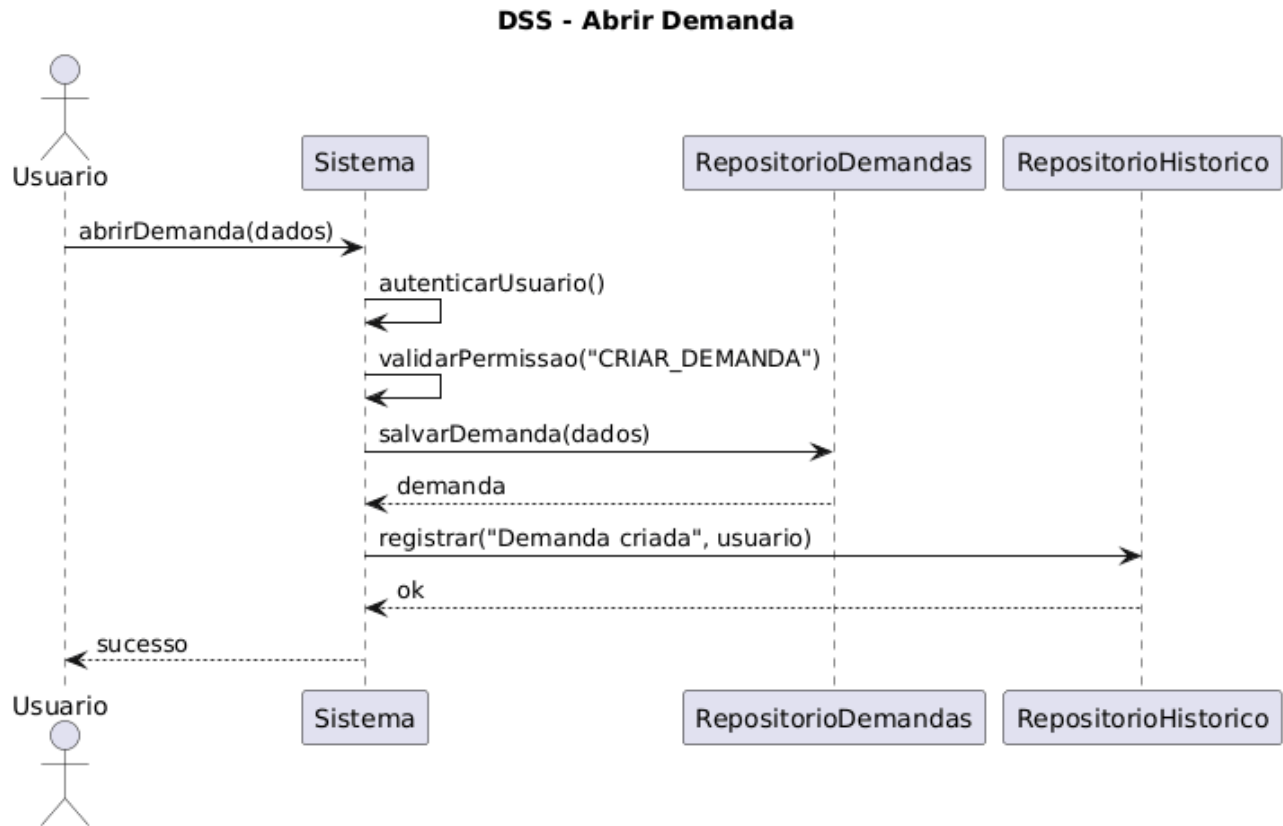


Figura 15 - DSS: Abrir Demanda

Operação	abrirDemanda(dados)
Referências Cruzadas	UC27 – Abrir Demanda
Pré-condições	Usuário autenticado Permissão CRIAR_DEMANDA
Pós-condições	Demanda criada Histórico registrado

Tabela 19 - Contrato de Operação: Abrir Demanda

A Figura 16 descreve a atualização dos dados de uma demanda. O sistema verifica permissões, atualiza a demanda e registra a mudança no histórico. O DSS mostra erros por usuário inexistente ou demanda inválida. A Tabela 20 exibe o Contrato de Operações equivalente a operação.

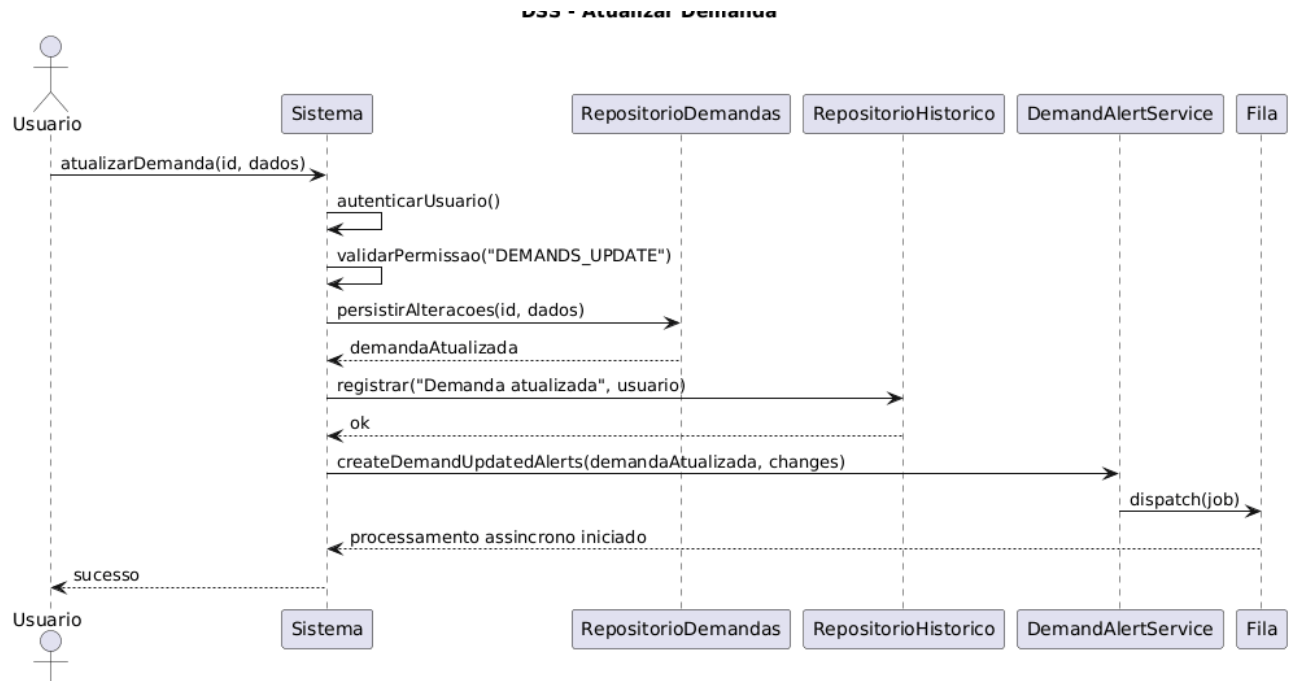


Figura 16 - DSS: Atualizar Demanda

Operação	atualizarDemanda(id, novosDados)
Referências Cruzadas	UC30 – Atualizar demanda
Pré-condições	Usuário autenticado Permissão ATUALIZAR_DEMANDA
Pós-condições	Demanda atualizada

Tabela 20 - Contrato de Operação: Atualizar Demanda

A Figura 17 demonstra o momento em que dados de uma pessoa atendida são associados a uma demanda e a Tabela 21 o contrato de operação equivalente. Após validações, a informação é gravada e registrada no histórico. O DSS contempla casos de sucesso e inconsistências nos dados da pessoa.

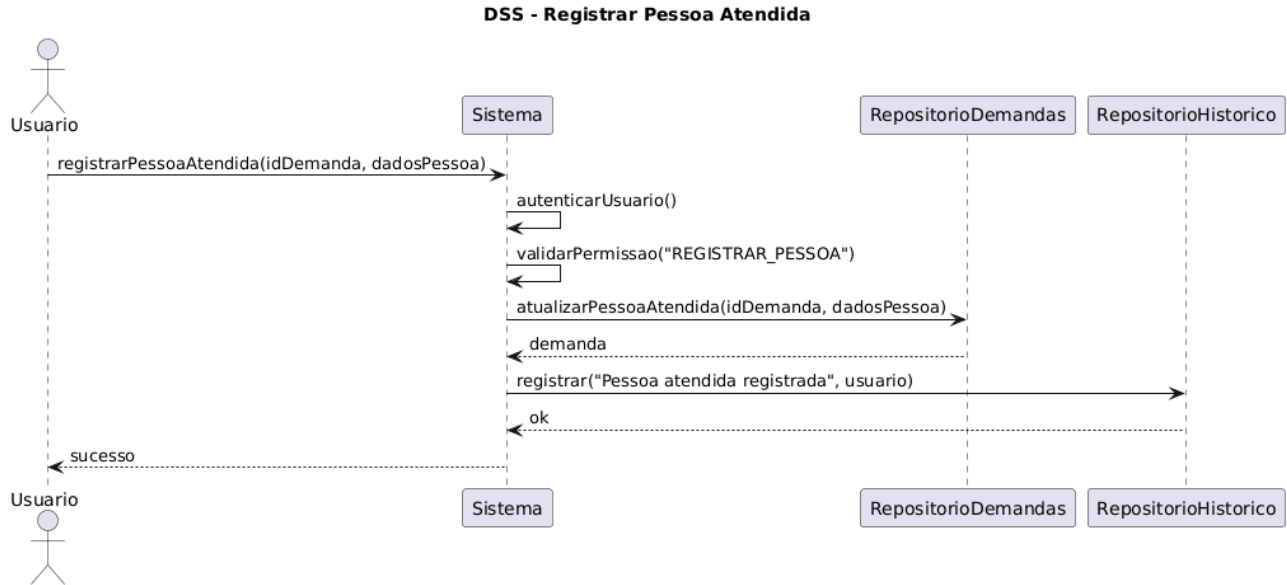


Figura 17 - DSS: Registrar Pessoa Atendida

Operação	registrarPessoaAtendida(id, dadosPessoa)
Referências Cruzadas	UC31 – Registrar Pessoa Atendida
Pré-condições	Usuário autenticado Permissão REGISTRAR_PESSOA
Pós-condições	Pessoa vinculada Histórico registrado

Tabela 21 - Contrato de Operação: Registrar Pessoa Atendida

A Figura 18 mostra o cadastro de um evento na agenda e a Tabela 22 o Contrato dessa operação. Após autenticação e validação de permissão, o sistema registra o evento e opcionalmente aciona regras de conflito de datas. O DSS prevê erros de datas inconsistentes ou permissões insuficientes.

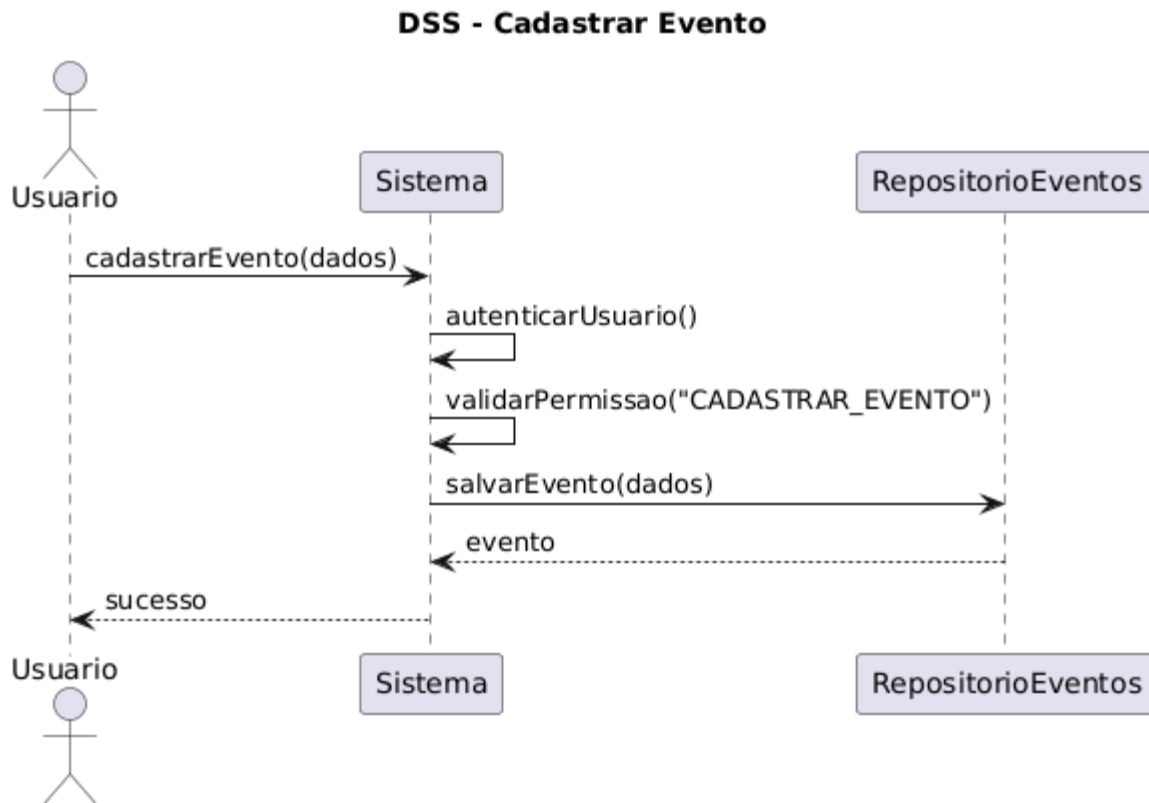


Figura 18 - DSS: Cadastrar Evento

Operação	cadastrarEvento(dados)
Referências Cruzadas	UC33 – Cadastrar Evento
Pré-condições	Usuário autenticado Permissão CADASTRAR_EVENTO
Pós-condições	Evento criado

Tabela 22 - Contrato de Operação: Cadastrar Evento

A Figura 19 descreve o fluxo de identificação e cadastro do cidadão no *Chatbot* do Gabinete Virtual, e a Tabela 23 apresenta o contrato de operação correspondente. O processo inicia-se quando o cidadão envia uma mensagem pelo WhatsApp e tem seu telefone identificado pela *Chatbot* API. Caso já exista um cidadão vinculado a esse número, o fluxo segue sem novo cadastro; caso contrário, o sistema solicita os dados necessários para complementar a identificação, registra o cidadão no *backend* e associa o telefone ao respectivo cadastro. Ao final, o *chatbot* confirma a conclusão da etapa e permite o prosseguimento para as demais funcionalidades do atendimento automatizado.

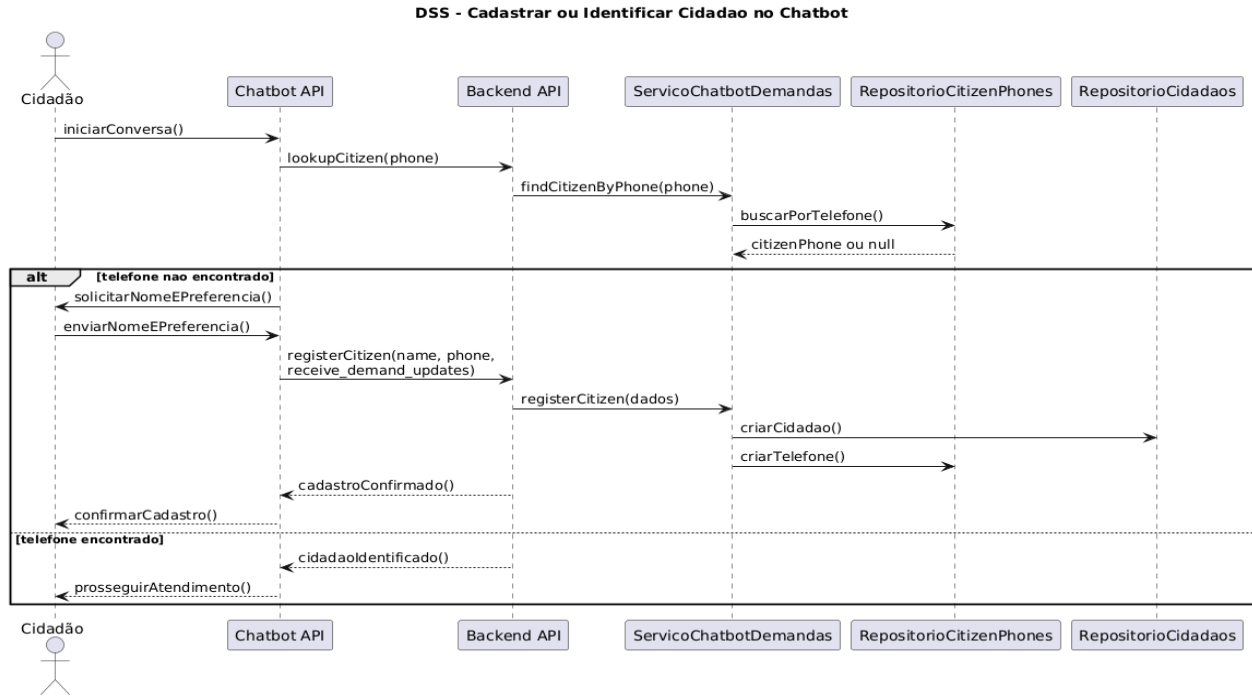


Figura 19 - DSS: Cadastrar Cidadão

Operação	cadastrarCidadao(dadosCidadao)
Referências Cruzadas	UC-CH01
Pré-condições	Cidadão não cadastrado
Pós-condições	Cidadão registrado no sistema

Tabela 23 - Contrato de Operação: Cadastrar Cidadão

O diagrama da Figura 20, em conjunto com a Tabela 24, representa o fluxo de registro de demandas por meio do *Chatbot* do Gabinete Virtual. Após identificar o cidadão, o *chatbot* coleta a cidade, apresenta as instituições disponíveis para aquele município e permite que o campo de instituição permaneça em branco quando necessário. Em seguida, a mensagem da demanda é submetida a validações de idioma, conteúdo ofensivo e similaridade com demandas recentes. Se a demanda for considerada inadequada ou duplicada, ela ainda é enviada ao *backend* como descartada, com o motivo correspondente; caso contrário, é registrada normalmente e vinculada ao cidadão identificado.

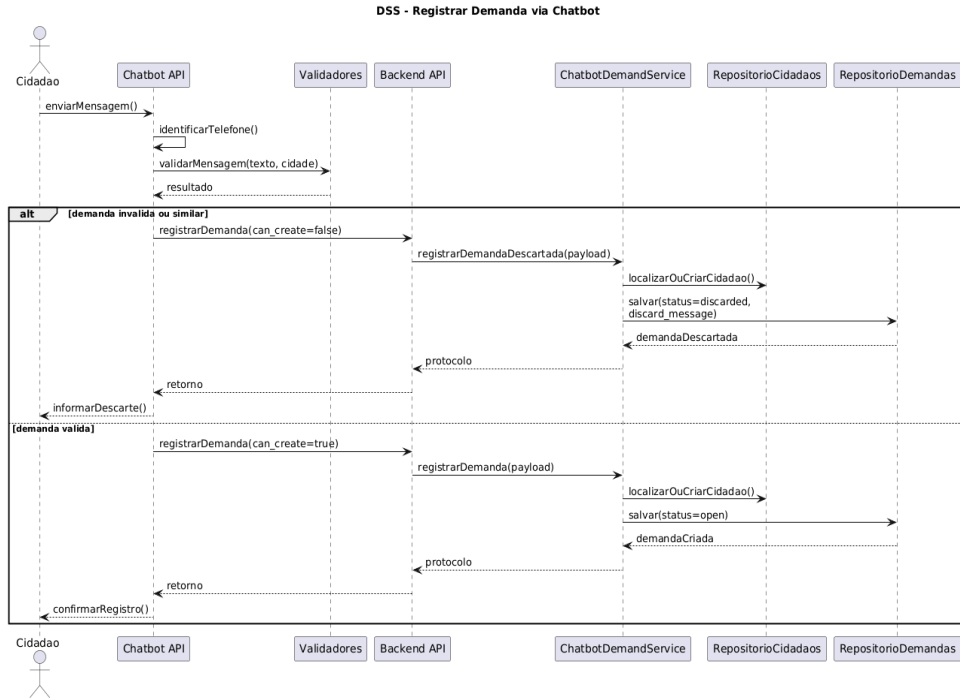


Figura 20 - DSS: Registrar Demanda

Operação	registrarDemanda(dadosDemanda)
Referências Cruzadas	UC-CH02
Pré-condições	Cidadão autenticado
Pós-condições	Demanda registrada com status inicial

Tabela 24 - Contrato de Operação: Registrar Demanda

A Figura 21 descreve o fluxo de notificações automáticas enviadas ao cidadão sempre que ocorre uma atualização relevante em uma demanda vinculada ao seu cadastro, e a Tabela 25 apresenta o contrato de operação correspondente. Diferentemente dos fluxos iniciados pelo usuário, esse processo é disparado pelo Sistema Gabinete Virtual quando alterações como atualização de dados, mudança de responsável, ajuste de prazo ou evolução do atendimento geram um alerta. O *backend* registra a atualização, aciona o *chatbot* e a mensagem é encaminhada ao cidadão pelo WhatsApp quando ele optou por receber avisos.

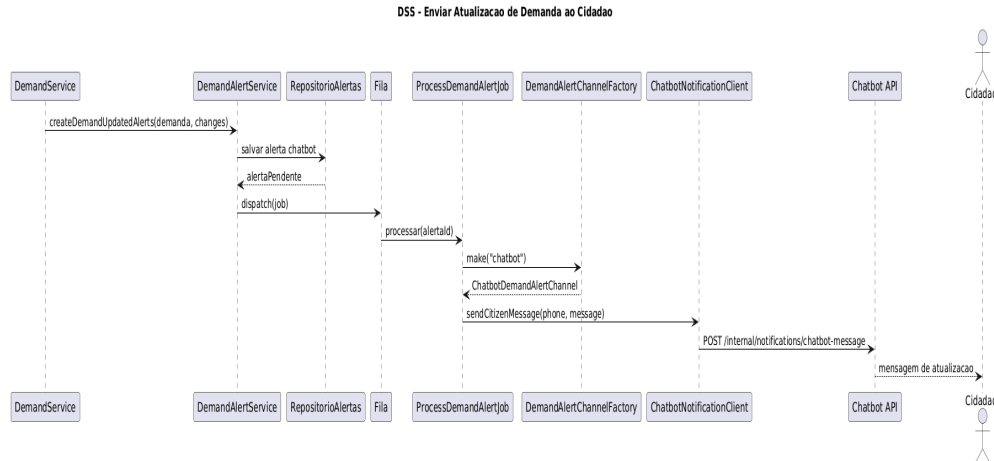


Figura 21 - DSS: Receber Atualização

Operação	notificarAtualizacao(statusDemanda)
Referências Cruzadas	UC-CH04
Pré-condições	Demanda existe
Pós-condições	Cidadão Notificado

Tabela 25 - Contrato de Operação: Receber Atualização

O Diagrama de Sequência do Sistema referente à opção de recebimento de conteúdos descreve o fluxo no qual o cidadão manifesta interesse em receber informações institucionais por meio do *chatbot*. O processo inicia-se quando o cidadão seleciona a opção de ativar o recebimento de conteúdos. O *chatbot* encaminha essa preferência ao Sistema Gabinete Virtual, que registra a escolha associada ao cadastro do cidadão. Após a atualização das preferências, o sistema retorna a confirmação ao *chatbot*, que informa o cidadão sobre a ativação da funcionalidade. Esse fluxo está vinculado ao caso de uso UC-CH05 e caracteriza uma funcionalidade opcional do sistema. O Diagrama e Contrato de Operações desse fluxo estão representados pela Figura 22 e Tabela 26 respectivamente.

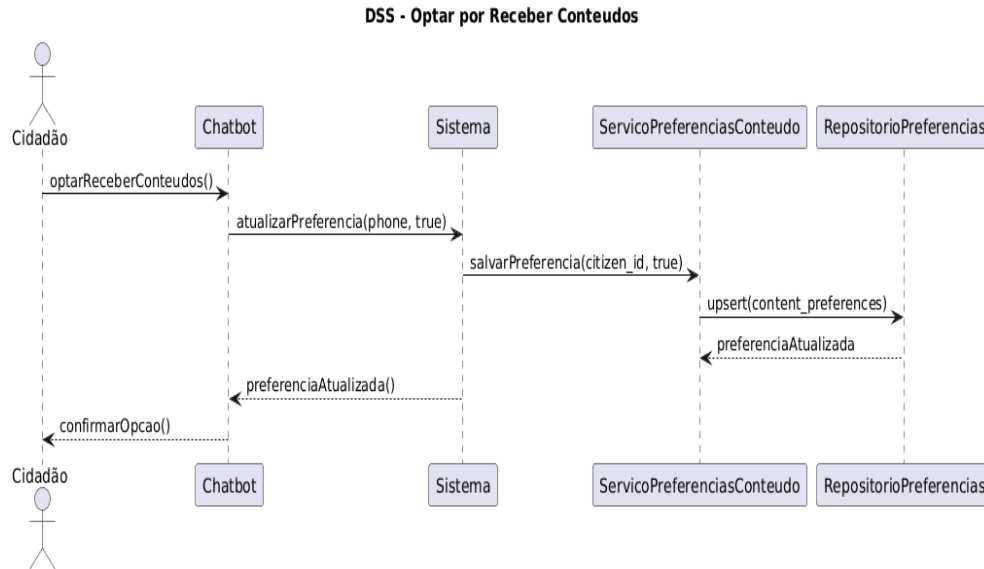


Figura 22 - DSS: Optar por Receber Conteúdos

Operação	atualizarPreferencias(cpf, receberConteudos)
Referências Cruzadas	UC-CH05
Pré-condições	Cidadão cadastrado
Pós-condições	Preferência salva

Tabela 26 - Contrato de Operação: Optar por Receber Conteúdos

3. Modelos de Projeto

Os modelos de projeto detalham a estrutura lógica, arquitetural e funcional do sistema, servindo como base para a implementação e documentação técnica. Nesta seção, são apresentados diagramas UML e C4 que descrevem o comportamento interno do sistema, suas camadas, componentes, fluxos de informação e infraestrutura de implantação.

3.1 Diagrama de Classes

A Figura 23 apresenta o diagrama de classes do sistema Gabinete Virtual, organizado em pacotes para evidenciar a separação lógica entre os diferentes domínios da aplicação. Essa organização permite visualizar com maior clareza como os módulos administrativos, parlamentares, de atendimento e de comunicação com o cidadão se relacionam dentro da solução.

No pacote Autorização estão as classes PerfilAcesso, Permissao e Usuario, que sustentam o controle de acesso do sistema. Nesse núcleo, o usuário autenticado é associado a um perfil e a permissões específicas, permitindo que as regras de autorização sejam verificadas de acordo com o recurso acessado. Esse conjunto é fundamental para garantir que diferentes perfis internos do gabinete possuam acessos compatíveis com suas responsabilidades.

O pacote Referências reúne classes de apoio compartilhadas por vários módulos, como Cidade, Instituicao e Lideranca. Essas entidades representam dados estruturais do sistema e servem de base para demandas, eventos, emendas e projetos exibidos no *site*, mantendo coerência territorial e administrativa entre os módulos.

Uma das principais evoluções do modelo está no pacote *Chatbot* e Cidadãos, que inclui `Cidadao``, `TelefoneCidadao`, `SessaoChatbot` e `PreferenciaConteudo`. Esse conjunto reflete o fluxo mais recente do sistema, no qual o cidadão pode ser identificado pelo telefone, possuir múltiplos números associados, manter sessões de atendimento via *chatbot* e registrar preferências relacionadas ao recebimento de conteúdos. A modelagem mostra que o *chatbot* deixou de ser apenas uma integração externa e passou a fazer parte efetiva do domínio da aplicação.

No pacote Demandas encontra-se a classe Demanda, que representa o ciclo completo de atendimento do gabinete. Além dos atributos tradicionais, como título, descrição, status e prioridade, a classe também contempla área de atendimento, mensagem de descarte e metadados de ofício, refletindo o comportamento atual da aplicação. Associadas a ela estão `HistoricoDemanda`, responsável pelo registro das alterações realizadas, e `AlertaDemanda`, que modela os alertas gerados para usuários internos ou para cidadãos vinculados à demanda. Esse conjunto reforça a rastreabilidade e a comunicação automática derivadas das alterações efetuadas no sistema.

O pacote Agenda reúne `Evento` e `AlertaEvento`. A classe `Evento` representa compromissos cadastrados no sistema, incluindo informações como data, período, local, descrição e cidade relacionada. Já `AlertaEvento` modela os lembretes associados a esses compromissos, permitindo representar os avisos internos disparados pelo sistema para acompanhamento da agenda. Essa estrutura substitui a visão anterior de alertas como uma funcionalidade isolada e passa a tratá-los como consequência natural da gestão de eventos.

No pacote Parlamentar aparecem as classes Emenda e ProjetoLei, que representam os módulos de acompanhamento legislativo e administrativo do gabinete. Essas entidades concentram informações específicas do domínio parlamentar e utilizam *enums* próprios para manter padronização dos estados válidos de cada fluxo.

O pacote CMS contempla `SecaoCms`, `Noticia` e `ProjetoSite`, responsáveis pelo conteúdo institucional e público exibido pela plataforma. Essas classes modelam tanto o conteúdo fixo do *site* quanto publicações e projetos divulgados, sempre vinculados à autoria de um usuário interno.

Por fim, o diagrama também destaca os enums `StatusDemanda`, `Prioridade`, `AreaAtendimento`, `CanalAlertaDemanda`, `CanalAlertaEvento`, `StatusEnvioAlerta`, `StatusEmenda`, `StatusProjetoLei` e `StatusProjetoSite`. Esses tipos enumerados padronizam valores relevantes do domínio e reforçam a consistência das regras de negócio implementadas na aplicação.

Dessa forma, o diagrama de classes sintetiza a estrutura conceitual atual do sistema, mostrando não apenas os módulos centrais do gabinete, mas também a evolução do projeto para suportar *chatbot*, alertas em tempo real, histórico de alterações, descarte automatizado de demandas e integração entre funcionalidades administrativas e atendimento ao cidadão.

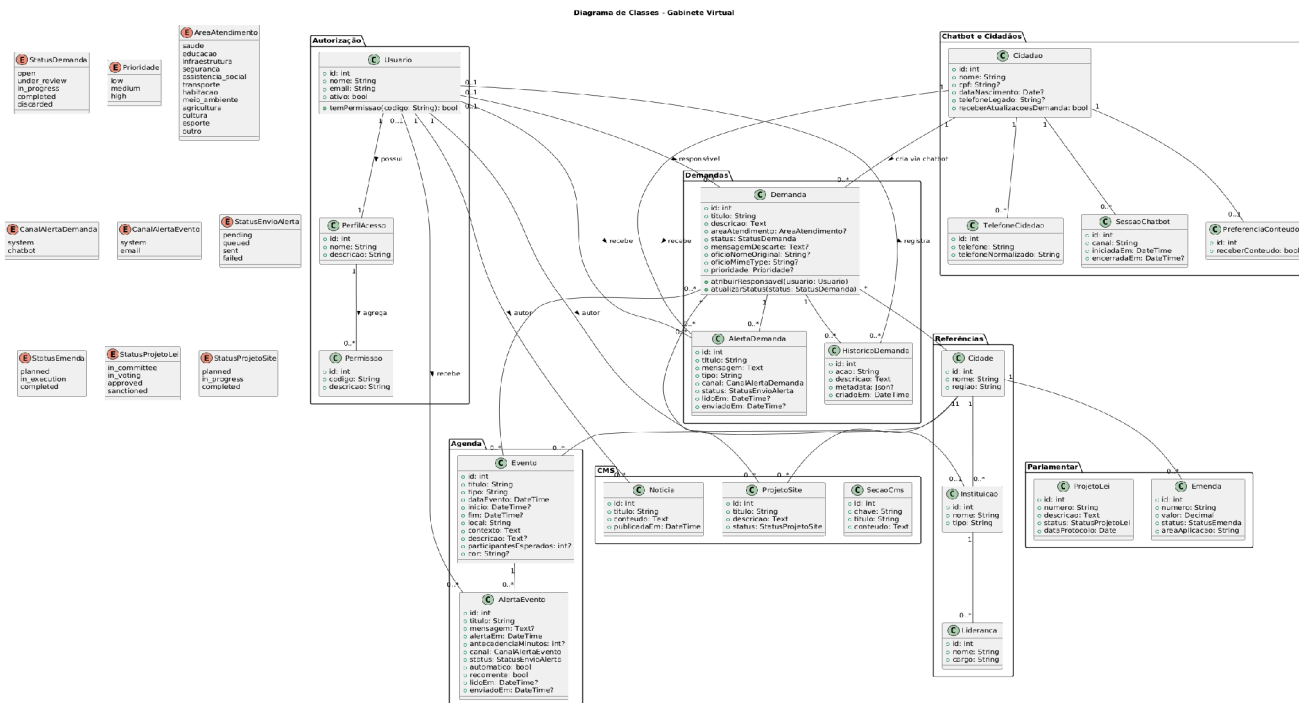


Figura 23 - Diagrama de Classes do Sistema Gabinete Virtual

A Figura 24 apresenta o diagrama de classes específico do módulo de *chatbot* do Gabinete Virtual, destacando as entidades diretamente relacionadas ao atendimento automatizado do cidadão e à comunicação realizada por meio desse canal.

No pacote *Chatbot* estão concentradas as classes principais desse contexto. A classe *Cidadao* representa o usuário externo atendido pelo sistema e armazena informações essenciais para sua identificação e acompanhamento, como nome, dados opcionais de cadastro e a preferência por receber atualizações sobre demandas. Associada a ela está a classe *TelefoneCidadao*, responsável por modelar os números de telefone vinculados ao cidadão, permitindo que a identificação seja feita prioritariamente por esse meio, o que é coerente com o funcionamento do atendimento via WhatsApp.

A classe *SessaoChatbot* representa o contexto de conversa mantido entre o cidadão e o *chatbot*, registrando o canal utilizado e os instantes de início e encerramento da interação. Essa modelagem permite compreender o *chatbot* como um mecanismo de atendimento contínuo, e não apenas como um disparador isolado de mensagens. Também nesse pacote está a classe *PreferenciaConteudo*, que registra a escolha do cidadão quanto ao recebimento de conteúdos institucionais, como notícias e projetos, preservando a decisão do usuário sobre esse tipo de comunicação.

No pacote *Demandas* aparecem as classes *Demanda* e *AlertaDemanda*, que se relacionam diretamente com o funcionamento do *chatbot*. A classe *Demanda* representa as solicitações registradas pelo cidadão, inclusive aquelas criadas por meio do atendimento automatizado. Já *AlertaDemanda* modela as mensagens geradas a partir das alterações nessas demandas, permitindo distinguir o envio por canal interno do envio realizado ao cidadão por meio do *chatbot*. Dessa forma, o diagrama evidencia que o *chatbot* participa não apenas da entrada da demanda, mas também do fluxo posterior de atualização e retorno ao usuário.

O relacionamento entre *Cidadao*, *Demanda* e *AlertaDemanda* reforça que o *chatbot* atua como uma interface de comunicação integrada ao domínio do sistema. O cidadão pode abrir demandas, receber alertas e manter sessões de atendimento sem que seja necessária a duplicação das regras centrais do negócio. Em vez disso, o módulo reutiliza entidades já existentes e se conecta ao sistema principal de maneira consistente.

Assim, a Figura 24 demonstra que o *chatbot* do Gabinete Virtual funciona como um canal especializado de atendimento e comunicação, apoiado em entidades próprias para identificação,

sessão e preferências, mas profundamente integrado aos módulos centrais de demandas e notificações. Essa modelagem reforça a coerência do sistema e evidencia a evolução da solução para um atendimento automatizado mais completo e aderente ao funcionamento real da aplicação.

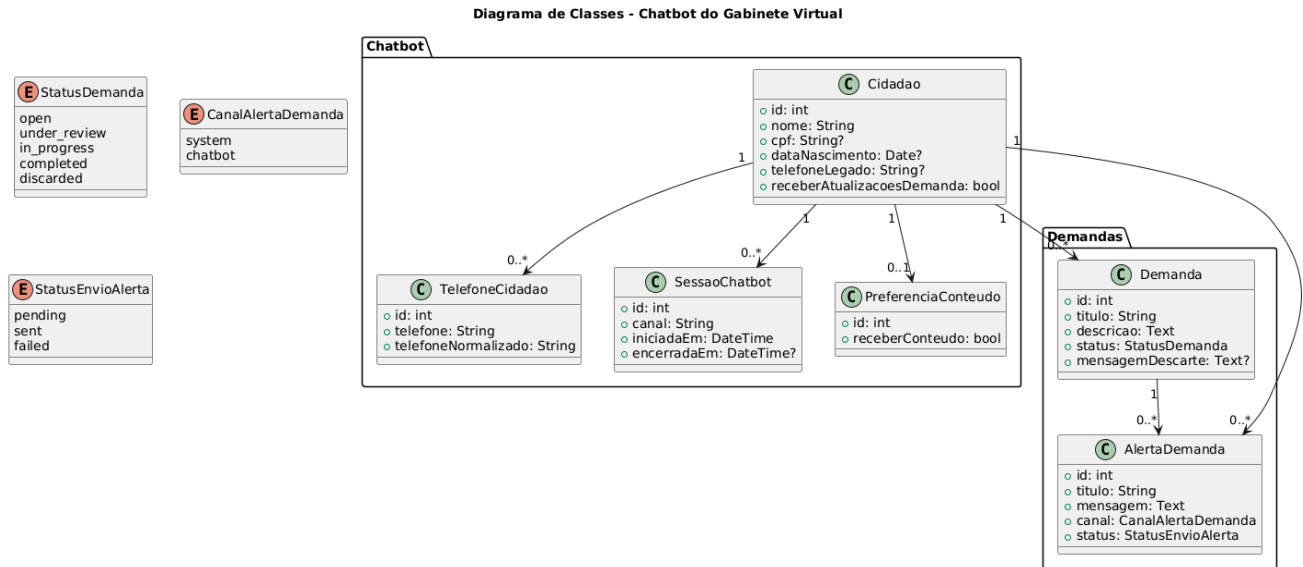


Figura 24 - Diagrama de Classes do *Chatbot* do Gabinete Virtual

3.2 Diagramas de Sequência

A presente subseção descreve os Diagramas de Sequência elaborados para cada uma das operações do sistema no contexto do projeto Gabinete Virtual. Esses diagramas têm por objetivo detalhar, de forma clara e sistemática, o fluxo de mensagens trocadas entre as diferentes camadas da aplicação: *Frontend* (React), Controladores (API Laravel), Serviços, Repositórios/Modelos, além de serviços auxiliares, como validações, autenticação, registro de histórico e armazenamento de arquivos.

O diagrama da Figura 25 representa o fluxo do UC01 - Autenticar no Sistema + Validar Permissão, descrevendo como o usuário realiza login e como o sistema valida credenciais, perfil e permissões iniciais. O processo inicia no *Frontend* (React) quando o usuário submete seu e-mail e senha. O AuthController recebe a requisição e repassa para o AuthService, que valida credenciais e consulta o registro do usuário no modelo Usuario. Caso a senha esteja incorreta ou o usuário esteja inativo, o sistema retorna erro ao *Frontend*.

Após autenticar, o AuthService executa uma validação de permissões iniciais por meio do AuthorizationService, garantindo que o usuário tenha um perfil válido (como Deputada, Chefe de Gabinete, Gestora de Demandas etc.). Em caso de sucesso, o serviço gera o *token* JWT, envia ao controller e, por fim, retorna ao *frontend* para iniciar a sessão.

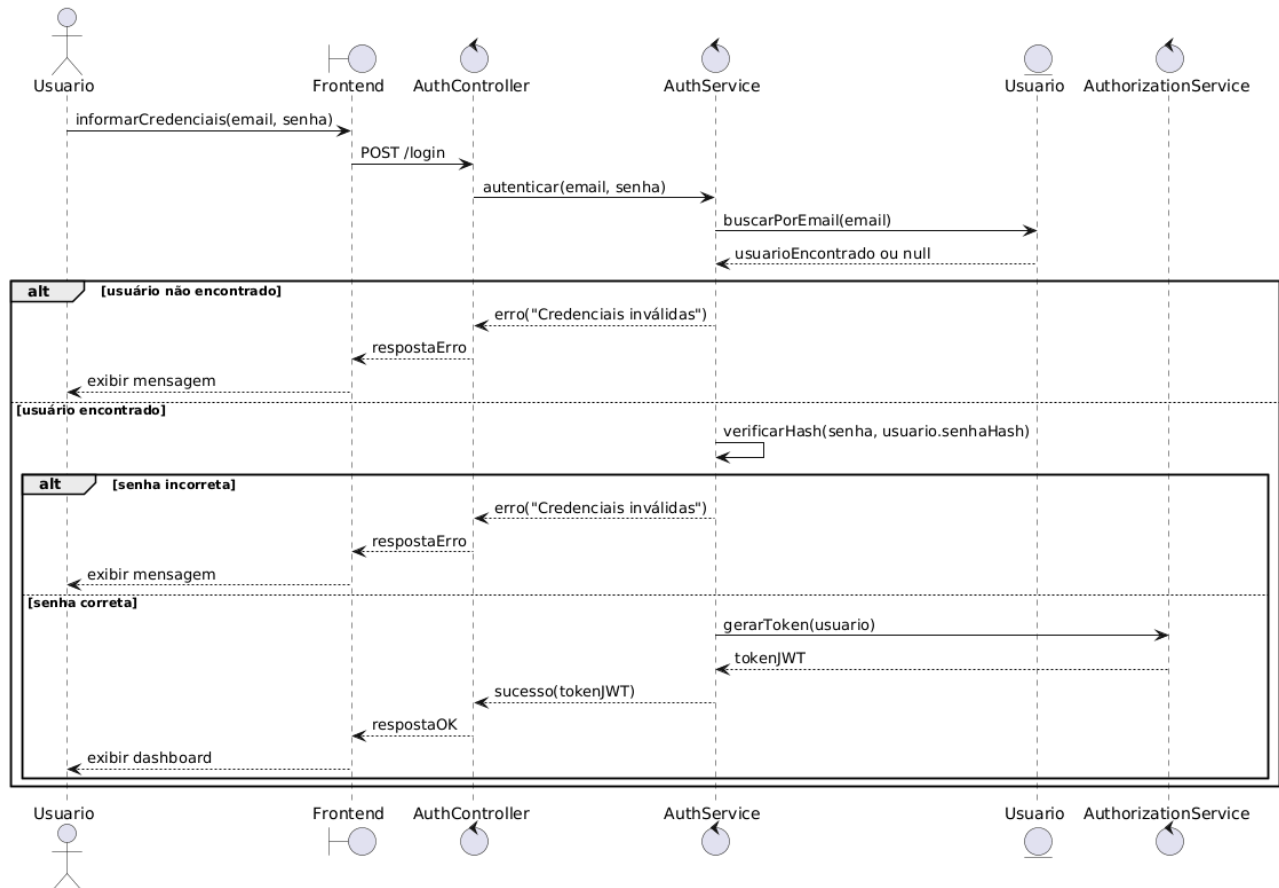


Figura 25 - Diagrama de Sequência do Sistema: Autenticar no Sistema

A Figura 26 representa um dos fluxos correspondentes ao UC02 – Gerenciar Cadastros, descrevendo como o usuário preenche dados de uma instituição no *Frontend* e como o *backend* processa, valida e persiste essas informações. O *InstituicaoController* recebe a requisição e chama o *AuthorizationService* para garantir que o perfil do usuário pode executar o cadastro.

O *ValidationService* valida campos, como nome, tipo e município. Em seguida, o *InstituicaoService* verifica previamente se o município informado existe, consultando o modelo *Municipio*. Caso esteja

tudo correto, cria-se a nova instituição via modelo *Instituicao*, registra-se o histórico com o *HistoryService*, e a operação é finalizada com resposta de sucesso ao *Frontend*.

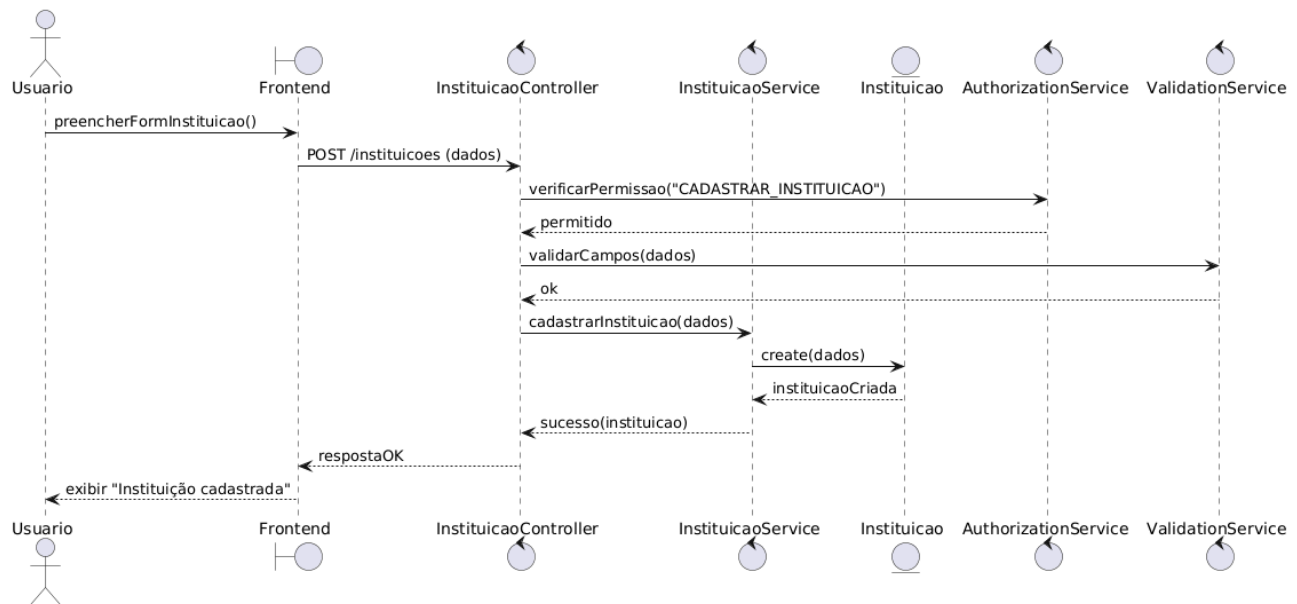


Figura 26 – Diagrama de Sequência: Cadastrar Instituição

A Figura 27 apresenta o fluxo de consulta de instituições, uma operação prevista dentro do caso de uso UC02. O usuário solicita a listagem através do *Frontend*, e o *InstituicaoController* valida permissões antes de acionar o service. O *InstituicaoService* aplica filtros recebidos e consulta o modelo *Instituicao*, retornando o conjunto de resultados ao controller, que o repassa ao usuário. Como é uma operação de leitura, não há registro de histórico.

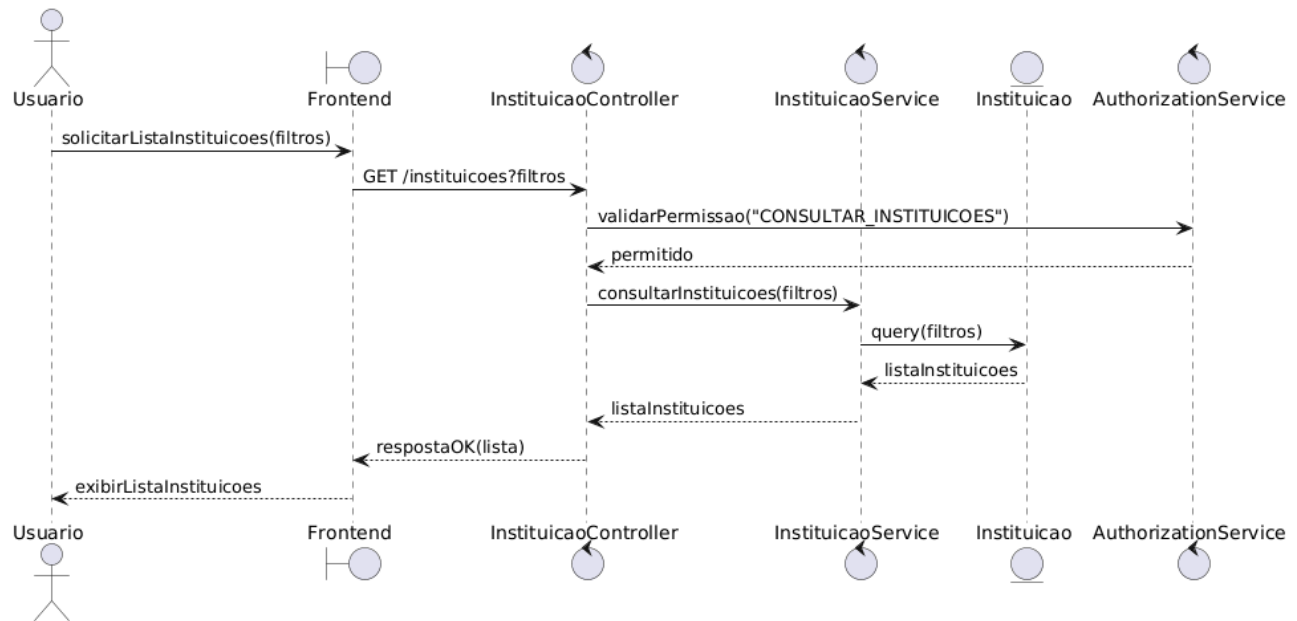


Figura 27 – Diagrama de Sequência: Consultar Instituição

Na Figura 28, é representado o fluxo de criação de uma liderança, conforme o caso de uso UC02. Após o envio dos dados pelo *Frontend*, o controller realiza a verificação de permissão e aciona o *ValidationService*. O *LiderancaService* valida a existência da instituição vinculada e, caso não haja inconsistências, cria o registro no modelo *Lideranca*. O *HistoryService* registra o evento e a resposta de sucesso é enviada ao usuário.

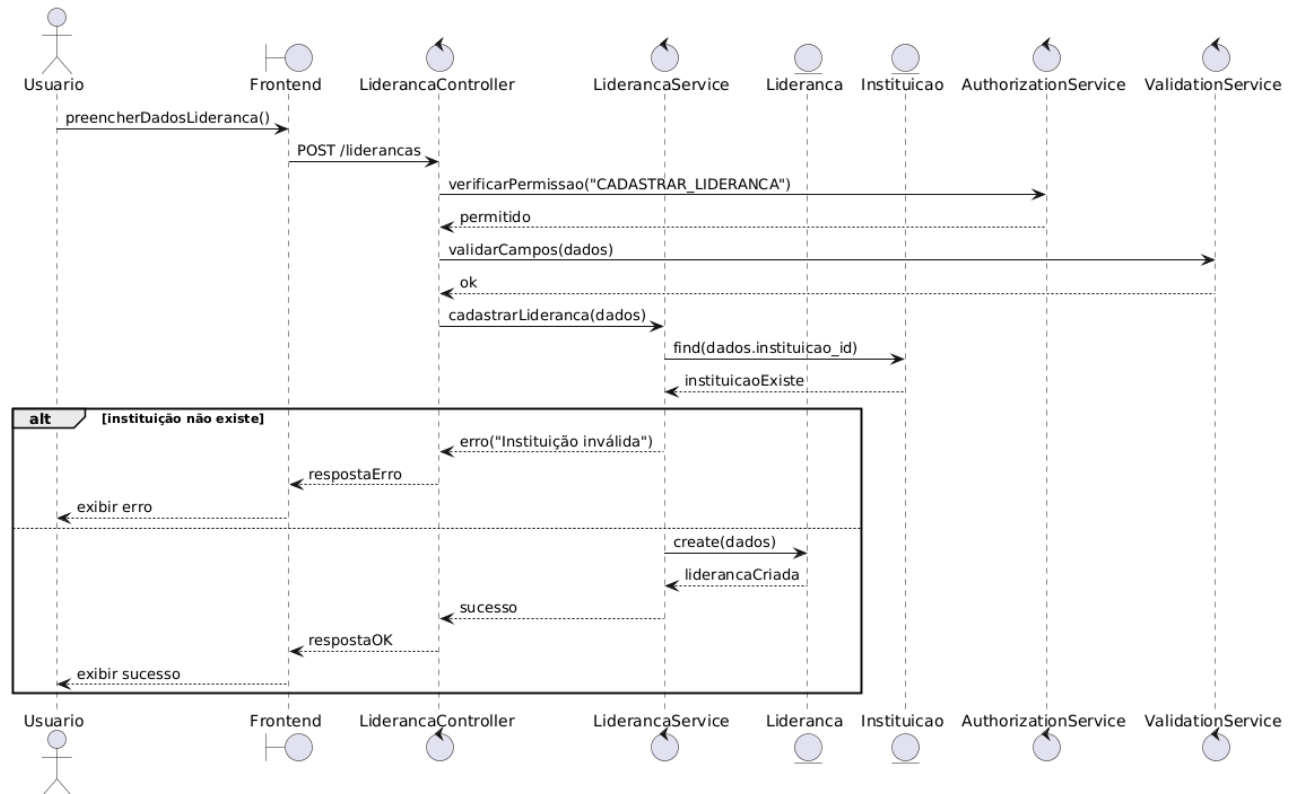


Figura 28 – Diagrama de Sequência: Cadastrar Lideranças

Como mostra a Figura 29, o diagrama de sequência de consulta de lideranças descreve o fluxo previsto no UC02. O *Frontend* solicita a listagem ao controller, que primeiro valida permissões e então aciona o service. O *LiderancaService* monta a consulta com base nos filtros informados e retorna os resultados ao controller, que envia a resposta ao cliente.

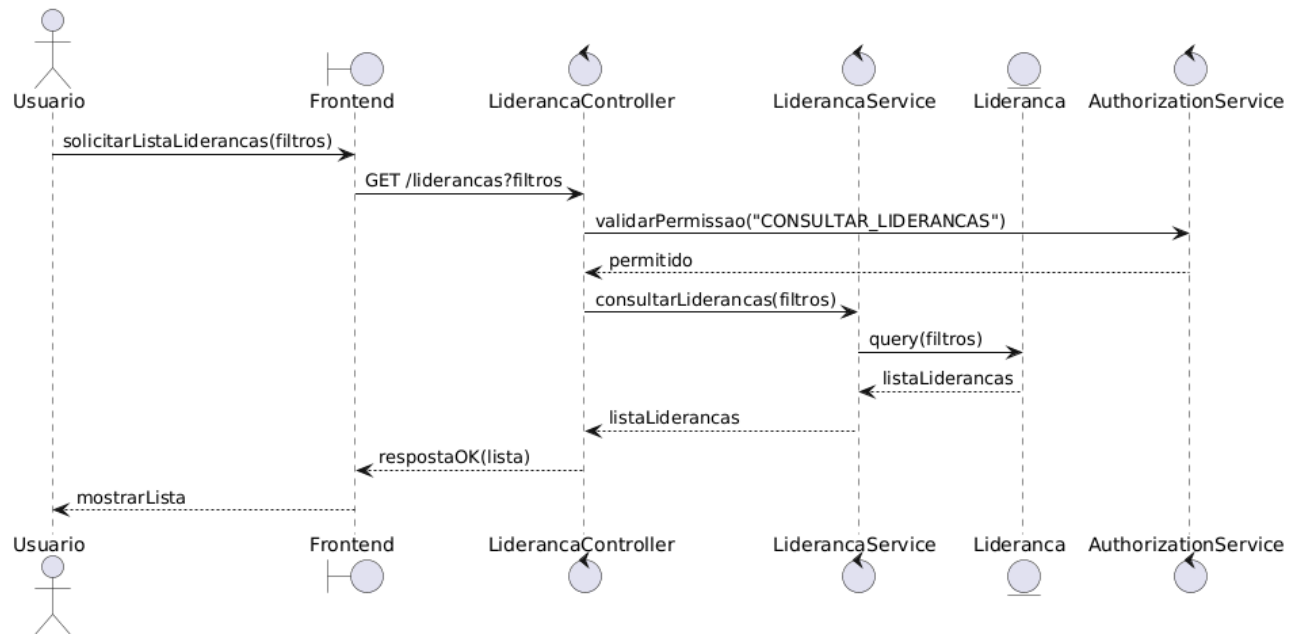


Figura 29 – Diagrama de Sequência: Consultar Lideranças

Na Figura 30 observa-se o fluxo associado ao UC02, onde o usuário requisita o cadastro de uma cidade. Após validação de permissão e integridade dos dados, o MunicipioService cria o novo registro no modelo Municipio e registra a ação no HistoryService. O sistema responde ao *Frontend* confirmando a criação.

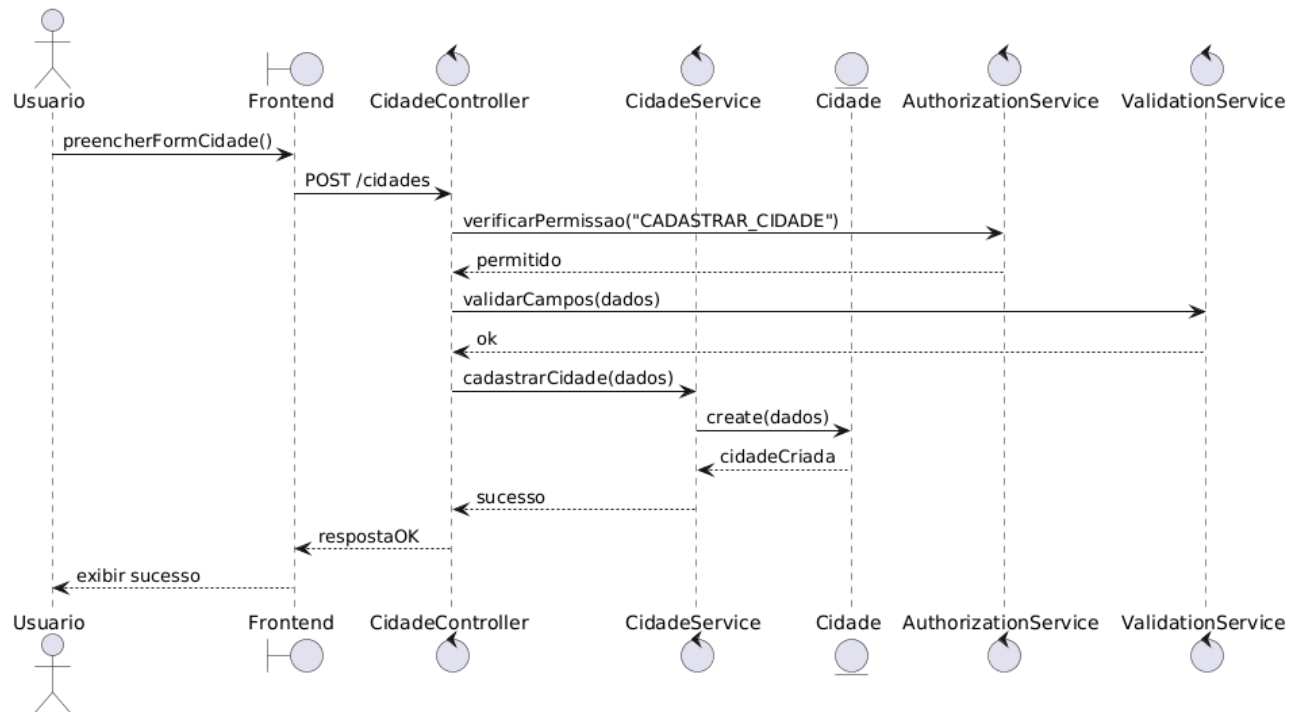


Figura 30 – Diagrama de Sequência: Cadastrar Cidades

A Figura 31 descreve a operação de consulta de cidades prevista no UC02. O *Frontend* solicita a listagem ao controller, que chama o *MunicipioService*. Este realiza a consulta no modelo *Municipio* e retorna a lista final ao usuário. Não há alteração de dados nem registro de histórico.

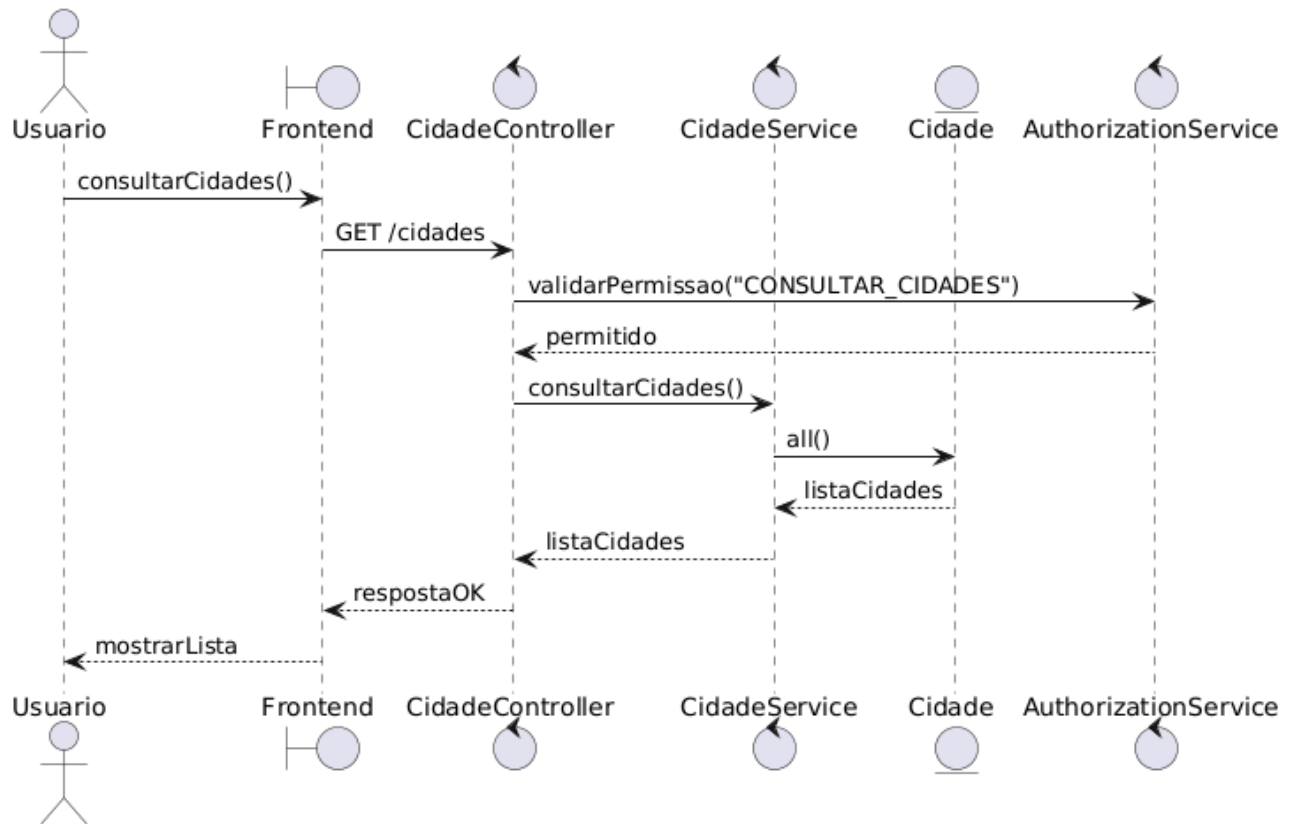


Figura 31 – Diagrama de Sequência: Consultar Cidades

Na Figura 32, é exibido o fluxo de criação de uma notícia, conforme definido no caso de uso UC03. O usuário envia dados como título e conteúdo ao controller, que valida permissões e aciona o `ValidationService`. O `NoticiaService` cria o registro no modelo `Noticia` e registra o evento no `HistoryService`. A confirmação é enviada ao *Frontend*.

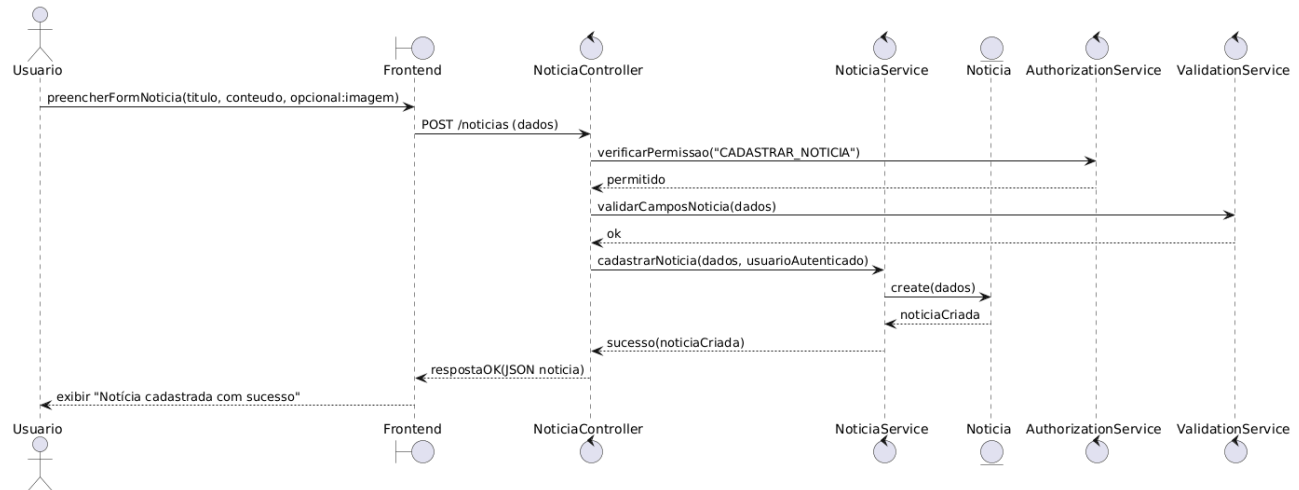


Figura 32 – Diagrama de Sequência: Publicar Notícias

Como mostra a Figura 33, o diagrama de sequência representa a operação de consulta de notícias prevista no caso de uso UC03. O usuário inicia a requisição a partir do *Frontend*, informando filtros opcionais como título, data ou autor. O *NoticiaController* recebe a solicitação, valida permissões conforme o perfil do usuário e aciona o *NoticiaService*, que constrói a consulta no modelo *Noticia* aplicando eventuais filtros. O resultado é retornado ao controller e enviado ao *Frontend*, que exibe a lista ao usuário. Por se tratar de uma operação de leitura, não há alterações no estado do sistema nem registro de histórico.

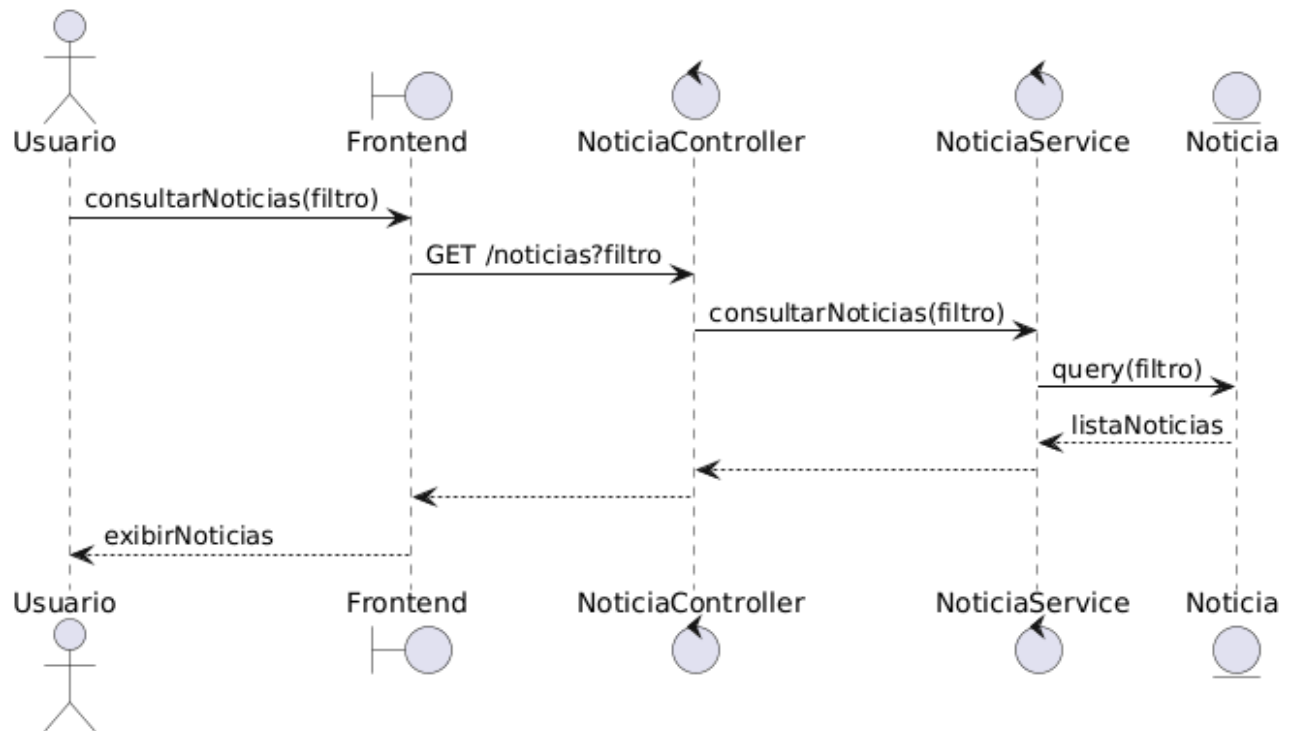


Figura 33 – Diagrama de Sequência: Consultar Notícias

Na Figura 34, o diagrama de sequência demonstra o fluxo da operação de atualização do conteúdo institucional de missão, conforme previsto no caso de uso UC03. Após o envio dos novos dados pelo *Frontend*, o controller valida permissões e utiliza o *ValidationService* para assegurar que o texto fornecido atende aos requisitos mínimos. O *ConteudoService* atualiza o registro no modelo correspondente e o *HistoryService* registra a alteração para auditoria. O *Frontend* recebe a confirmação de sucesso ao final do processo.

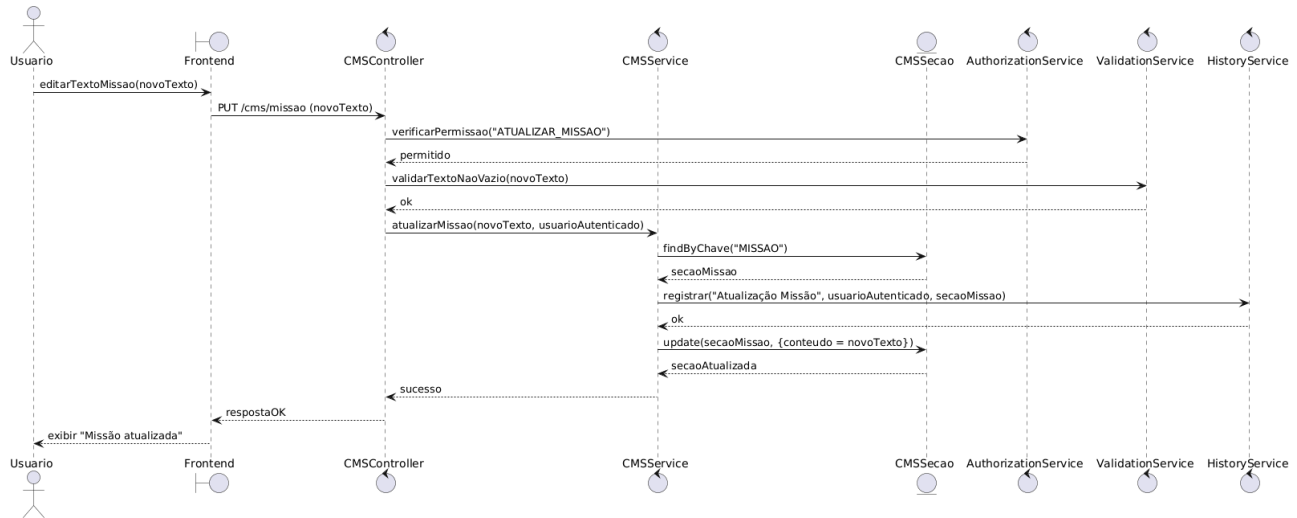


Figura 34 – Diagrama de Sequência: Atualizar Missão

A Figura 35 detalha a operação de atualização da seção de trajetória institucional, integrante do caso de uso UC03. O fluxo é iniciado pelo *Frontend* e passa pela validação de permissão, seguido pela verificação da integridade dos dados no *ValidationService*. O *ConteudoService* atualiza o registro e envia o evento ao *HistoryService*, garantindo rastreabilidade da mudança. Finalmente, o controller retorna o status de sucesso ao usuário.

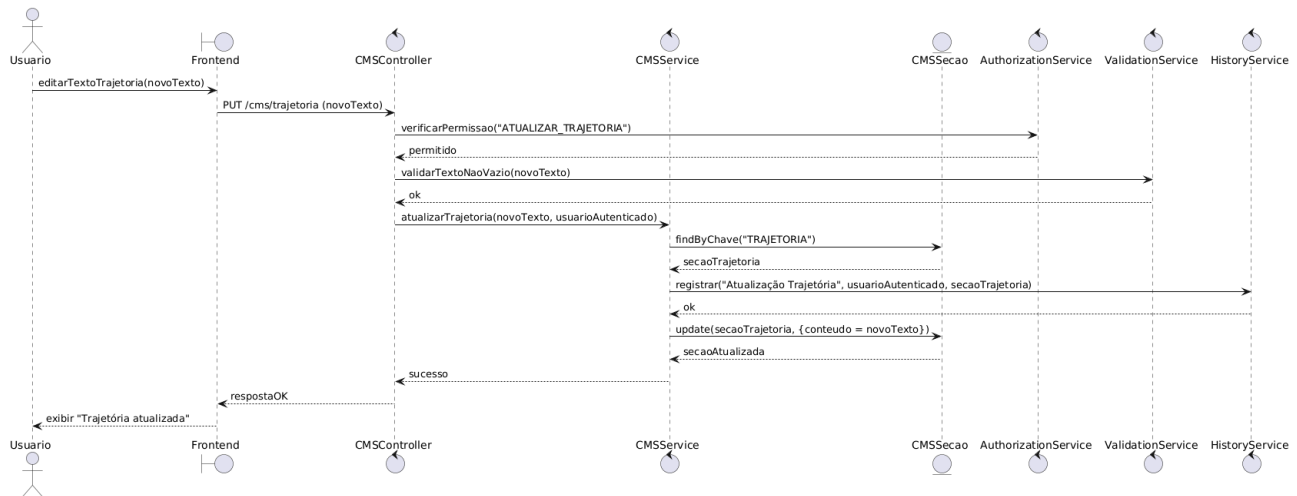


Figura 35 – Diagrama de Sequência: Atualizar Trajetória

Como mostra a Figura 36, o fluxo de criação de um projeto exibido no *site* faz parte do caso de uso UC03 e envolve permissão, validação e persistência. O controller recebe os dados enviados pelo

Frontend, valida as permissões e aciona o *ValidationService*. O *ProjetoSiteService* cria o novo registro no modelo *ProjetoSite* e registra o evento no *HistoryService*. O usuário recebe a confirmação da operação.

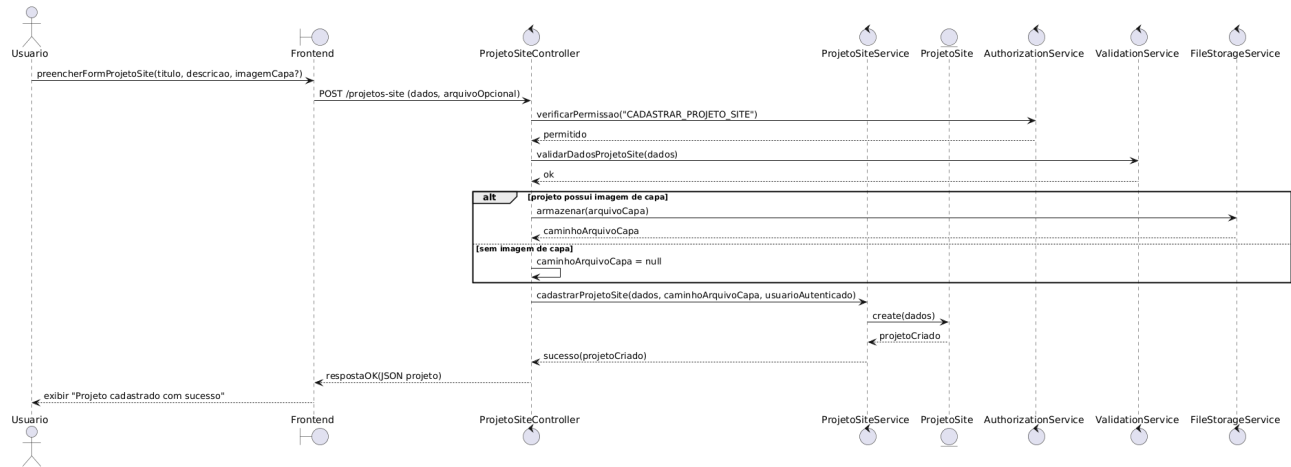


Figura 36 – Diagrama de Sequência: Cadastrar Projetos *Site*

Na Figura 37, o diagrama de sequência demonstra a operação de consulta de projetos exibidos no portal, pertencente ao caso de uso UC03. O *Frontend* requisita a listagem ao controller, que aciona o *ProjetoSiteService* para consultar o modelo *ProjetoSite*. A lista retornada é repassada ao usuário sem alterações no estado do sistema.

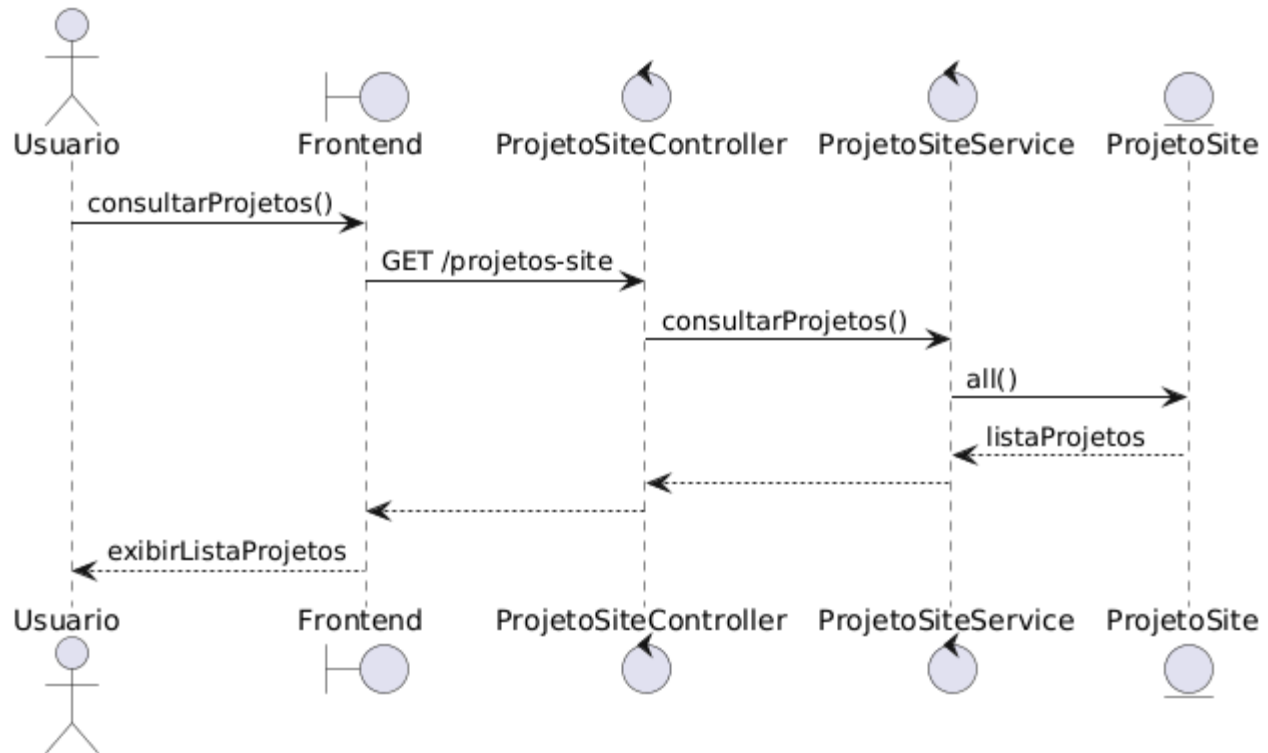


Figura 37 – Diagrama de Sequência: Consultar Projetos Site

A Figura 38 representa o fluxo de criação de um Projeto de Lei, conforme descrito no caso de uso UC04. Após validação de permissão e integridade dos dados, o ProjetoLeiService verifica se o número informado já existe. Em caso positivo, retorna erro ao usuário; caso contrário, cria o registro no modelo ProjetoLei e registra o histórico da operação. O *Frontend* recebe a resposta confirmando a criação do PL.

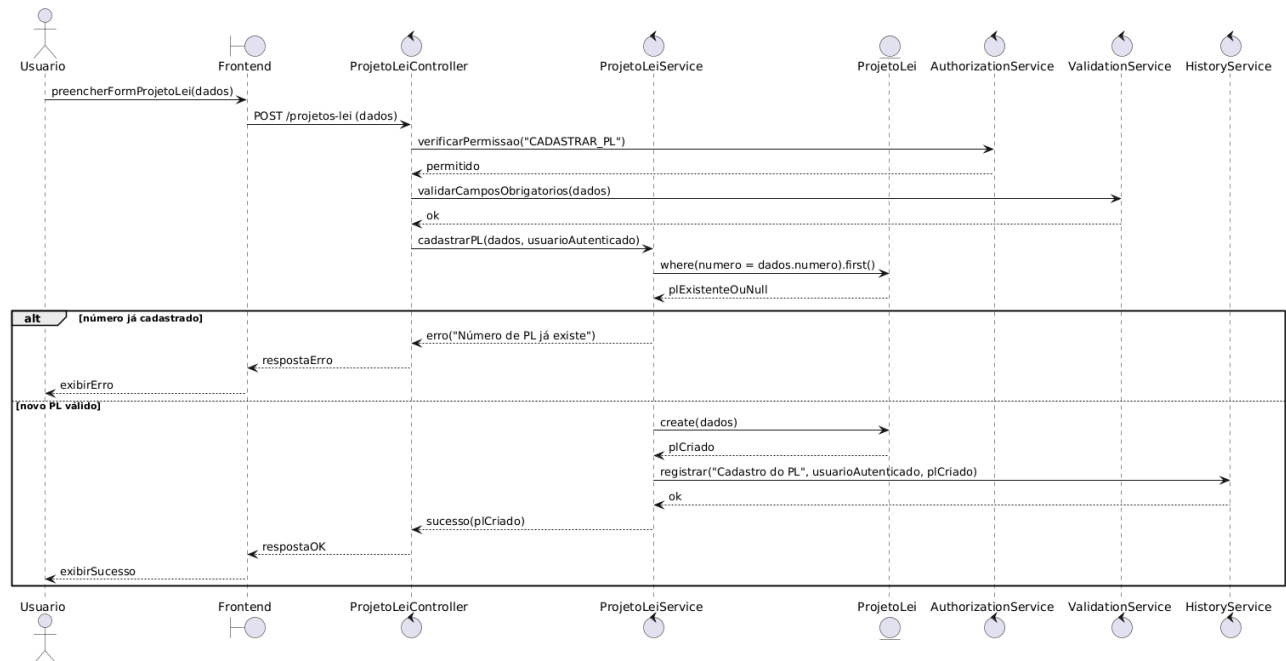


Figura 38 – Diagrama de Sequência: Cadastrar Projetos de Lei

Na Figura 39, observamos o fluxo da operação de atualização de um Projeto de Lei prevista no caso de uso UC04. O controller valida permissões e aciona o ValidationService para confirmar se as novas informações são permitidas. O ProjetoLeiService localiza o PL e atualiza o modelo ProjetoLei. O HistoryService registra a alteração e o resultado é enviado ao *Frontend*. Caso o PL não seja encontrado, retorna-se erro.

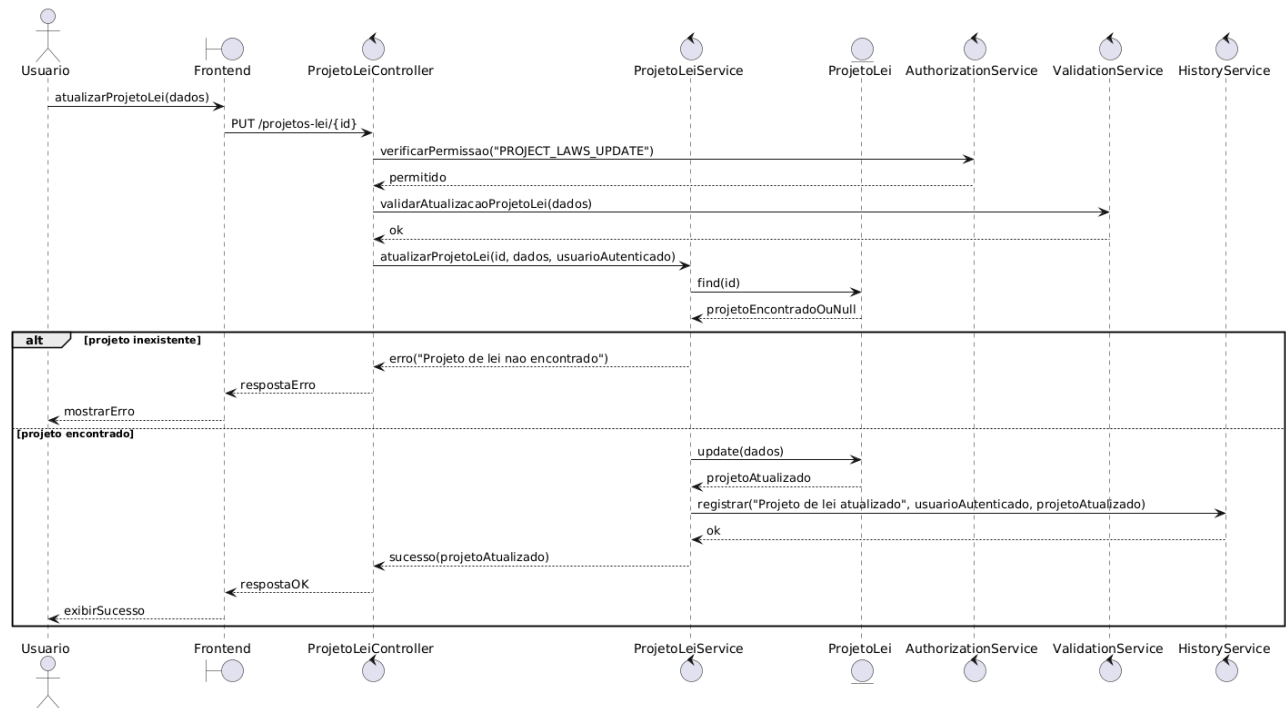


Figura 39 – Diagrama de Sequência: Atualizar Projeto de Lei

Como indica a Figura 40, o diagrama de sequência demonstra a operação de consulta de Projetos de Lei, parte integrante do caso de uso UC04. O usuário envia filtros através do *Frontend*; o controller valida permissões e aciona o *ProjetoLeiService*, que consulta o modelo *ProjetoLei* com base nos filtros. Os resultados são retornados ao usuário sem alteração de estado.

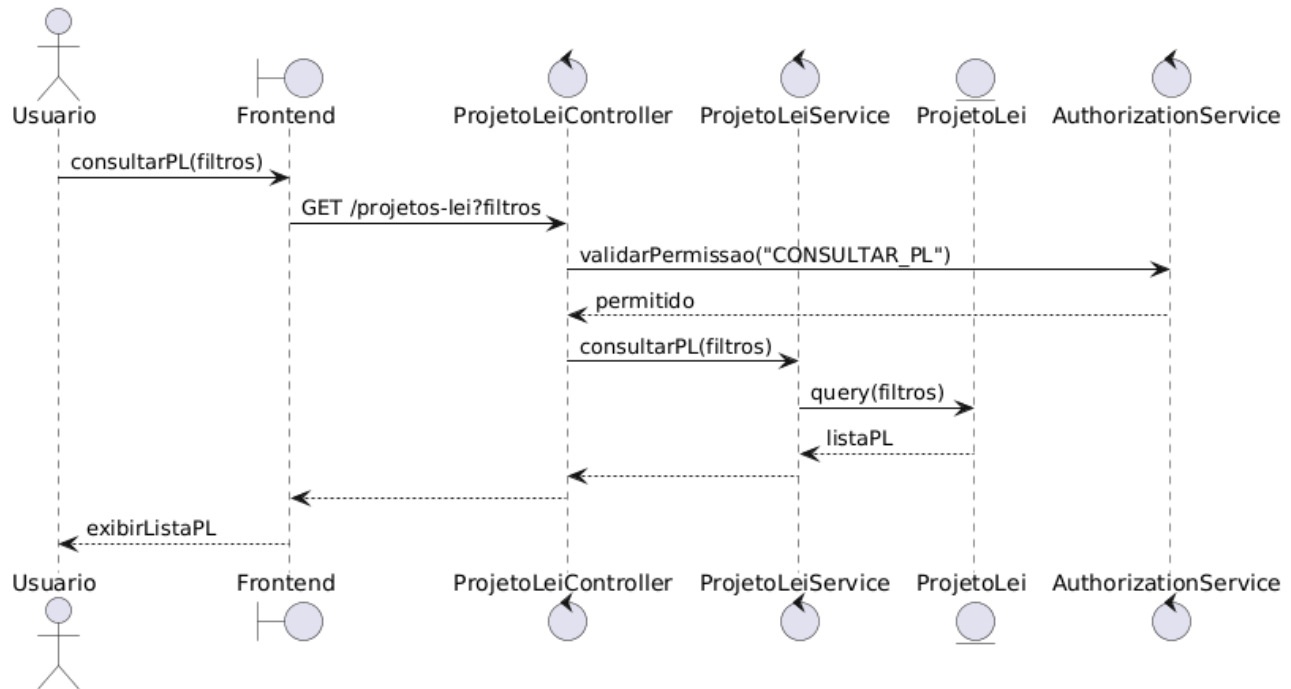


Figura 40 – Diagrama de Sequência: Consultar Projetos de Lei

Na Figura 41, o diagrama de sequência reflete o processo de cadastro de emenda previsto no caso de uso UC05. Após validação de permissão e integridade, o EmendaService confirma a existência da cidade associada e cria o registro no modelo Emenda. O HistoryService registra o evento e uma resposta de sucesso é enviada ao usuário.

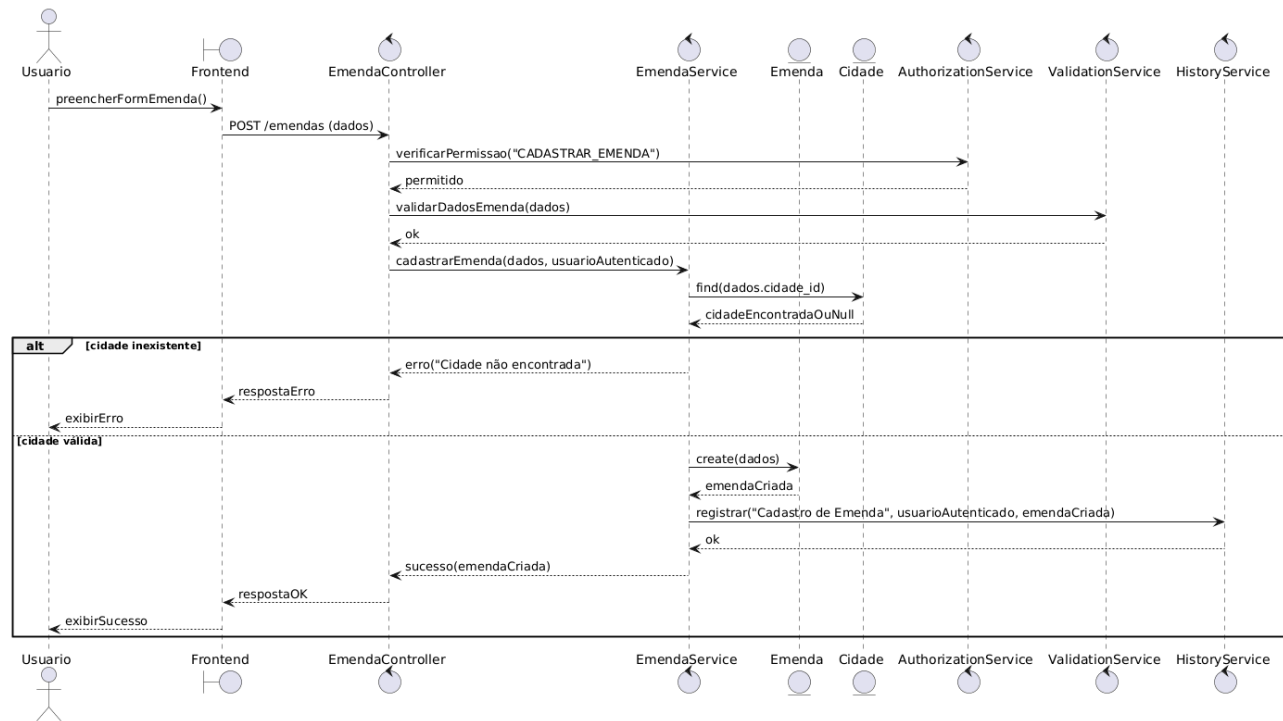


Figura 41 – Diagrama de Sequência: Cadastrar Emenda

A Figura 42 ilustra a mudança de uma emenda conforme o caso de uso UC05. Após permissão e validação dos novos dados, o EmendaService localiza a emenda e atualiza os campos. O histórico é gravado pelo HistoryService e o resultado retorna ao *Frontend*. Emendas inexistentes resultam em erros.

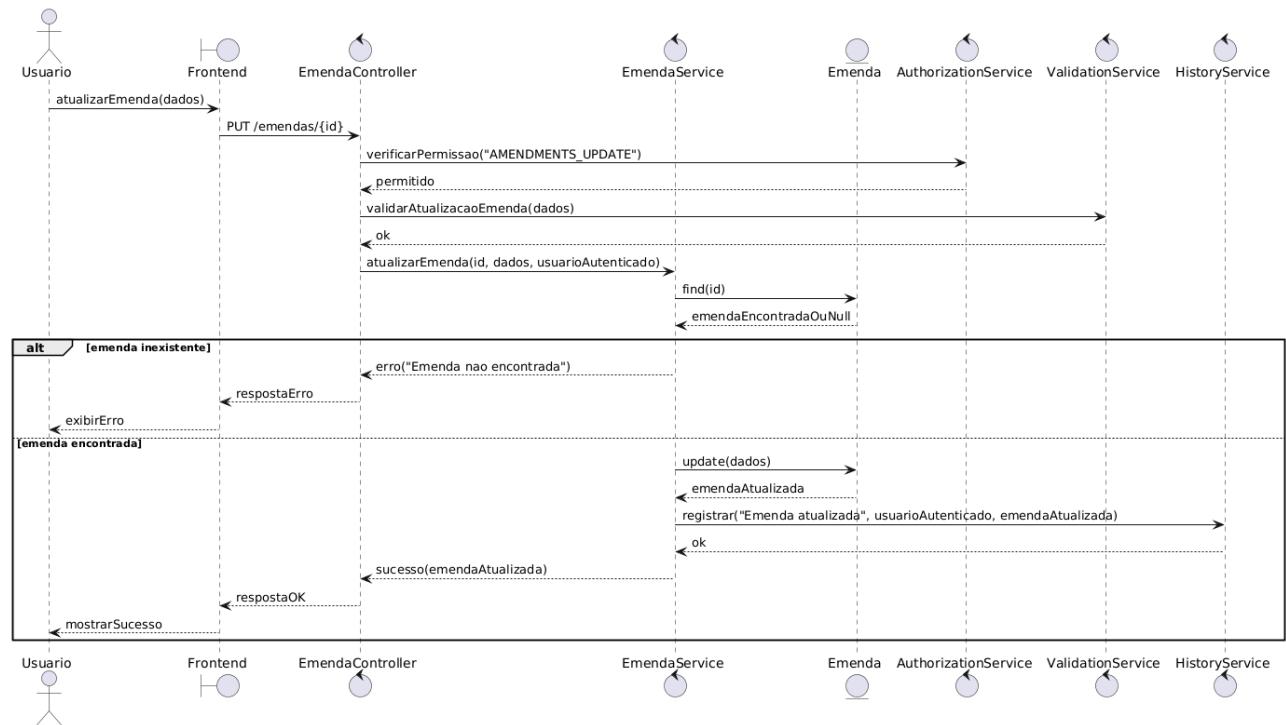


Figura 42 – Diagrama de Sequência: Atualizar Emenda

Na Figura 43, observa-se a operação de consulta de emendas, prevista no caso de uso UC05. O controller recebe filtros do *Frontend* e aciona o *EmendaService*, que monta a query no modelo *Emenda*. O resultado é exibido ao usuário. Como é uma operação de leitura, não há histórico nem atualização de estado.

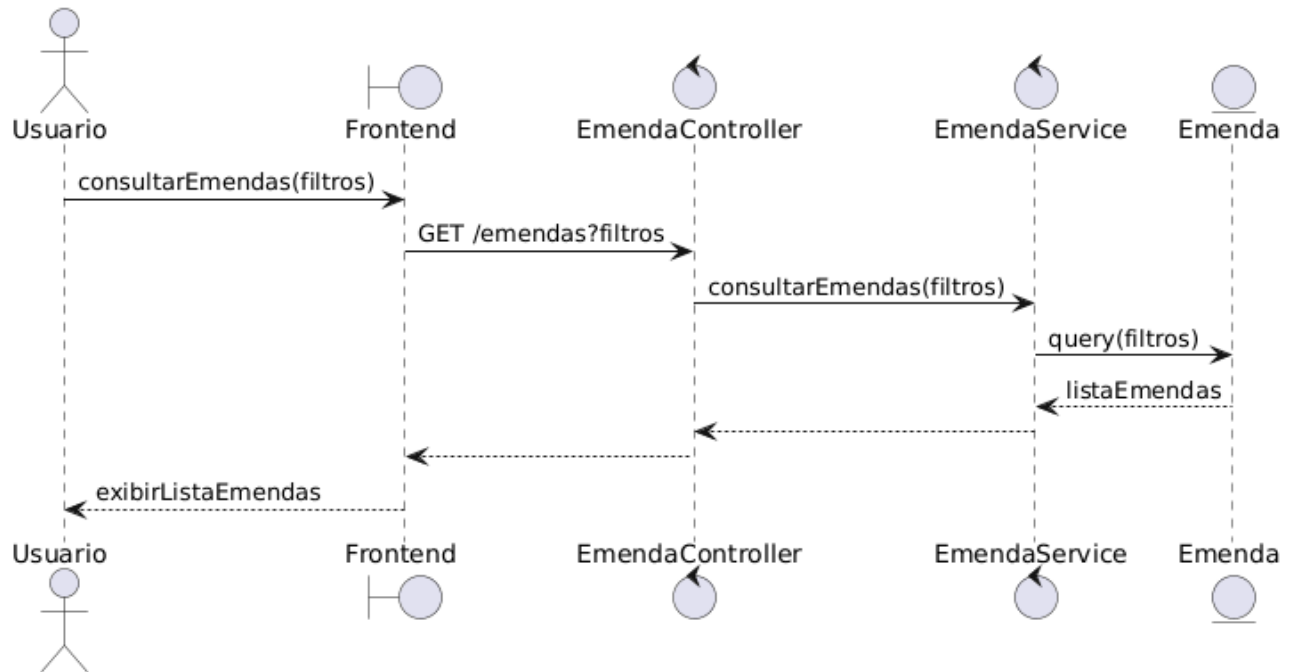


Figura 43 – Diagrama de Sequência: Consultar Emendas

Como mostra a Figura 44, o fluxo de abertura de demanda faz parte do caso de uso UC06. O *Frontend* envia os dados ao controller, que valida permissões e aciona o *ValidationService*. O *DemandaService* confirma a existência de cidade e instituição, cria o registro no modelo *Demanda* e aciona o *HistoryService* para registrar a criação. O retorno de sucesso é enviado ao usuário.

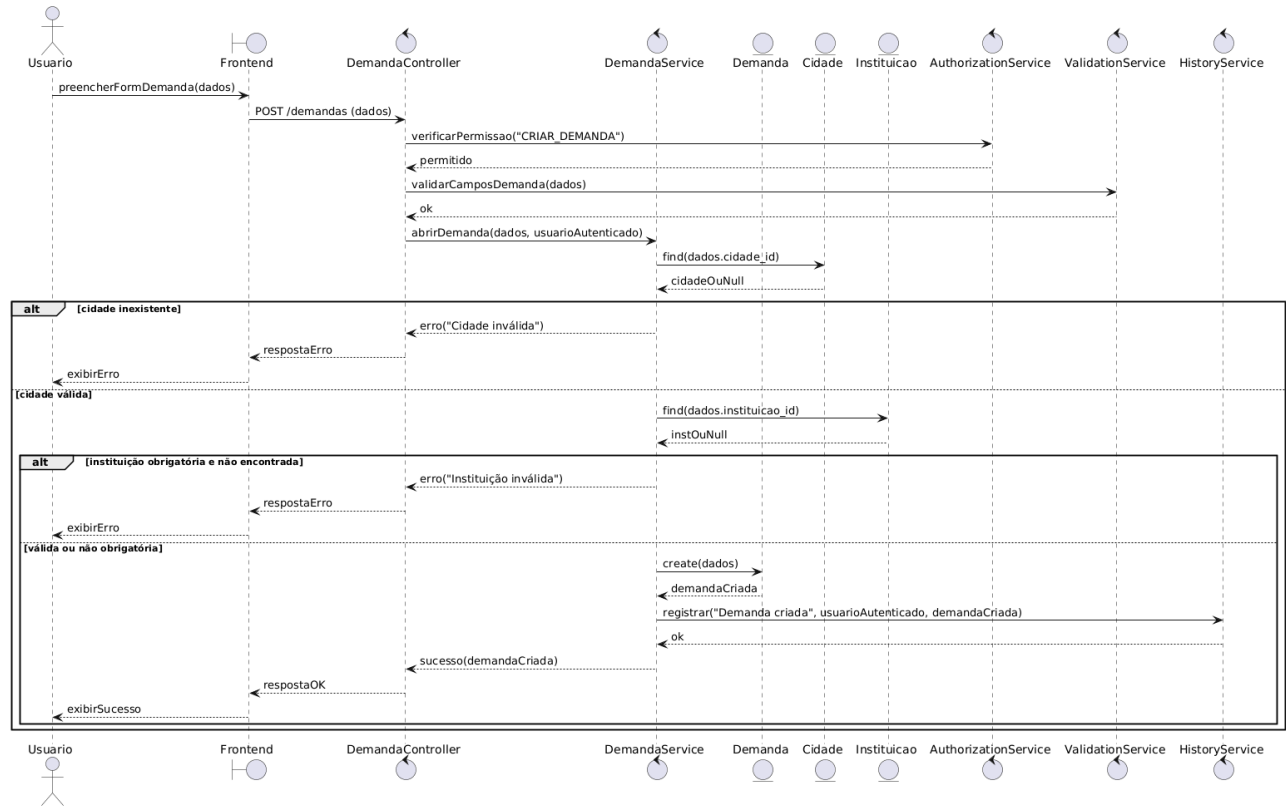


Figura 44 – Diagrama de Sequência: Abrir Demanda

Na Figura 45, o diagrama demonstra a operação de atualizar demanda prevista no caso de uso UC06. O controller valida se o usuário tem permissão para atualizar e o ValidationService verifica se os dados são válidos. O DemandaService confirma se o usuário destino existe antes de atualizar a demanda. O histórico é registrado e a resposta retornada ao *Frontend*.

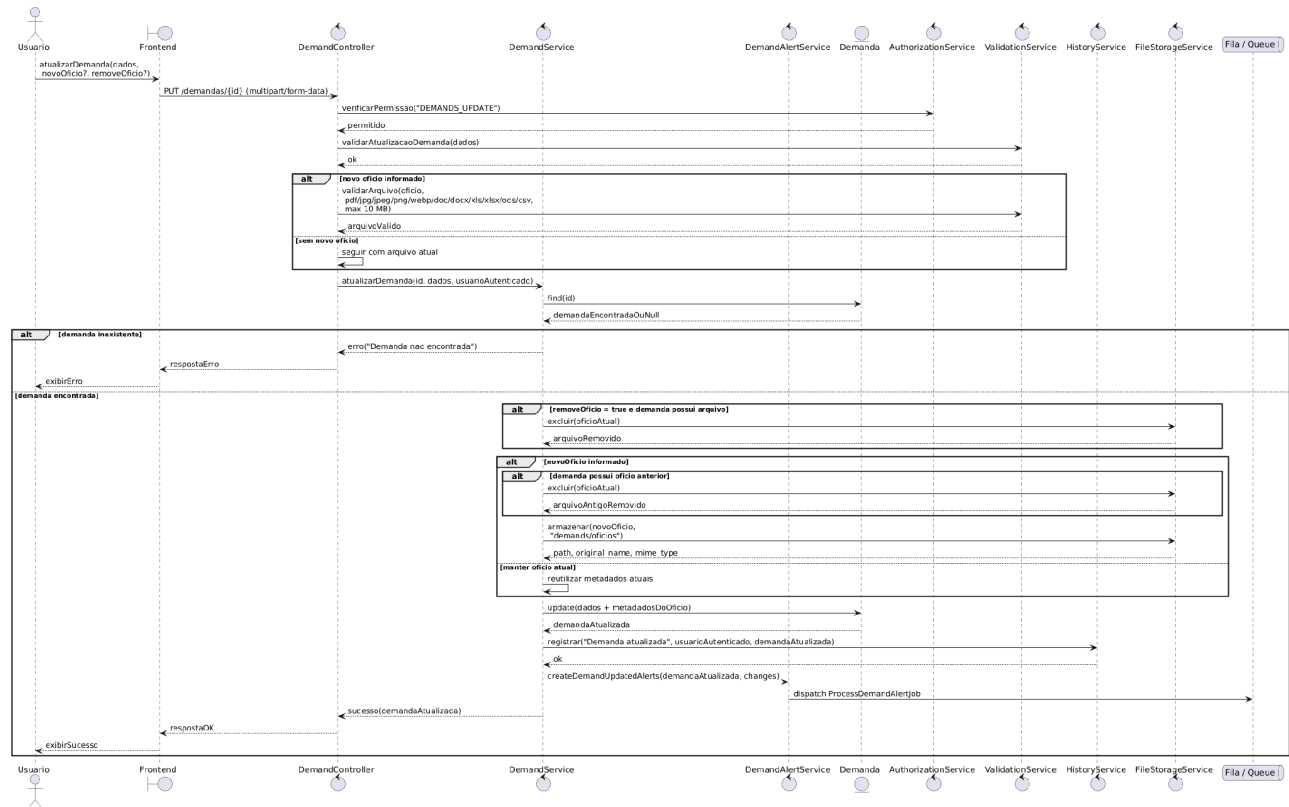


Figura 45 – Diagrama de Sequência: Atualizar Demanda

A Figura 46 representa o fluxo de registro de pessoa atendida na demanda, pertencente ao caso de uso UC06. Após validação de permissão e integridade dos dados, o DemandService atualiza a demanda com nome, CPF e contato da pessoa atendida. O evento é registrado no histórico e o usuário recebe a confirmação da operação.

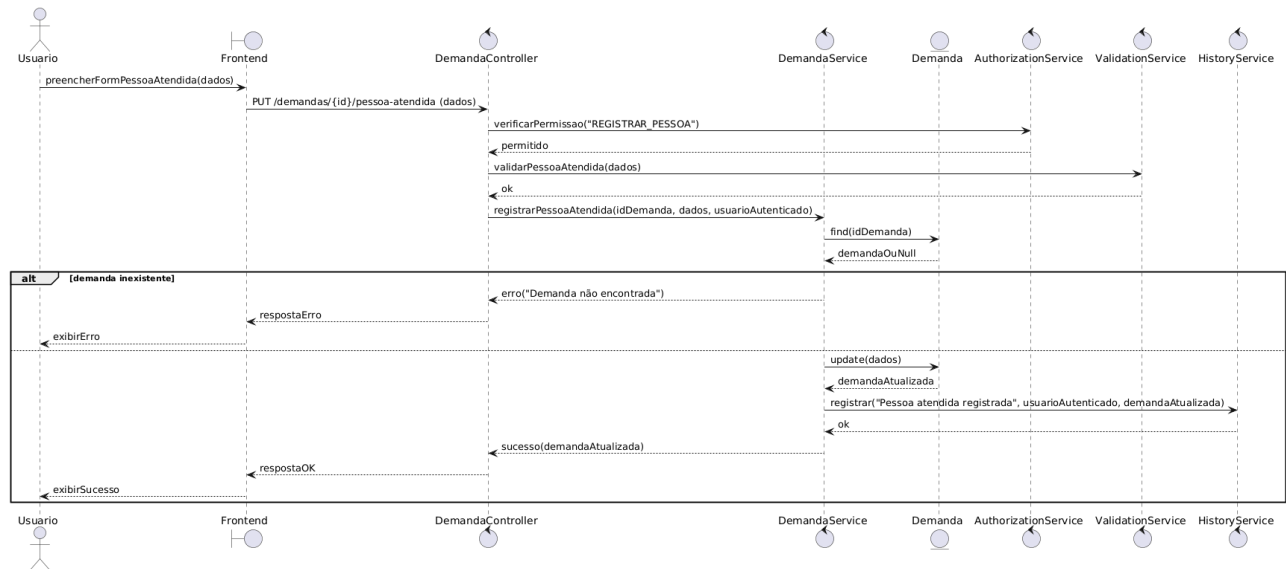


Figura 46 – Diagrama de Sequência: Registrar Pessoa Atendida

A Figura 47 apresenta a operação de criação de evento prevista no caso de uso UC07. Após validação de permissão e dados, o AgendaService cria o registro no modelo Evento e registra o histórico. O usuário recebe uma confirmação de que o evento foi criado com sucesso.

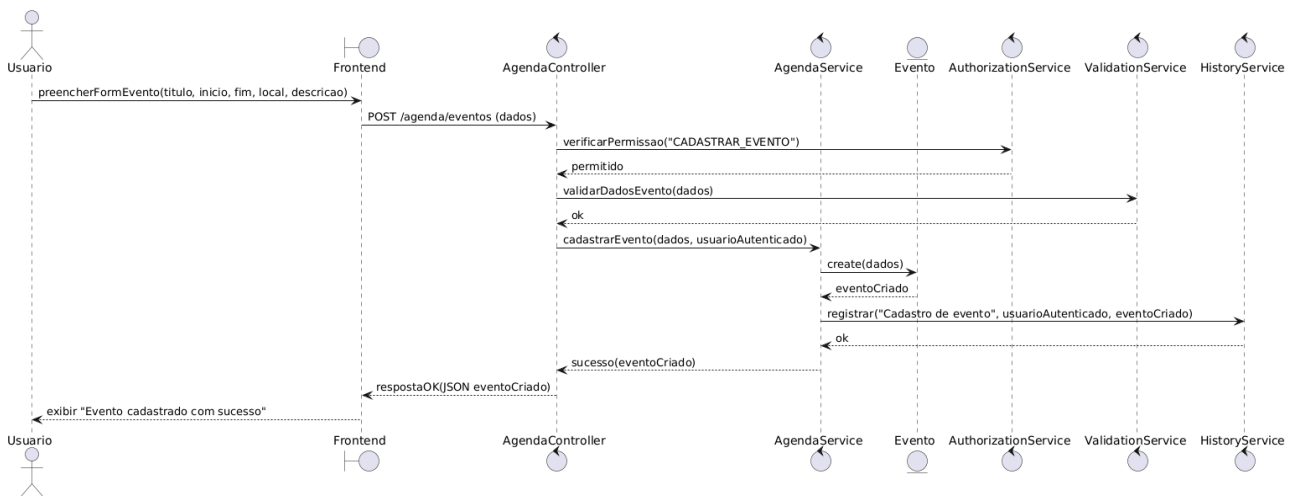


Figura 47 – Diagrama de Sequência: Cadastrar Evento

Na Figura 48, observa-se o fluxo de consulta do calendário, integrante do caso de uso UC07. O controller recebe a requisição do *Frontend* e aciona o *AgendaService* para recuperar os eventos do mês. Os dados retornam ao usuário que visualiza o calendário atualizado.

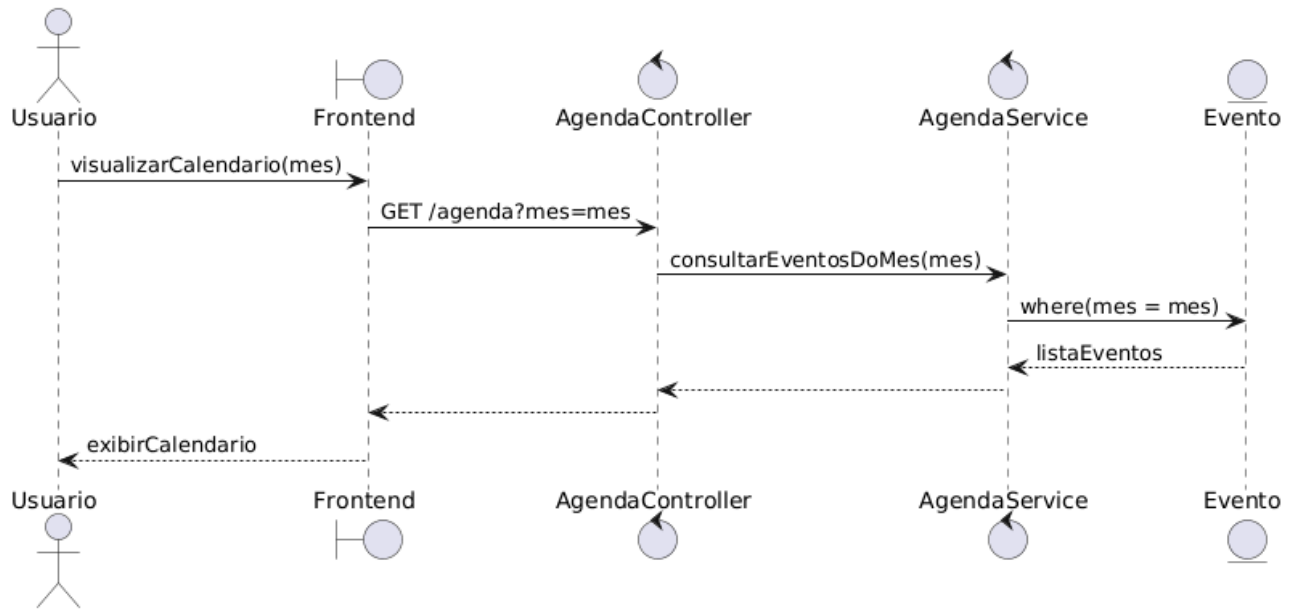


Figura 48 – Diagrama de Sequência: Visualizar Agenda

A Figura 49 apresenta a operação de geração do relatório de Projetos de Lei, prevista no caso de uso UC08. Após validação de permissão específica, o **RelatorioService** consulta todos os PLs e gera o documento consolidado. O relatório é devolvido ao *Frontend* para download.

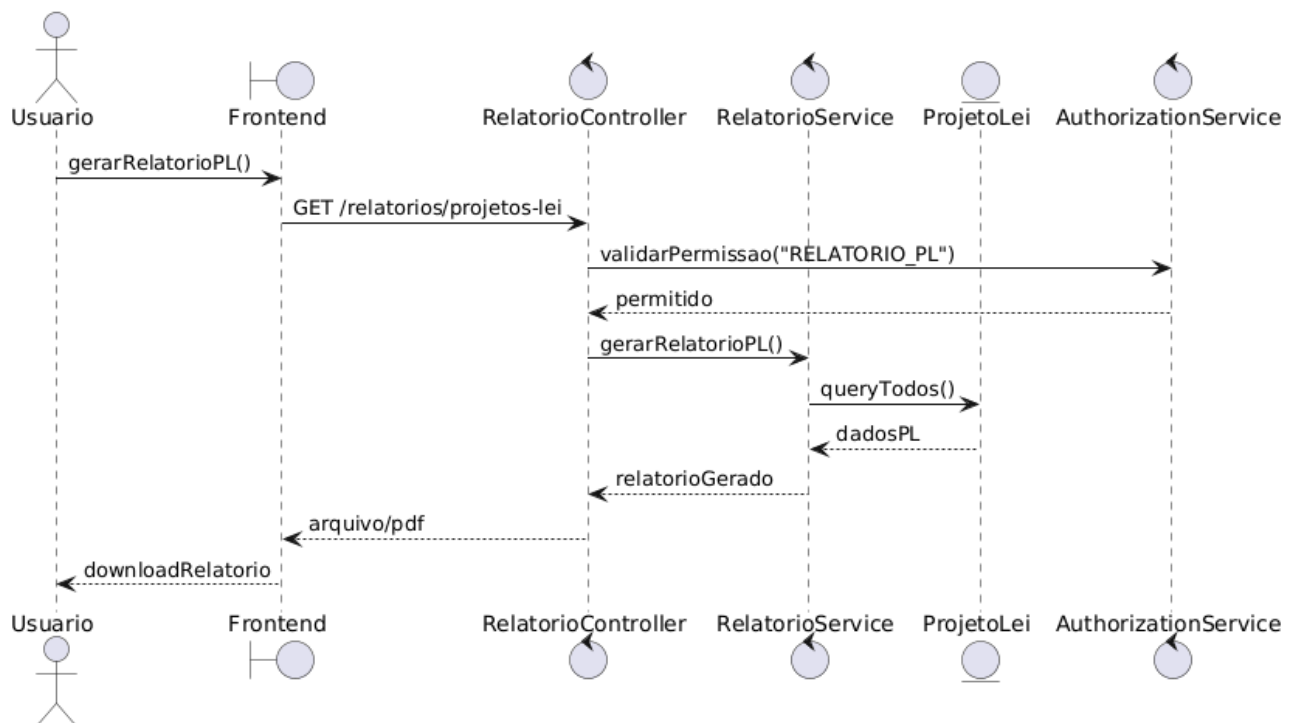


Figura 49 – Diagrama de Sequência: Relatório de Projetos de Lei

Como mostra a Figura 50, o controller valida a permissão do usuário, aciona o RelatorioService e o serviço consulta o conjunto de emendas antes de gerar o relatório. O *Frontend* recebe o arquivo gerado para download.

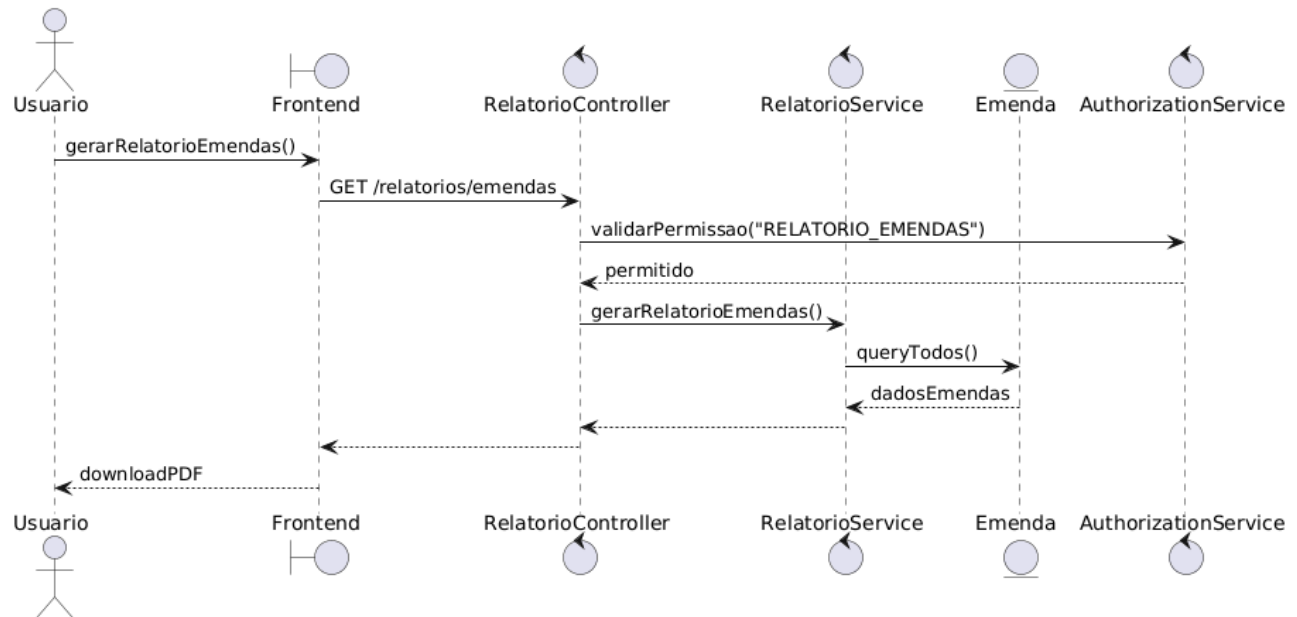


Figura 50 – Diagrama de Sequência: Relatório de Emendas

Na Figura 51, o diagrama evidencia a geração do relatório de demandas conforme o caso de uso UC08, com validação de permissão, consulta de dados e geração do documento consolidado.

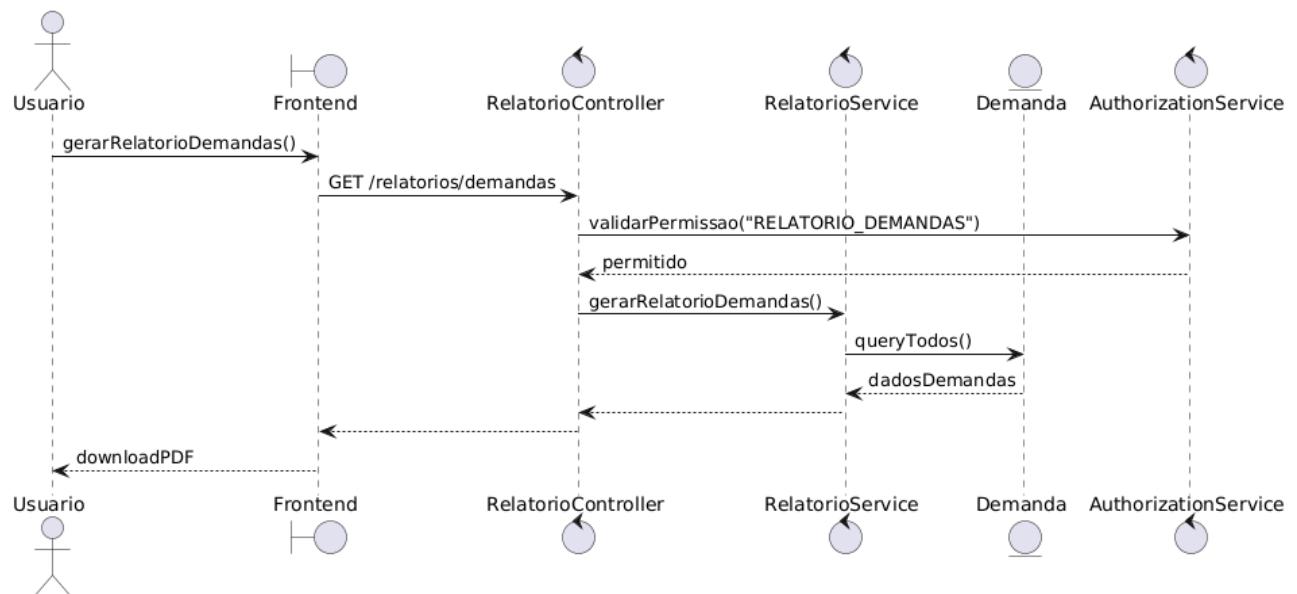


Figura 51 – Diagrama de Sequência: Relatório de Demandas

A Figura 52 demonstra a operação mais ampla do caso de uso UC08, na qual o RelatorioService consolida dados de emendas, projetos de lei e demandas em um único documento acessível apenas à Deputada. O controller valida a permissão, o serviço consulta todas as entidades e gera o relatório final para download.

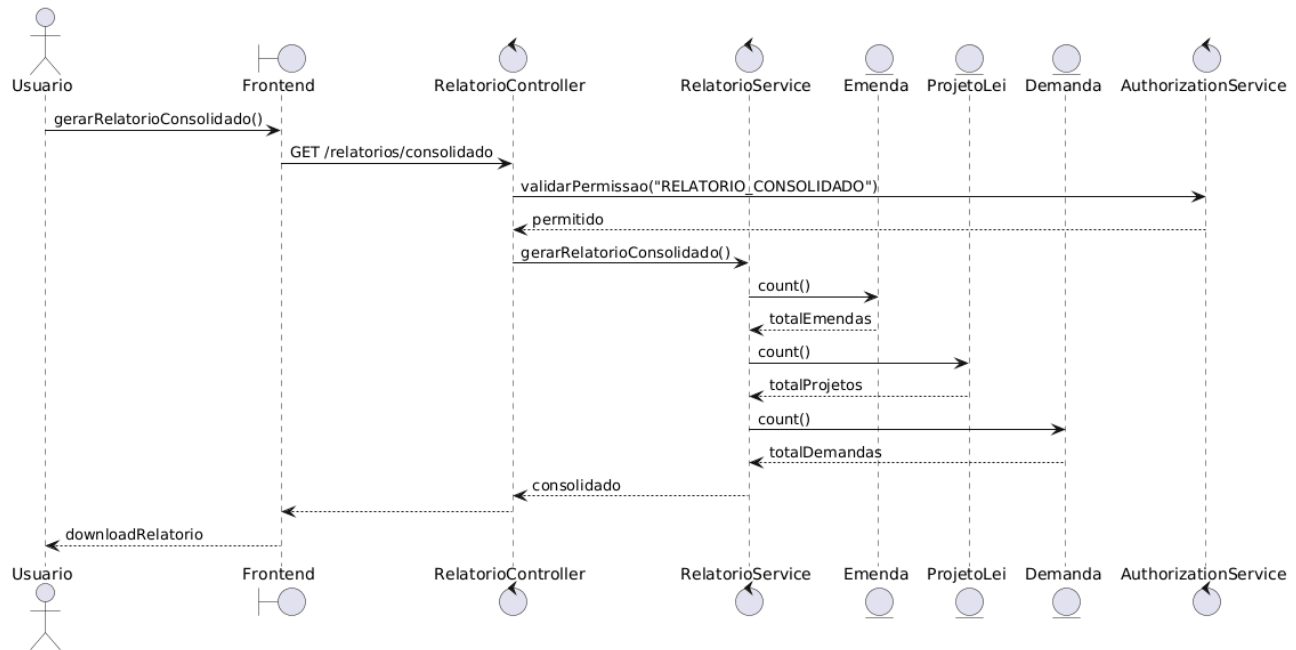


Figura 52 – Diagrama de Sequência: Relatório Consolidado

A Figura 53 apresenta o fluxo de identificação e cadastro do cidadão no *Chatbot* do Gabinete Virtual. A interação inicia-se no WhatsApp, que atua como canal de comunicação entre o cidadão e o *chatbot*. As mensagens são encaminhadas à *Chatbot* API, responsável por identificar o telefone do remetente, verificar se já existe um cidadão vinculado e, quando necessário, solicitar os dados complementares de cadastro. Em seguida, o *backend* persiste ou atualiza as informações do cidadão e devolve a confirmação ao usuário pelo próprio WhatsApp, permitindo o uso das demais funcionalidades do *chatbot*.

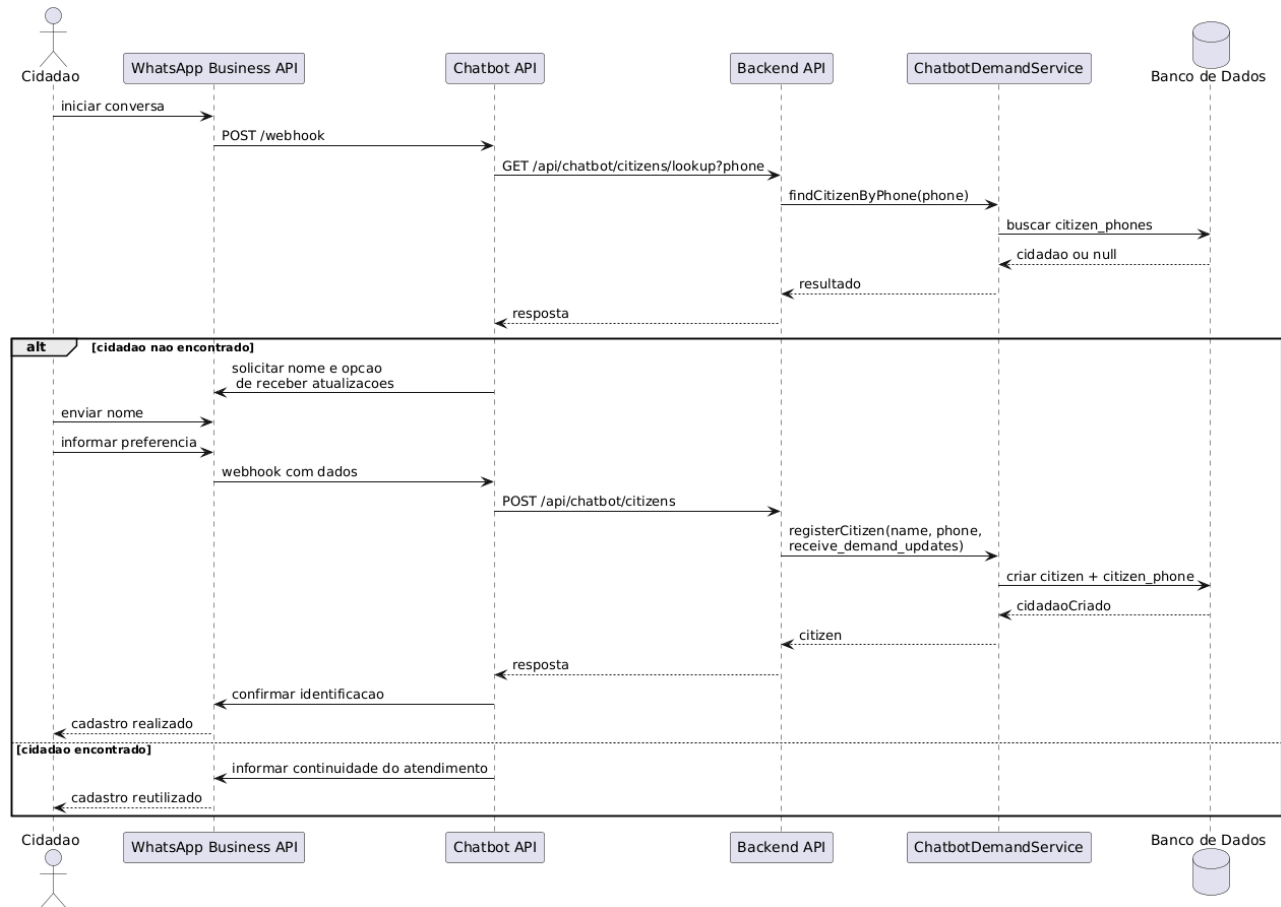


Figura 53 – Diagrama de Sequência: Cadastrar Cidadão

Como mostra a Figura 54, o diagrama descreve o envio de demandas pelo *chatbot* a partir de um fluxo mais completo de atendimento. O cidadão interage pelo WhatsApp, a *Chatbot API* identifica o usuário, consulta as cidades e instituições disponíveis e coleta os dados necessários para a solicitação. Antes do envio ao *backend*, a mensagem passa por validações de idioma, conteúdo ofensivo e similaridade com demandas recentes. O serviço de demandas então registra a solicitação, seja em fluxo normal ou como demanda descartada, garantindo persistência, rastreabilidade e vínculo com o cidadão identificado.

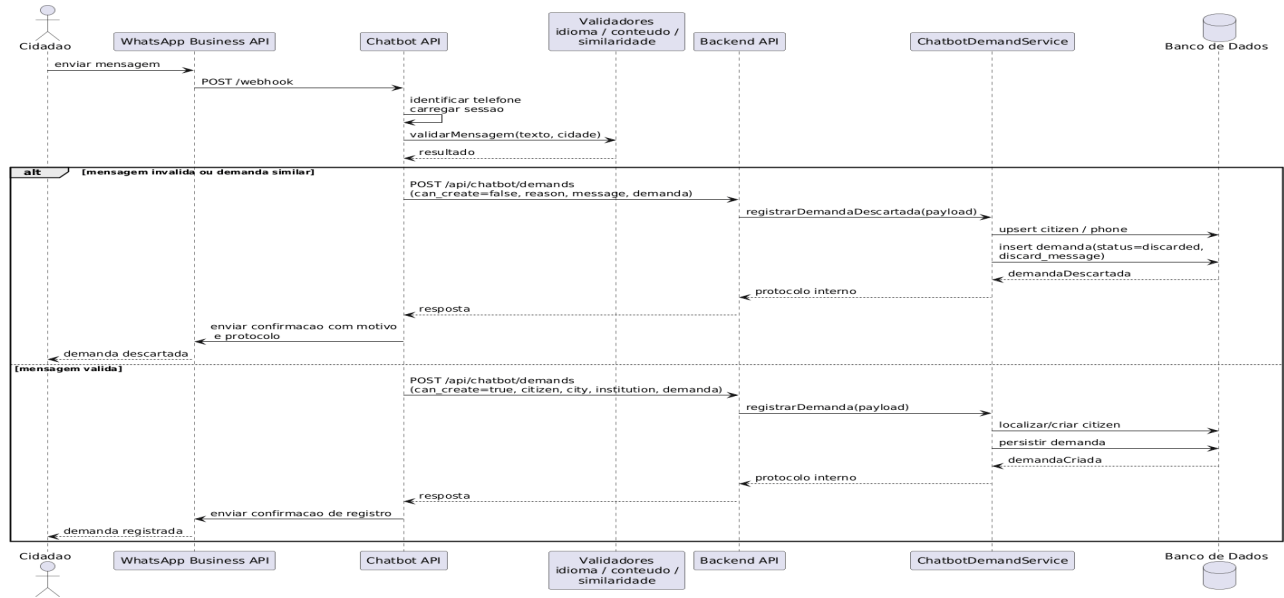


Figura 54 – Diagrama de Sequência: Registrar Demanda

A Figura 55 apresenta o fluxo de notificações automáticas relacionadas a atualizações de demandas. Sempre que uma demanda vinculada a um cidadão sofre uma alteração relevante, o *backend* registra a mudança, cria o alerta correspondente e notifica a *Chatbot API*. Em seguida, o *chatbot* encaminha a mensagem ao cidadão por meio do WhatsApp, sem necessidade de solicitação explícita, desde que ele tenha optado por receber esse tipo de aviso.

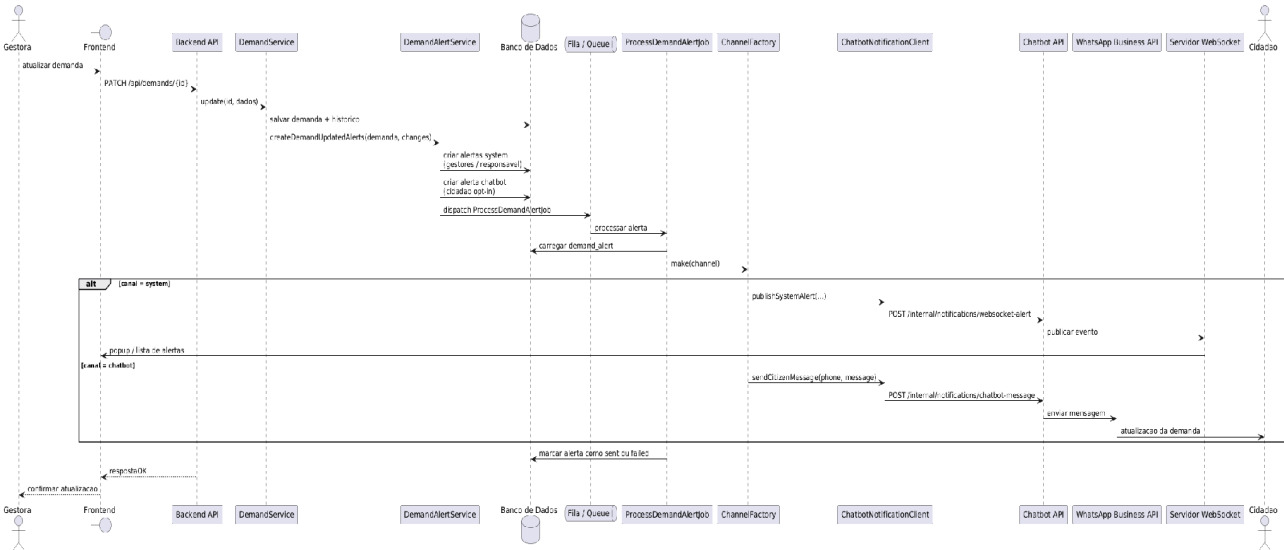


Figura 55 – Diagrama de Sequência: Receber Atualização

Conforme demonstrado na Figura 56, o diagrama representa o fluxo do caso de uso UC-CH05, no qual o cidadão opta por receber conteúdos institucionais pelo *chatbot*. A escolha é encaminhada à

Chatbot API via WhatsApp e registrada pelo PreferenciaService no banco de dados, garantindo que o envio de informações respeite a decisão do usuário.

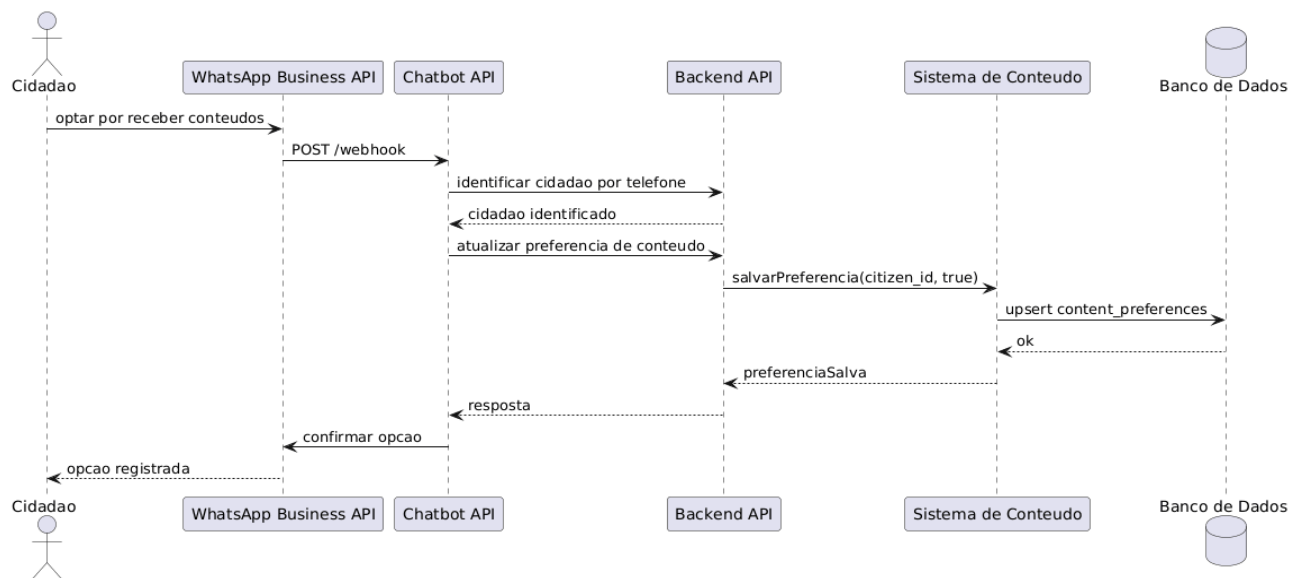


Figura 56 – Diagrama de Sequência: Optar por Receber Conteúdo

3.3 Diagramas de Comunicação

Representam o fluxo de mensagens entre os objetos participantes dos casos de uso, destacando as relações estruturais e a cooperação entre componentes durante as operações do sistema.

Como mostra a Figura 57, o diagrama de comunicação referente à autenticação representa o fluxo de entrada seguro no sistema Gabinete Virtual, sendo um processo transversal e obrigatório para todos os demais casos de uso. A interação inicia-se quando o usuário informa suas credenciais por meio do *frontend*, que encaminha a requisição ao controlador de autenticação da API. O *AuthController* delega a validação ao *AuthService*, responsável por consultar o repositório de usuários, verificar as credenciais informadas e, em caso de sucesso, acionar o serviço de geração de *tokens* JWT. Ao final do processo, o *token* de autenticação é retornado ao *frontend*, permitindo o acesso seguro às funcionalidades conforme o perfil e permissões do usuário.

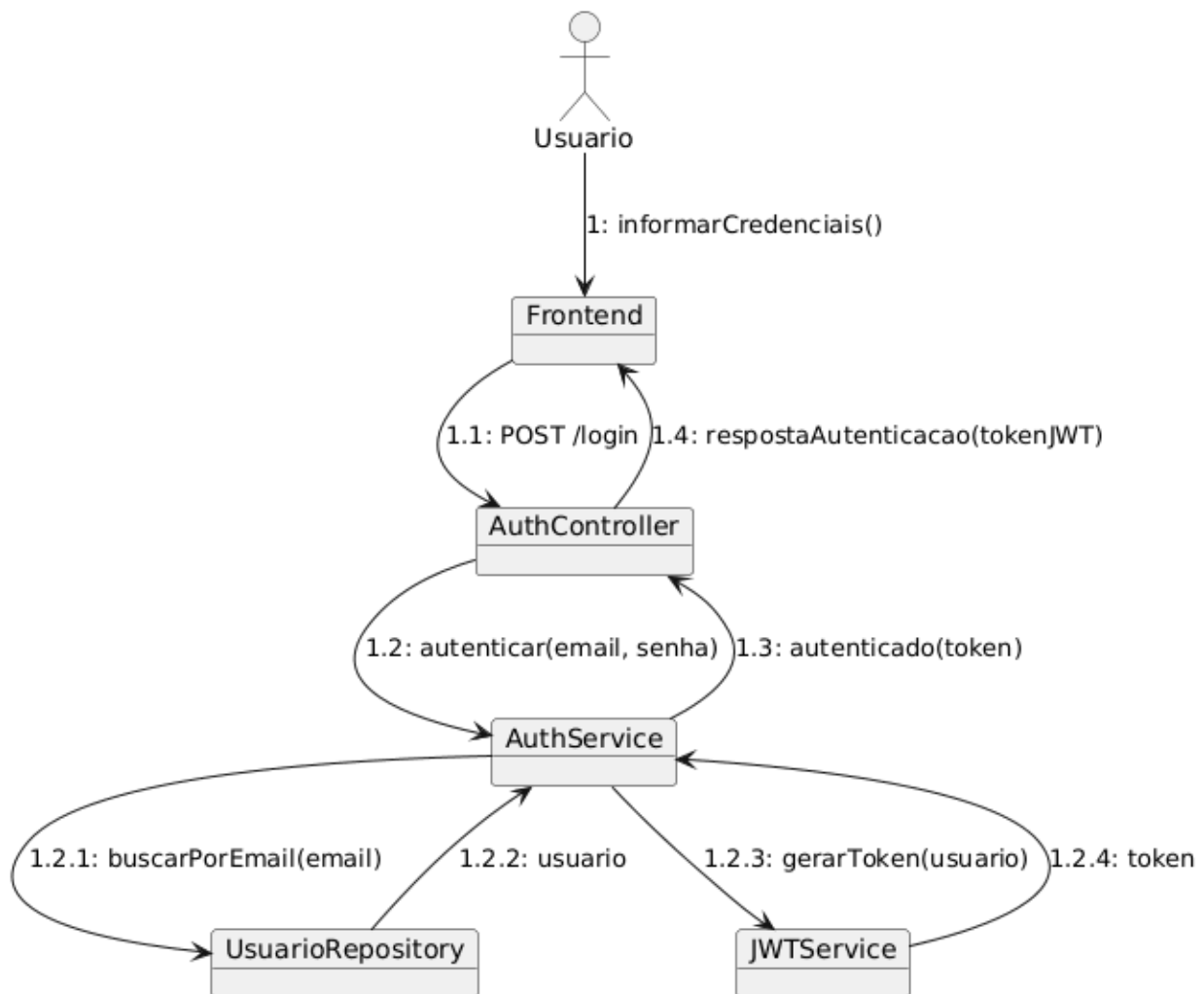
UC01 - Autenticar no Sistema + Validar Permissão

Figura 57 – Diagrama de Comunicação: Autenticar no Sistema

Na Figura 58, o diagrama de comunicação apresenta o gerenciamento dos cadastros básicos do sistema, fundamentais para o correto funcionamento dos demais módulos. O usuário inicia a ação pelo *frontend*, que envia a solicitação ao controlador responsável. Este controlador repassa a operação ao serviço de aplicação correspondente, que aplica regras de negócio, validações e consistência dos dados. Em seguida, o serviço interage com o repositório adequado para persistir ou consultar as informações. O resultado da operação é então retornado ao *frontend*, permitindo a confirmação visual da ação realizada e mantendo a integridade dos dados institucionais utilizados em demandas, emendas, agenda e relatórios.

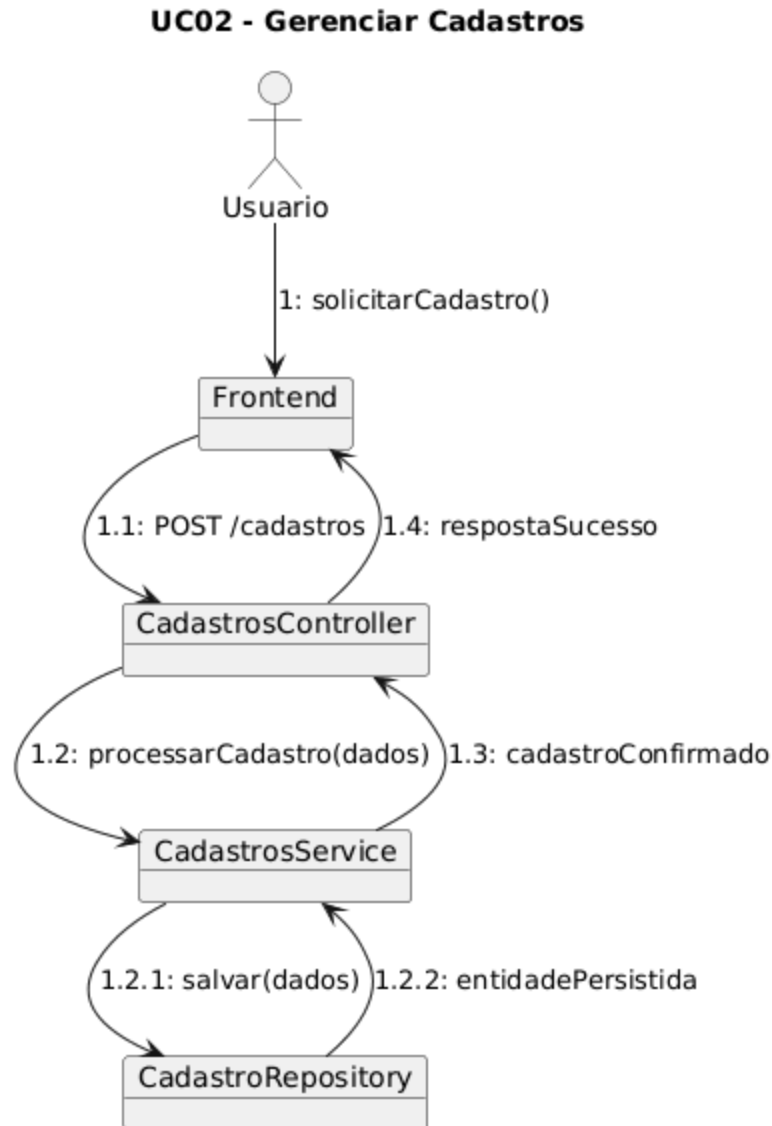


Figura 58 – Diagrama de Comunicação: Gerenciar Cadastros

A Figura 59 apresenta o diagrama de comunicação relacionado ao gerenciamento do conteúdo institucional disponibilizado no *site* público do gabinete, como notícias, missão, trajetória e projetos. O usuário interage inicialmente com o *frontend* para criar ou editar conteúdos, que são encaminhados ao controlador do CMS. O CMSService centraliza a lógica de validação, versionamento e organização das informações, interagindo com o repositório responsável pela persistência dos dados. Após a conclusão da operação, o sistema retorna uma confirmação ao usuário, assegurando atualização controlada e confiável das informações públicas.

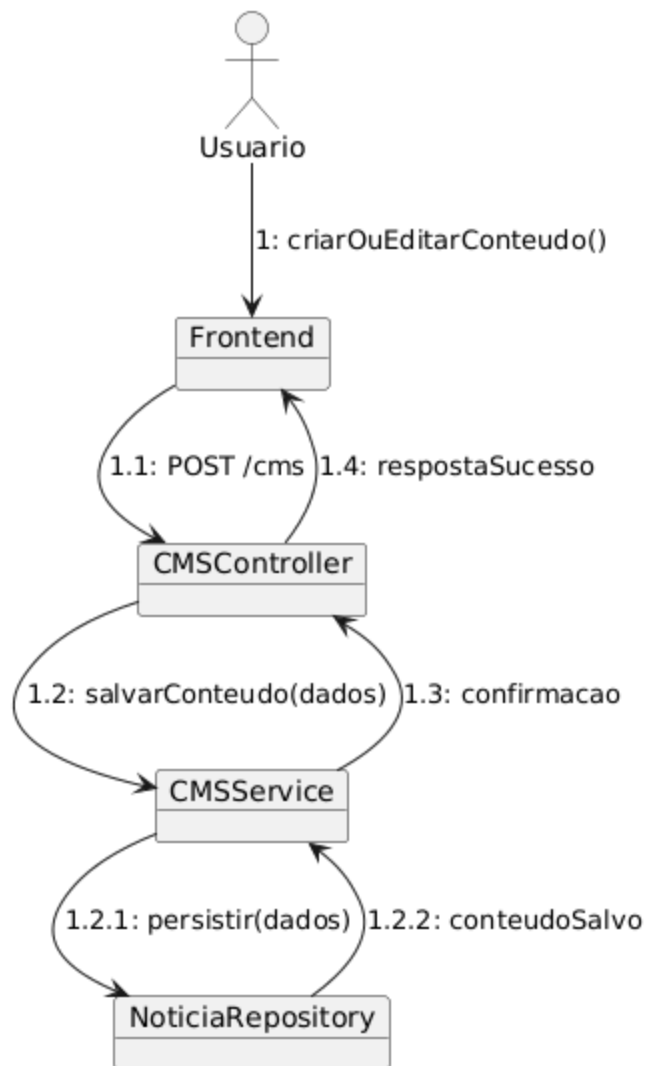
UC03 - Gerenciar Conteúdo do Site

Figura 59 – Diagrama de Comunicação: Gerenciar Conteúdo do Site

Como mostra a Figura 60, o diagrama de comunicação detalha o processo de gerenciamento dos projetos de lei acompanhados pelo gabinete. A interação começa no *frontend*, onde o usuário solicita o cadastro, a atualização ou a consulta de um projeto de lei. O controlador específico encaminha a solicitação ao serviço de negócios, que valida os dados, aplica as regras pertinentes e delega a persistência ao repositório de projetos de lei. Após a execução da operação, a confirmação é enviada de volta ao *frontend*, garantindo rastreabilidade e organização das informações legislativas.

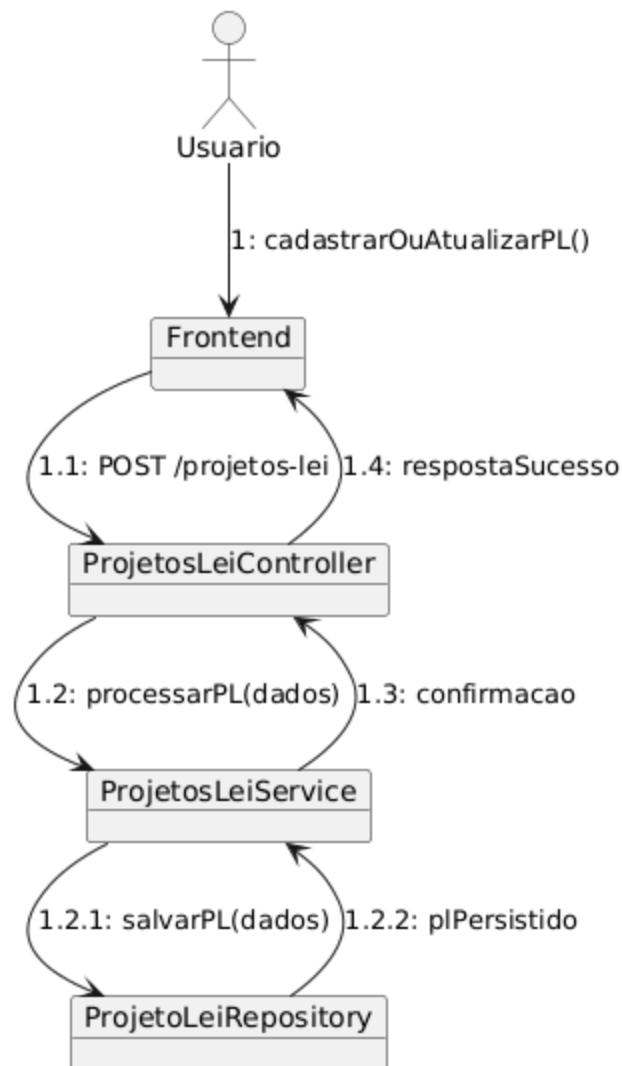
UC04 - Gerenciar Projetos de Lei

Figura 60 – Diagrama de Comunicação: Gerenciar Projetos de Lei

Na Figura 61, o diagrama de comunicação descreve o gerenciamento das emendas parlamentares, incluindo cadastro, atualização e consulta. O fluxo evidencia a interação entre *frontend*, controlador de emendas e o serviço de aplicação, que centraliza as regras de negócio relacionadas a valores, situação e vínculo com municípios. O serviço utiliza o repositório para persistência das informações e retorna o resultado da operação ao controlador, que responde ao *frontend*. Esse caso de uso é essencial para o controle transparente das emendas e para o suporte à geração de relatórios legislativos.

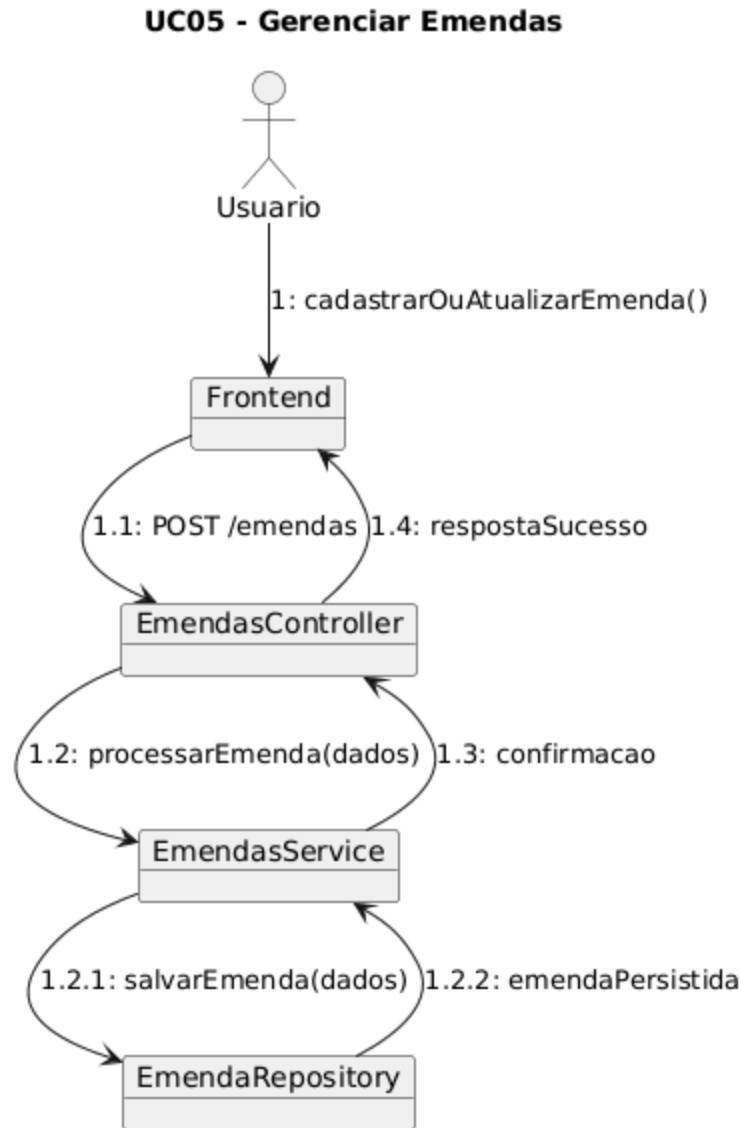


Figura 61 – Diagrama de Comunicação: Gerenciar Emendas

A Figura 62 apresenta o diagrama de comunicação do gerenciamento de demandas, principal fluxo operacional do sistema. O usuário inicia a ação no *frontend*, que encaminha a solicitação ao controlador de demandas. O serviço processa as informações, aplica regras de prioridade, valida vínculos necessários, registra histórico e interage com o repositório para persistir os dados. Quando uma atualização exige comunicação adicional, o fluxo também aciona o componente de notificações para informar usuários internos e cidadãos vinculados. Ao final, o sistema retorna a confirmação ao *frontend*, garantindo organização, rastreabilidade e eficiência no atendimento à população.

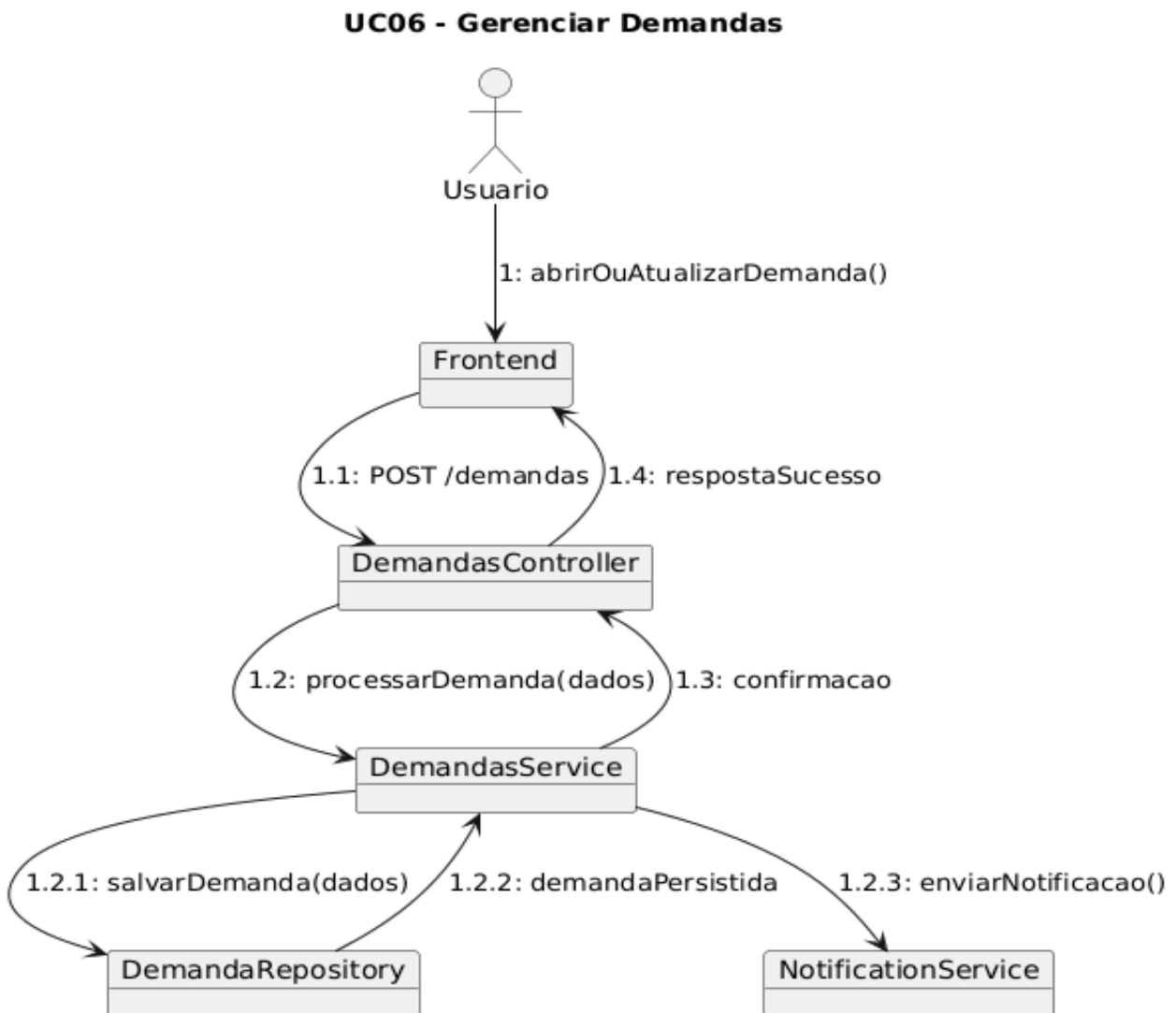


Figura 62 – Diagrama de Comunicação: Gerenciar Demandas

Como mostra a Figura 63, o diagrama de comunicação da agenda institucional representa o fluxo de criação, atualização e acompanhamento dos eventos do gabinete. O usuário solicita a operação pelo *frontend*, que envia a requisição ao controlador de agenda. O serviço valida horários, identifica possíveis conflitos, persiste os eventos no repositório correspondente e sincroniza os lembretes automáticos derivados da agenda. A resposta final é enviada ao *frontend*, permitindo o acompanhamento visual dos compromissos e contribuindo para a organização das atividades parlamentares.

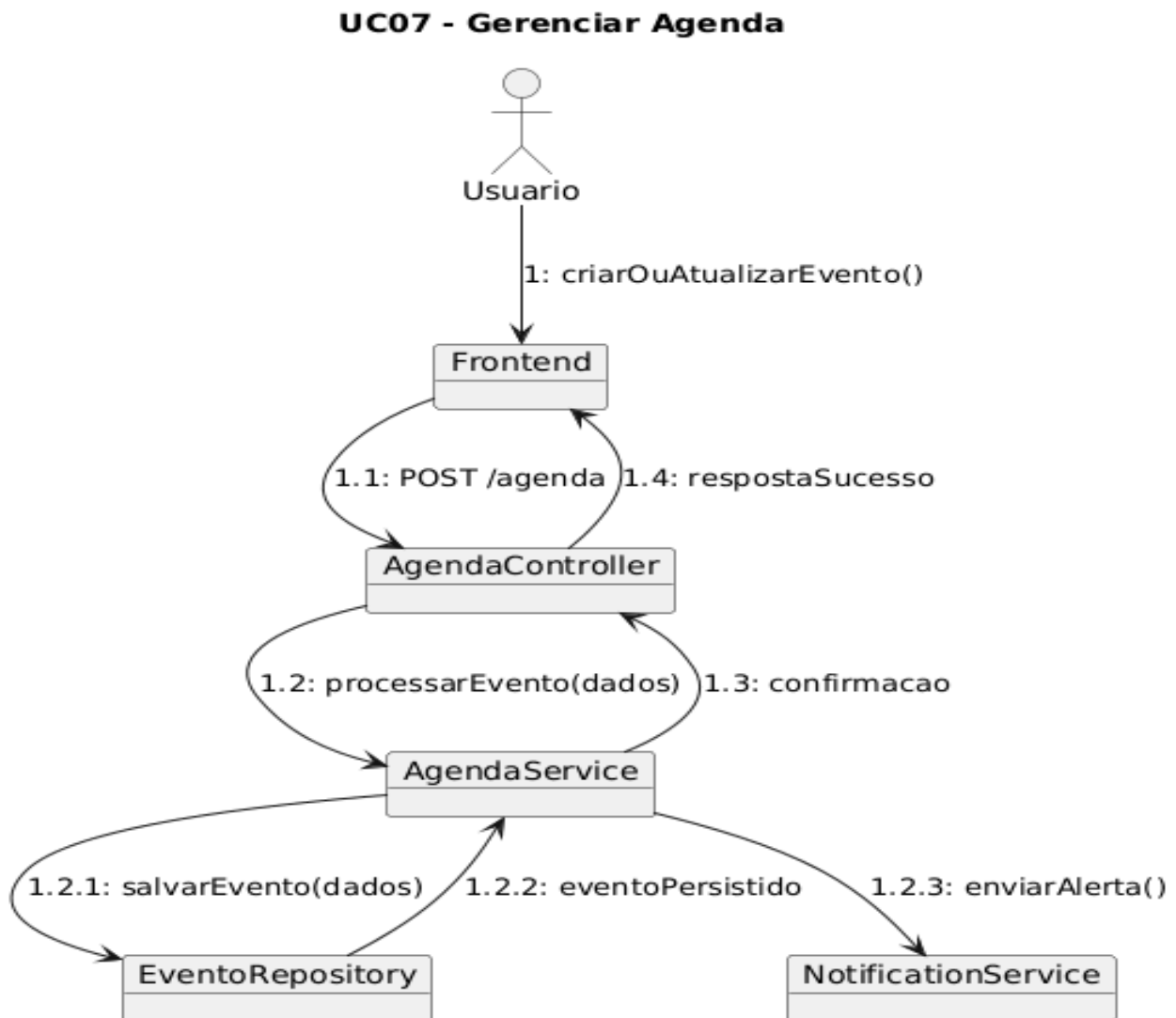


Figura 63 – Diagrama de Comunicação: Gerenciar Agenda

Na Figura 64, o diagrama de comunicação representa o processo de geração de relatórios gerenciais e estratégicos do sistema. O usuário solicita o relatório por meio do *frontend*, que encaminha a requisição ao controlador de relatórios. O serviço consolida os dados a partir dos repositórios correspondentes, aplicando filtros, regras de acesso e agregações conforme o tipo de relatório solicitado. Após a consolidação, o resultado é retornado ao *frontend* para visualização ou exportação, oferecendo suporte à tomada de decisão e visão analítica do funcionamento do gabinete.

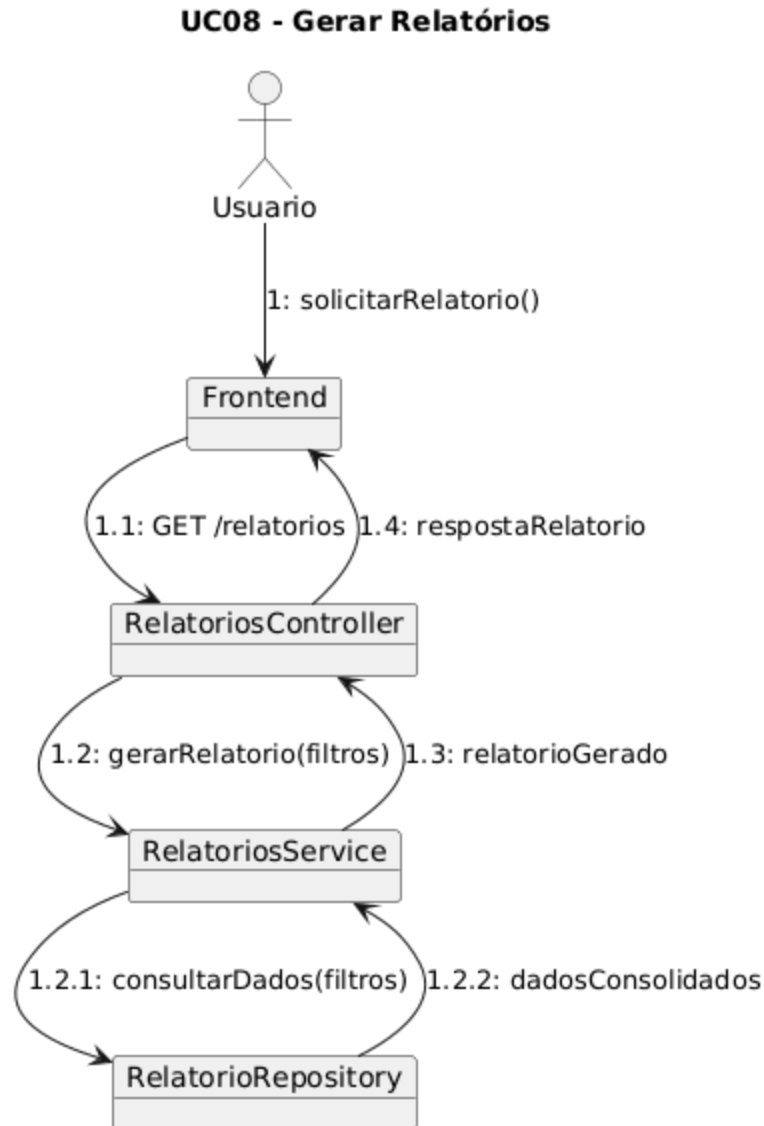


Figura 64 – Diagrama de Comunicação: Gerar Relatórios

3.4 Arquitetura

A arquitetura lógica do sistema Gabinete Inteligente foi modelada utilizando o C4 Model, uma abordagem amplamente adotada para representar sistemas de *software* em diferentes níveis de abstração. Neste trabalho, foram utilizados dois níveis principais: o Nível 1, correspondente ao Diagrama de Contexto, que apresenta a visão mais ampla do sistema e de seus atores externos; e o Nível 2, correspondente ao Diagrama de Contêineres, que detalha os principais blocos de execução

internos da solução. Cada nível possui diagrama e descrição próprios, organizados em subseções específicas, de forma a facilitar a compreensão progressiva da arquitetura adotada.

O Diagrama de Contexto (Nível 1), representado na Figura 65, descreve o ambiente geral em que o sistema opera, detalhando quem interage com o Gabinete Virtual e de que maneira essas interações ocorrem. Nessa camada, a solução é apresentada em termos de seus grandes componentes visíveis externamente, evidenciando os limites do sistema e suas relações com usuários e serviços integrados.

Como mostra o diagrama, os principais atores internos são a Deputada, o Chefe de Gabinete, a Gestora de Demandas e a Equipe de Atendimento. A Deputada utiliza o sistema para consultas estratégicas relacionadas a cidades, emendas e agenda. O Chefe de Gabinete interage com a plataforma para organizar compromissos e acompanhar prioridades operacionais. A Gestora de Demandas utiliza o sistema para atribuir responsáveis, atualizar demandas e gerar relatórios. Já a Equipe de Atendimento registra e atualiza demandas, além de interagir com lideranças e demais dados de apoio necessários ao funcionamento do gabinete.

O diagrama também evidencia dois perfis distintos de cidadão. O primeiro é o Cidadão (Consulta Pública), que acessa o *frontend* web para visualizar conteúdos institucionais, notícias e demais informações públicas do gabinete. O segundo é o Cidadão (*Chatbot*), que interage com o sistema por meio de uma plataforma de mensagens, enviando solicitações e recebendo respostas automatizadas.

Diferentemente da versão anterior da arquitetura, o sistema passa a ser representado por quatro blocos principais: o *Frontend* Web (React), responsável pela interface acessada no navegador; o *Backend* API (Laravel/PHP), onde se concentram regras de negócio, persistência e integração entre módulos; a *Chatbot* API (FastAPI/*Webhook*), responsável pelo tratamento das mensagens recebidas e enviadas ao cidadão; e o Servidor *WebSocket*, responsável pela entrega de alertas em tempo real ao *frontend*. Essa modelagem torna explícita a separação entre os diferentes canais de acesso e os mecanismos especializados de processamento e comunicação da solução.

No que se refere aos serviços externos, o diagrama mostra a integração com a plataforma de mensagens WhatsApp, com a infraestrutura de filas e *workers* utilizada no processamento assíncrono de alertas e notificações, com o banco de dados MySQL e com o armazenamento de arquivos em S3 ou disco local. O *backend* publica eventos e dispara *jobs* assíncronos, enquanto a

fila de processamento interage com o *chatbot* para o envio de mensagens ao cidadão. O *frontend*, por sua vez, além das requisições REST/JSON via HTTPS, também mantém comunicação com o servidor *WebSocket* para receber alertas e lembretes em tempo real.

Dessa forma, o Diagrama de Contexto estabelece uma visão macroscópica atualizada da arquitetura, mostrando que o Gabinete Inteligente não se resume mais à interação tradicional entre *frontend* e *backend*. A solução passou a incorporar explicitamente o *chatbot* como canal de atendimento ao cidadão e o *WebSocket* como mecanismo de comunicação em tempo real, preservando, contudo, a centralização das regras de negócio no *backend* principal e mantendo coerência com o funcionamento integrado de toda a plataforma.

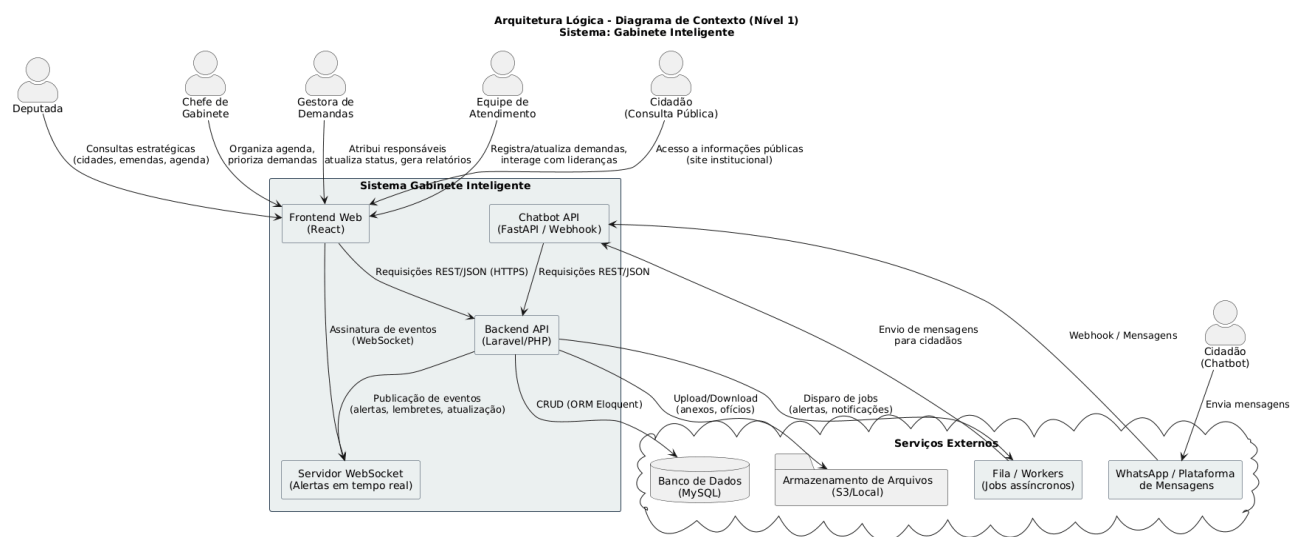


Figura 65 – Diagrama de Contexto

A Figura 66 apresenta o Diagrama de Contêineres (Nível 2), aprofundando a visão estrutural do sistema e detalhando os principais blocos de execução que compõem a solução. Nesse nível, a arquitetura deixa de ser observada apenas como um conjunto de atores e grandes interfaces e passa a explicitar os contêineres responsáveis pela execução das funcionalidades internas do Gabinete Inteligente.

O primeiro contêiner é o *Frontend Web (React)*, executado no navegador do usuário. Ele é responsável pela interface da aplicação, navegação entre páginas, consumo da API REST, interação com os módulos administrativos e exibição de alertas em tempo real. Além da comunicação tradicional via HTTPS com a API principal, esse contêiner também mantém conexão com o servidor *WebSocket*, permitindo a recepção imediata de *pop-ups*, alertas de demandas e lembretes da agenda.

O segundo contêiner é a API REST em Laravel/PHP, que centraliza as regras de negócio do sistema. Esse contêiner é organizado em módulos internos, incluindo autenticação, demandas, emendas, cidades e

instituições, agenda, alertas, *chatbot* e relatórios. O módulo de autenticação controla os fluxos de login, geração e validação de *tokens*, bem como as regras de autorização. O módulo de demandas trata da criação, atualização, descarte, histórico e vinculação de cidadãos. O módulo de emendas concentra o cadastro e a atualização das emendas parlamentares. O módulo de cidades e instituições administra os dados de referência compartilhados pelo restante do sistema. O módulo de agenda gerencia eventos e seus lembretes automáticos. O módulo de alertas concentra a criação e o encaminhamento de notificações internas e externas. Já o módulo de relatórios responde pela consolidação de dados e geração de exportações.

O terceiro contêiner é a *Chatbot* API, implementada em FastAPI, responsável pelo atendimento automatizado via plataformas de mensageria. Esse contêiner recebe webhooks oriundos do WhatsApp, conduz o fluxo conversacional, identifica o cidadão pelo telefone, executa validações sobre a mensagem recebida e encaminha os dados necessários ao *backend* principal. Diferentemente de uma camada meramente passiva, o *chatbot* possui responsabilidades próprias relacionadas ao canal de atendimento, como manutenção do contexto da conversa, integração com a API do WhatsApp e execução das validações de idioma, conteúdo ofensivo e similaridade. Ainda assim, as regras centrais de negócio continuam concentradas na API Laravel, preservando a coerência do sistema.

O quarto contêiner é o Servidor *WebSocket*, responsável pela distribuição de alertas em tempo real para o *frontend*. Sua função é permitir que gestores, responsáveis e demais usuários internos recebam imediatamente notificações de alteração em demandas e lembretes da agenda, sem depender de novas consultas manuais à API REST. Esse componente reforça a responsividade operacional da aplicação.

O quinto contêiner relevante é o *Worker* de Filas, responsável pelo processamento assíncrono de *jobs* e notificações. Ele trata o envio dos alertas gerados pelo sistema, tanto para o *frontend* por meio do *WebSocket* quanto para o cidadão por meio do *chatbot* e da plataforma de mensagens. A presença desse contêiner permite desacoplar a execução das ações de notificação do fluxo principal de uso da aplicação, favorecendo escalabilidade e melhor tempo de resposta.

Esses contêineres se integram com componentes de infraestrutura igualmente relevantes. O banco de dados MySQL armazena as entidades centrais do sistema, como usuários, demandas, cidadãos, sessões de *chatbot*, alertas, eventos, emendas e demais registros persistentes. O armazenamento de arquivos em S3 ou disco local é utilizado para anexos, ofícios e mídias relacionadas ao conteúdo do *site*. A plataforma WhatsApp atua como serviço externo de mensageria, permitindo o recebimento e o envio de mensagens aos cidadãos. Por fim, a infraestrutura de filas e *workers* sustenta o processamento assíncrono dos alertas e lembretes, integrando-se tanto ao *backend* quanto ao *chatbot*.

O Diagrama de Contêineres demonstra, portanto, que a arquitetura do sistema evoluiu para uma solução modular, com responsabilidades bem distribuídas entre *frontend*, *backend* principal, *chatbot*, serviço de comunicação em tempo real e processamento assíncrono. Essa decomposição torna a aplicação mais clara do

ponto de vista arquitetural, mais aderente ao comportamento efetivamente implementado e mais preparada para acomodar integrações e expansões futuras sem concentrar todas as funções em um único ponto da solução.

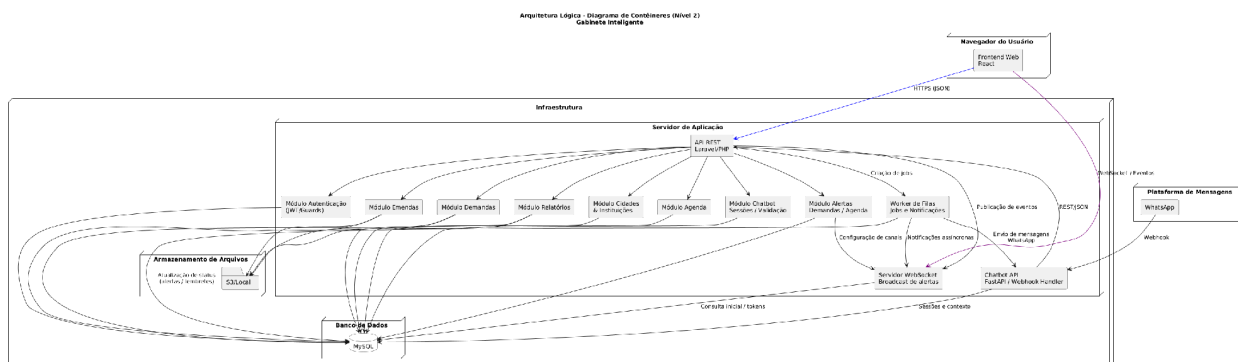


Figura 66 – Diagrama de Contêineres

Além da modelagem em níveis mais altos por meio do C4 Model, o sistema Gabinete Inteligente também foi projetado considerando boas práticas de arquitetura em nível micro, especialmente no contêiner *Backend API*. A Figura 67 apresenta essa organização interna, evidenciando uma estrutura baseada em separação de responsabilidades, serviços especializados e composição de componentes desacoplados. Essa abordagem contribui para reduzir acoplamento, melhorar a legibilidade do código e facilitar manutenção, evolução e testes das funcionalidades implementadas.

Na camada de entrada, os Controllers atuam como ponto de recepção das requisições HTTP e concentram apenas a orquestração inicial do fluxo. Em vez de incorporar regras de negócio diretamente, eles delegam o processamento para serviços específicos, preservando uma responsabilidade enxuta e coerente com o papel da camada de interface da aplicação. Em conjunto com os controllers, as classes de Request do Laravel exercem papel importante na arquitetura, pois concentram validações de entrada e regras de autorização, evitando a dispersão dessas verificações por diferentes partes do sistema.

A Service Layer concentra a maior parte da lógica de negócio e foi estruturada com foco em responsabilidade única. Em vez de um único serviço centralizador, o sistema foi dividido em serviços menores e especializados, como DemandService, DemandHistoryService, DemandAlertService, AgendaReminderService, ChatbotDemandService, NewsService e SiteProjectService. Essa separação permite que cada classe trate um aspecto específico do domínio, como atualização de demandas, registro de histórico, criação de alertas, sincronização de lembretes automáticos ou integração com o *chatbot*, tornando a lógica mais coesa e reutilizável.

O acesso aos dados é realizado por meio dos modelos Eloquent, que representam as entidades persistidas no banco e permitem manter uma separação clara entre manipulação de regras de negócio e persistência. Embora a aplicação utilize diretamente os recursos do ORM em vários serviços, essa organização continua preservando um fluxo arquitetural consistente, no qual controllers recebem a requisição, services aplicam as regras e models executam a persistência.

A arquitetura também incorpora padrões de projeto importantes em pontos específicos do *backend*. Um exemplo relevante é o uso do padrão *Factory* no envio de alertas de demandas. A classe `DemandAlertChannelFactory` é responsável por decidir, em tempo de execução, qual canal concreto deve ser utilizado para entregar a notificação, como o canal de *chatbot* ou o canal de alerta em tempo real. A partir disso, implementações distintas, como `ChatbotDemandAlertChannel` e `WebSocketDemandAlertChannel`, tratam comportamentos diferentes sem que o restante do sistema precise conhecer os detalhes de cada canal. Essa decisão melhora a extensibilidade da solução, permitindo adicionar novos meios de notificação com impacto reduzido sobre a lógica já existente.

Outro aspecto importante da arquitetura micro é o tratamento assíncrono de ações secundárias. Em vez de acoplar diretamente o envio de notificações ao fluxo principal de atualização de uma demanda ou disparo de lembretes, o sistema utiliza *jobs* como `ProcessDemandAlertJob` e `ProcessAgendaReminderJob`. Essa estratégia reduz o tempo de resposta das operações principais e organiza melhor tarefas que podem ser executadas posteriormente, como publicação de alertas no *WebSocket* ou envio de mensagens ao cidadão por meio do *chatbot*. Assim, o *backend* adota uma abordagem orientada a eventos e processamento assíncrono para lidar com comportamentos derivados sem sobrecarregar o fluxo principal.

Também merece destaque a existência de serviços transversais e estruturas auxiliares que consolidam preocupações compartilhadas. Regras de autorização são centralizadas em classes de apoio e códigos de permissão, enquanto o envio de notificações externas é encapsulado em clientes específicos, como o `ChatbotNotificationClient`. Esse tipo de encapsulamento evita duplicação de código, melhora a rastreabilidade do comportamento do sistema e torna mais clara a fronteira entre regras de negócio e integrações externas.

Dessa forma, a arquitetura em nível micro do *backend* demonstra que a solução não foi organizada apenas em módulos funcionais, mas também em componentes com responsabilidades bem definidas, aplicação de injeção de dependência, uso de processamento assíncrono e adoção de

padrões como *Factory* para tratamento de variações comportamentais. Essas decisões sustentam, no nível de implementação, a modularidade e a consistência arquitetural apresentadas nos níveis mais altos do modelo.

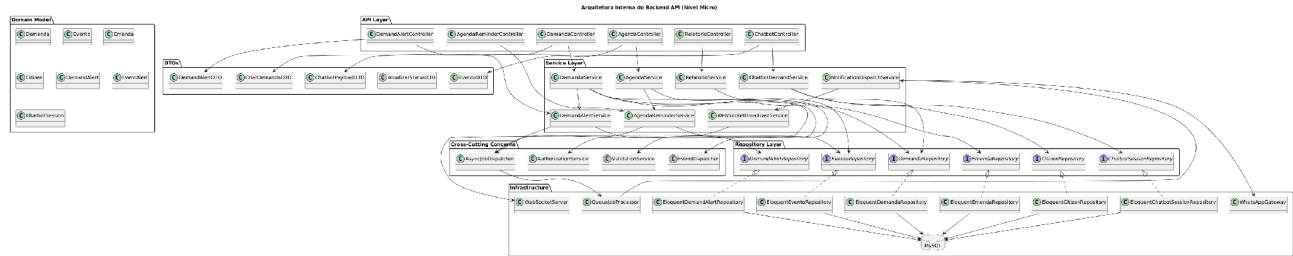


Figura 67 – Arquitetura *Backend*

De modo geral, a arquitetura do projeto foi concebida para priorizar organização, segurança, reutilização e possibilidade de evolução. Um dos pontos mais relevantes é a separação clara entre os diferentes papéis de cada componente da solução. O *frontend* é responsável pela experiência do usuário e pela apresentação das informações; o *backend* centraliza as regras de negócio, validações, permissões e persistência; o *chatbot* atua apenas como canal de atendimento e integração; e os mecanismos de fila e notificações em tempo real tratam comportamentos assíncronos sem sobrecarregar o fluxo principal da aplicação. Essa divisão torna o sistema mais legível, mais fácil de manter e mais preparado para receber novas funcionalidades ao longo do tempo.

Um aspecto importante dessa arquitetura é o fato de o *chatbot* não possuir acesso direto aos dados do sistema. Em vez de consultar ou alterar o banco de dados de forma independente, ele se comunica com o *backend* por meio de APIs específicas, reutilizando os mesmos fluxos de negócio aplicados ao restante da plataforma. Essa decisão fortalece a segurança da solução, evita duplicação de regras, reduz inconsistências e garante que toda operação sensível continue concentrada em um único núcleo de negócio. Na prática, isso significa que o *chatbot* funciona como um canal controlado de entrada e saída de informações, e não como um subsistema isolado com regras próprias dispersas.

No *frontend*, a aplicação também se destaca por adotar uma organização componentizada, com separação entre componentes reutilizáveis de núcleo e componentes específicos por domínio. Essa estratégia melhora a padronização visual, reduz repetição de código e facilita a manutenção da interface. Elementos como botões, inputs, selects, cards, modais e estruturas de listagem seguem uma base comum, permitindo consistência entre as telas e maior previsibilidade no

desenvolvimento. Além disso, a definição clara de uma paleta de cores institucional, com variações bem estabelecidas para estados como sucesso, alerta, perigo e prioridade, contribui para uma experiência visual mais coesa, profissional e adequada ao contexto do gabinete.

Outro ponto que valoriza o projeto é a preocupação em alinhar decisões visuais e estruturais. O *frontend* não foi tratado apenas como uma camada funcional, mas também como uma interface com identidade própria, com cores, estados, componentes e padrões pensados para manter consistência em diferentes módulos. Isso reforça a sensação de unidade do sistema e transmite maior maturidade de produto, especialmente em um projeto que reúne áreas distintas como demandas, agenda, CMS, emendas, projetos de lei e atendimento ao cidadão.

A arquitetura também demonstra maturidade ao incorporar mecanismos de comunicação assíncrona e tempo real. O uso de filas para processamento de alertas e de *WebSocket* para atualização imediata da interface torna a solução mais responsiva e mais próxima de um ambiente real de operação. Ao mesmo tempo, essas responsabilidades ficam desacopladas da execução principal do sistema, o que melhora desempenho, organização e escalabilidade.

Por fim, o projeto se destaca por combinar boas decisões técnicas com coerência arquitetural em todos os níveis. A separação entre canais de atendimento, o encapsulamento das regras de negócio no *backend*, o cuidado com componentização no *frontend*, a definição visual consistente e a integração controlada entre serviços mostram que a solução não foi construída apenas para funcionar, mas para funcionar de forma organizada, segura e sustentável. Isso fortalece a qualidade do trabalho e evidencia uma preocupação real com arquitetura de software, e não apenas com implementação pontual de funcionalidades.

3.5 Diagramas de Atividade

A presente subseção descreve o Diagrama de Atividades elaborado para o sistema Gabinete Virtual. O objetivo deste diagrama é apresentar o fluxo principal de gerenciamento de demandas de forma aderente ao comportamento atual da aplicação. O diagrama inicia com a autenticação e a validação de permissões, passa pela abertura da demanda e avança para os pontos de decisão relacionados à vinculação com cidadão, consistência dos dados e atualizações posteriores. Em vez de separar atribuição de responsável, definição de prazo, anexação de ofício e alteração de status como fluxos

independentes, o modelo passa a tratá-los como formas de atualização da demanda, capazes de gerar histórico e notificações. Também são contemplados os cenários de descarte automatizado, envio de alertas e conclusão do atendimento. Dessa forma, a Figura 68 sintetiza o ciclo de vida da demanda de maneira mais fiel à implementação atual do sistema.

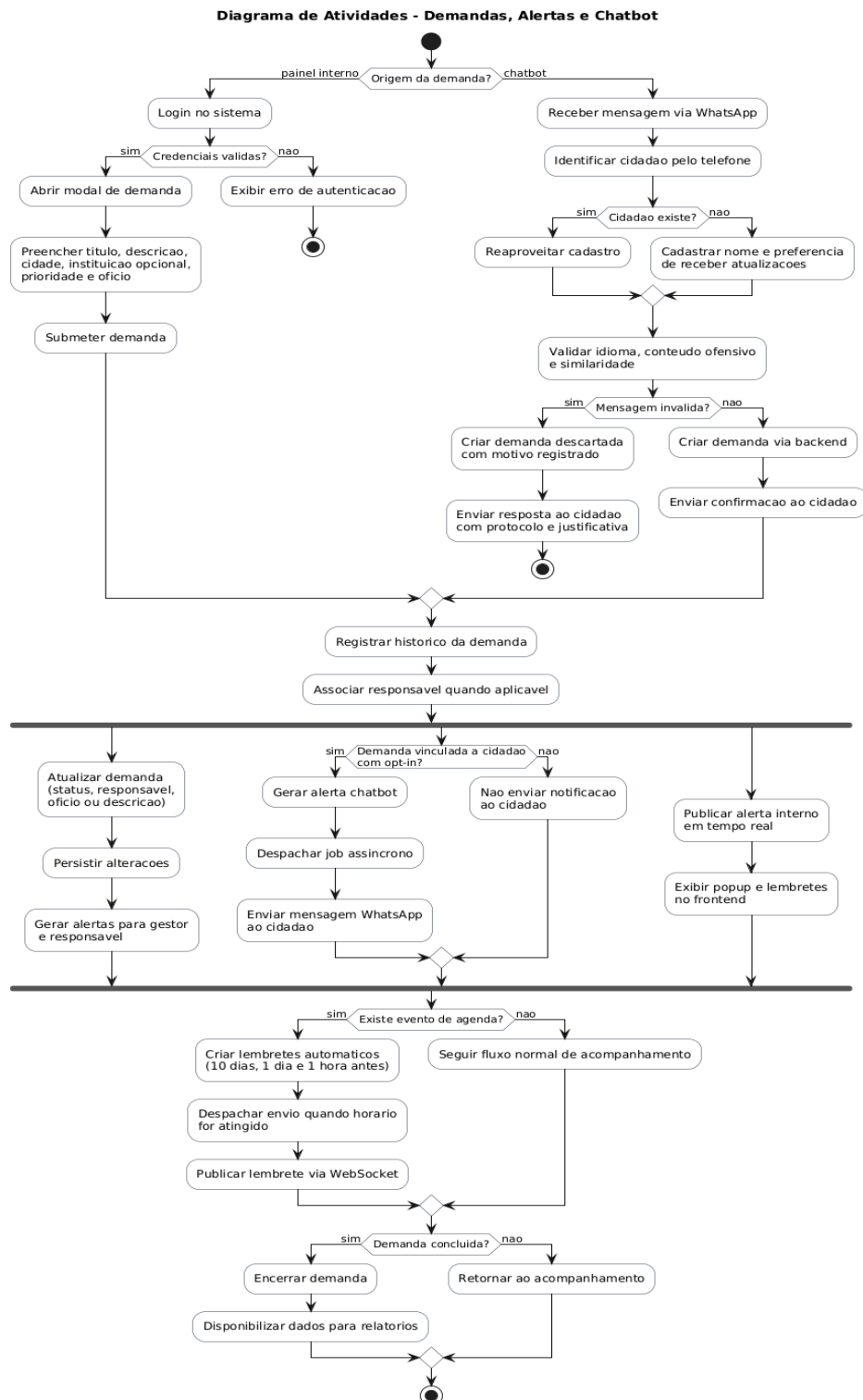


Figura 68 – Diagrama de Atividades

3.6 Diagrama de Componentes e Implantação.

Esta seção apresenta a descrição dos Diagramas de Componentes e de Implantação desenvolvidos para o sistema Gabinete Virtual. Esses diagramas representam a visão estrutural e física da aplicação, evidenciando como os módulos internos se organizam e como o sistema é distribuído em ambiente de execução.

O Diagrama de Componentes, apresentado na Figura 70, detalha a organização modular do *backend* desenvolvido em Laravel. O sistema é estruturado em subsistemas bem definidos, incluindo controladores, serviços de aplicação, mecanismos de autenticação e autorização, domínio e persistência, além da infraestrutura de suporte. Os controladores atuam como ponto de entrada das requisições, delegando a lógica de negócio para os serviços correspondentes. A persistência é centralizada nos modelos Eloquent e no banco de dados MySQL, enquanto serviços de armazenamento e e-mail são acessados por meio de componentes de infraestrutura.

O diagrama também evidencia a integração do *Chatbot* como um componente adicional do *backend*, responsável por receber mensagens provenientes da plataforma de mensageria instantânea (WhatsApp) e encaminhar comandos aos serviços internos já existentes, como Demandas, Relatórios e Autenticação. Essa abordagem garante reutilização da lógica de negócio e evita duplicação de responsabilidades, mantendo a coerência arquitetural do sistema.

O Diagrama de Implantação, apresentado na Figura 69, descreve a distribuição física do sistema em ambiente de execução. Nele, observa-se a separação entre o *frontend* hospedado como uma aplicação React (SPA), o servidor de aplicação que executa a API Laravel sobre PHP-FPM, e os serviços externos responsáveis por persistência de dados, armazenamento de arquivos e envio de e-mails. Adicionalmente, o diagrama contempla a plataforma de mensagens (WhatsApp) como um canal externo de comunicação, integrada ao *backend* por meio de webhooks seguros.

Ambos os diagramas reforçam que toda a comunicação ocorre por meio de protocolos seguros (HTTPS) e que o *backend* atua como núcleo central do sistema, concentrando as regras de negócio e garantindo consistência, segurança e escalabilidade, independentemente do canal de acesso utilizado.

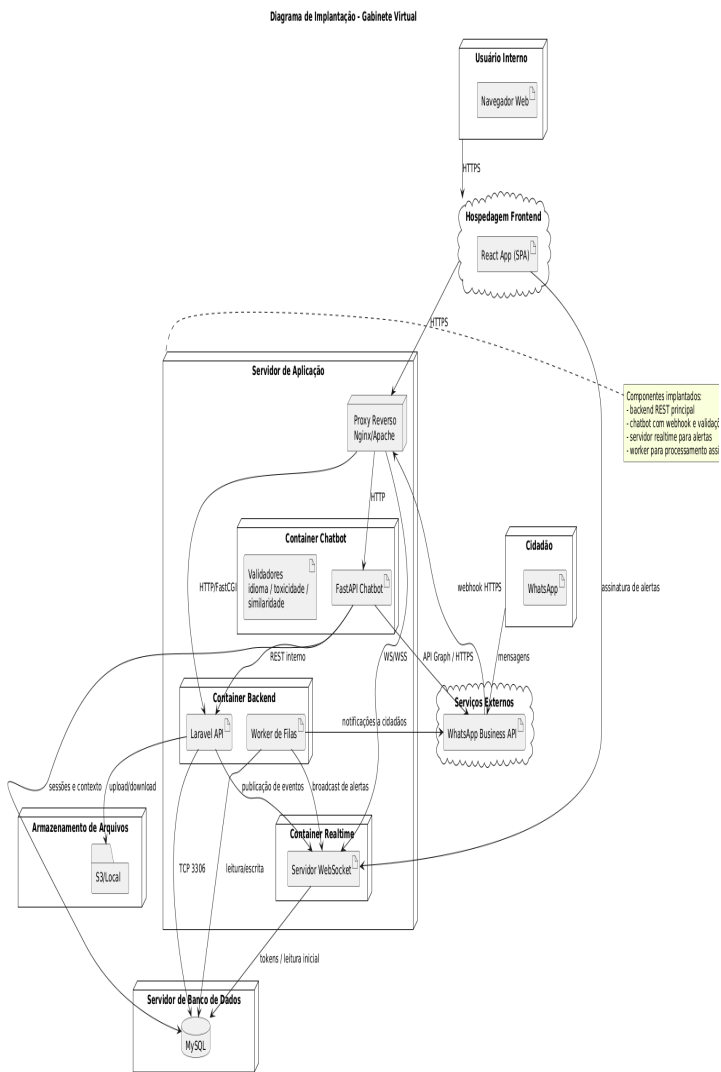


Figura 69 – Diagrama de Implantação

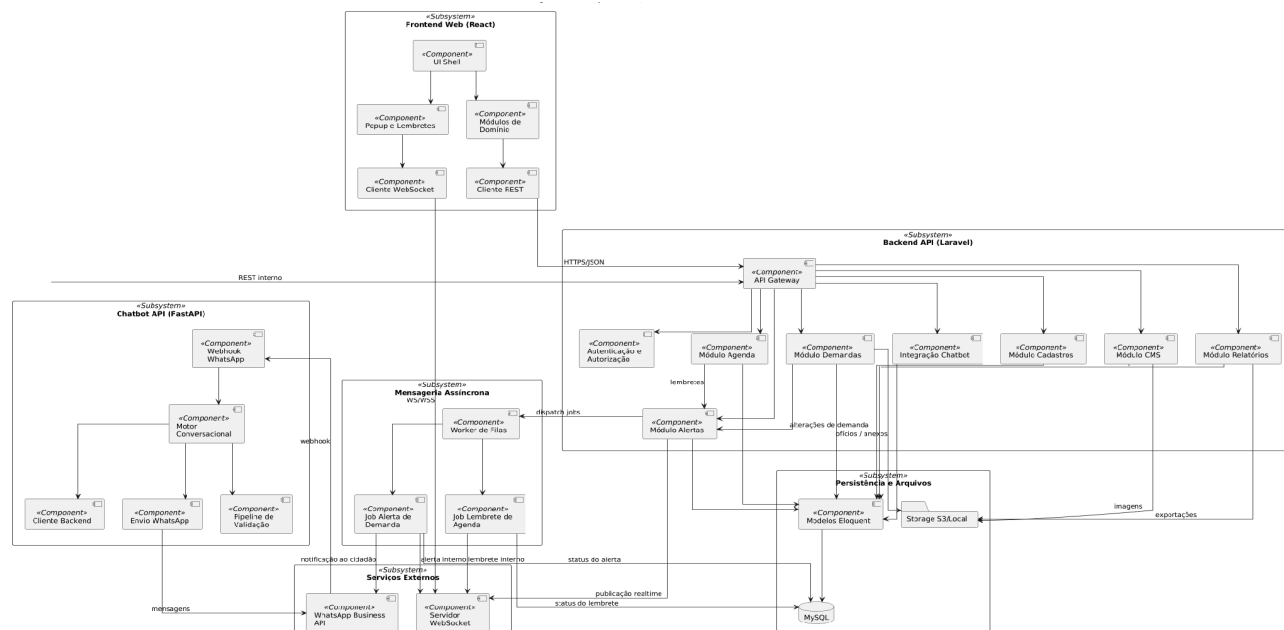


Figura 70 – Diagrama de Componentes

4. Projeto de Interface com Usuário

O projeto de interface define a experiência do usuário e o *design* das telas que compõem o sistema Gabinete Virtual. Nesta seção, são apresentados *wireframes* das principais telas do sistema, que servem como protótipos visuais para orientar a implementação do *frontend* React.

Os *wireframes* foram elaborados considerando os requisitos de usabilidade levantados nas personas, priorizando simplicidade, clareza e eficiência no uso diário do sistema.

4.1 Esboço das Interfaces

As imagens abaixo representam um esboço das principais telas do sistema. Foram feitos *wireframes* para permitir uma visualização inicial do conceito e usabilidade da aplicação. Portanto, ainda que bem desenhados, podem não representar a interface final do sistema.

A Figura 71 representa a tela de autenticação do sistema, primeira interface de contato de todos os usuários internos com a plataforma. Esta tela apresenta um layout centralizado e minimalista,

contendo campos para e-mail institucional e senha, além de um botão primário para efetuar o login. O design prioriza simplicidade e clareza, reduzindo barreiras de entrada ao sistema.

Perfis com acesso: Todos os perfis internos (Deputada, Chefe de Gabinete, Gestora de Demandas, Equipe de Atendimento e Funcionários).

Elementos principais: Campos de e-mail e senha, botão "Entrar", credenciais de demonstração para ambiente de testes.

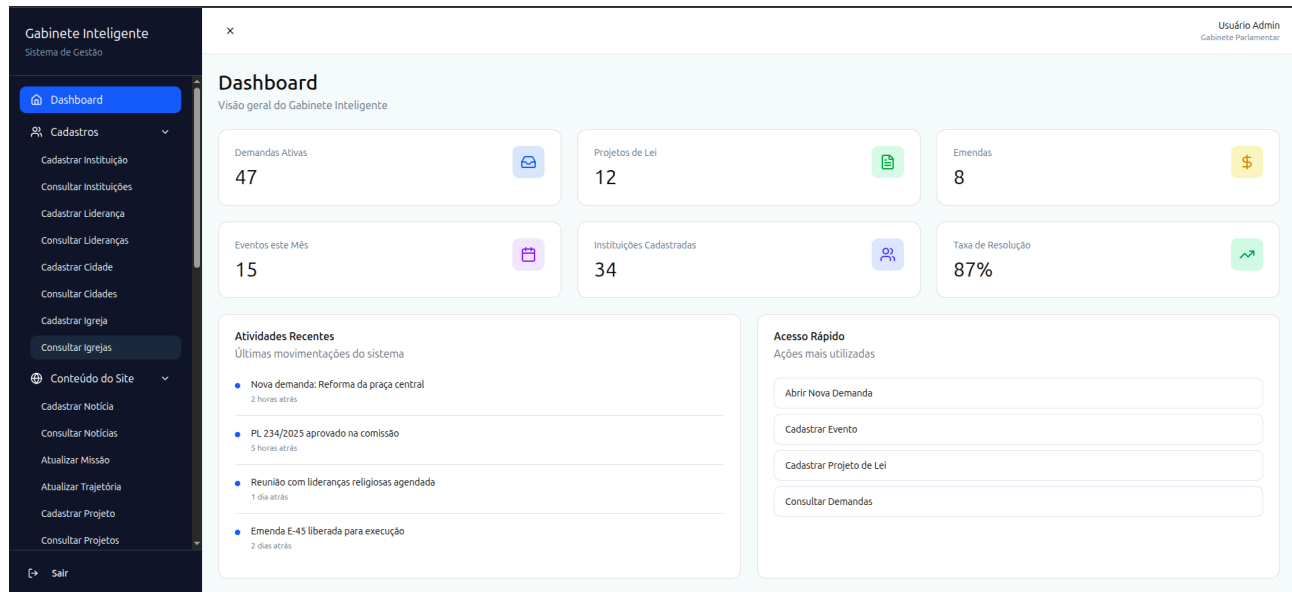


Figura 71 – Tela de Login

Como mostra a Figura 72, o *Dashboard* constitui a tela inicial após autenticação bem-sucedida, oferecendo uma visão consolidada das principais métricas do gabinete. A interface exibe cards informativos com totalizadores de demandas ativas, projetos de lei, emendas, eventos programados, instituições cadastradas e taxa de resolução. Complementam a tela painéis de atividades recentes e atalhos rápidos para as funcionalidades mais utilizadas, permitindo navegação eficiente.

Perfis com acesso: Deputada, Chefe de Gabinete, Gestora de Demandas.

Elementos principais: Indicadores quantitativos, painel de atividades recentes, menu lateral de navegação, atalhos para ações frequentes.

Figura 72 – Tela Inicial - *Dashboard*

A Figura 73 apresenta o formulário de cadastro de instituições, contendo campos para nome da instituição, tipo (Prefeitura, Câmara Municipal, Órgão Estadual, ONG/Associação, Sindicato) e município vinculado. A interface inclui botões de ação primária "Cadastrar Instituição" e secundária "Ver Todas", além de validações em tempo real.

Perfis com acesso: Chefe de Gabinete, Gestora de Demandas, Funcionários.

Elementos principais: Campos de texto, seletores de tipo e município, botões de ação.

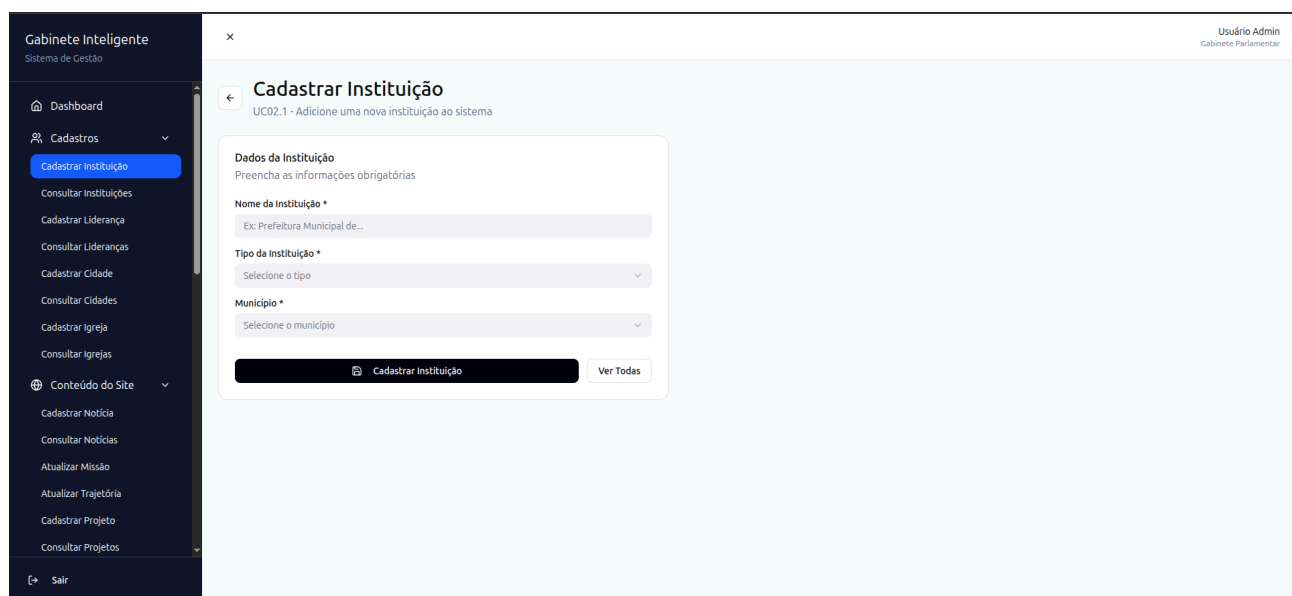


Figura 73 – Tela de Cadastro de Instituição

Na Figura 74, observa-se a interface de consulta que permite filtrar instituições por nome, tipo e município. Os resultados são exibidos em tabela com colunas de nome, tipo, município e ações (editar/excluir). Um botão "Nova Instituição" possibilita acesso rápido ao cadastro.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtros de busca, tabela de resultados, botões de ação por linha, botão de novo cadastro.

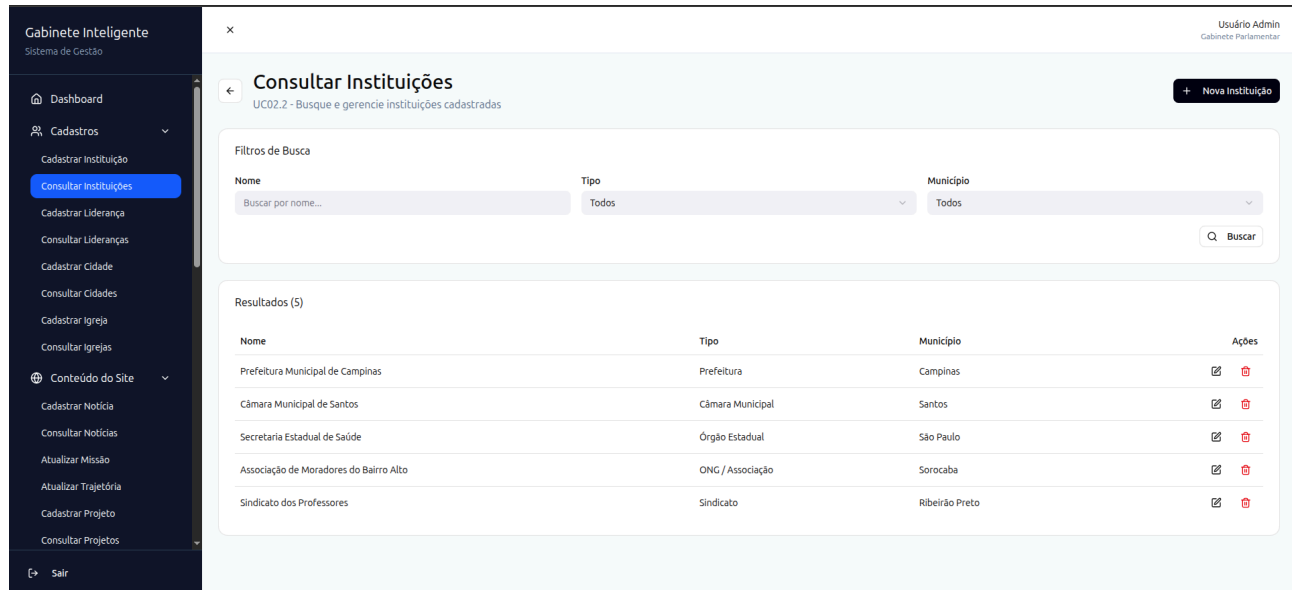


Figura 74 – Tela de Consulta de Instituições

A Figura 75 demonstra o formulário para registro de lideranças políticas e comunitárias, com campos para nome completo, cargo/função, telefone e instituição vinculada. A vinculação com instituição pré-cadastrada garante integridade referencial.

Perfis com acesso: Chefe de Gabinete, Gestora de Demandas, Funcionários.

Elementos principais: Campos de identificação, seletor de instituição, campo de telefone com máscara.

A interface de 'Cadastrar Liderança' apresenta um formulário para adicionar uma nova liderança ao sistema. O formulário contém campos para 'Nome Completo', 'Cargo / Função', 'Telefone' e 'Instituição'. Abaixo dos campos, há um botão 'Cadastrar Liderança' e um link 'Ver Todas'. O menu lateral à esquerda contém opções como 'Dashboard', 'Cadastros', 'Cadastrar Instituição', 'Consultar Instituições', 'Cadastrar Liderança' (destacado), 'Consultar Lideranças', 'Cadastrar Cidade', 'Consultar Cidades', 'Cadastrar Igreja', 'Consultar Igrejas', 'Conteúdo do Site', 'Cadastrar Notícia', 'Consultar Notícias', 'Atualizar Missão', 'Atualizar Trajetória', 'Cadastrar Projeto', 'Consultar Projetos' e 'Sair'. O topo da interface mostra o nome do usuário 'Usuário Admin' e o cargo 'Gabinete Parlamentar'.

Cadastrar Liderança
UC02.3 - Adicione uma nova liderança ao sistema

Dados da Liderança
Preencha as informações de contato

Nome Completo *
Ex: João Silva

Cargo / Função *
Ex: Prefeito, Secretário, Presidente

Telefone *
(00) 00000-0000

Instituição *
Selecione a instituição

Cadastrar Liderança **Ver Todas**

Figura 75 – Tela de Cadastro de Lideranças

Como mostra a Figura 76, a consulta de lideranças oferece filtros por nome e cargo, apresentando resultados tabelados com informações de nome, cargo, instituição, telefone e ações disponíveis.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtros de busca múltiplos, tabela com dados de contato, ícones de ação.

A interface de 'Consultar Lideranças' apresenta uma seção de filtros de busca com campos para 'Nome' e 'Cargo'. Abaixo dos filtros, há uma tabela com 5 resultados. Cada linha da tabela contém as informações de nome, cargo, instituição, telefone e ações disponíveis. O menu lateral à esquerda contém opções como 'Dashboard', 'Cadastros', 'Cadastrar Instituição', 'Consultar Instituições', 'Cadastrar Liderança', 'Consultar Lideranças' (destacado), 'Cadastrar Cidade', 'Consultar Cidades', 'Cadastrar Igreja', 'Consultar Igrejas', 'Conteúdo do Site', 'Cadastrar Notícia', 'Consultar Notícias', 'Atualizar Missão', 'Atualizar Trajetória', 'Cadastrar Projeto', 'Consultar Projetos' e 'Sair'. O topo da interface mostra o nome do usuário 'Usuário Admin' e o cargo 'Gabinete Parlamentar'.

Consultar Lideranças
UC02.4 - Busque e gerencie lideranças cadastradas

Filtros de Busca

Nome
Buscar por nome...

Cargo
Buscar por cargo...

Buscar

Resultados (5)

Nome	Cargo	Instituição	Telefone	Ações
João Silva	Prefeito	Prefeitura Municipal de Campinas	(19) 99876-5432	 
Maria Santos	Vereadora	Câmara Municipal de Santos	(13) 98765-4321	 
Pedro Oliveira	Secretário de Saúde	Secretaria Estadual de Saúde	(11) 97654-3210	 
Ana Costa	Presidente	Associação de Moradores do Bairro Alto	(15) 96543-2109	 
Carlos Ferreira	Diretor Sindical	Sindicato dos Professores	(16) 95432-1098	 

Figura 76 – Tela de Consulta de Lideranças

A Figura 77 apresenta o formulário simplificado para cadastro de municípios, contendo campos para nome da cidade, região administrativa e população estimada. Este cadastro é fundamental para vincular demandas, emendas e instituições.

Perfis com acesso: Chefe de Gabinete, Funcionários.

Elementos principais: Campos de texto, seletor de região, campo numérico de população.

A imagem mostra a interface de usuário do sistema "Gabinete Inteligente". No topo, há uma barra de navegação com o nome do sistema e o perfil do usuário "Usuário Admin - Gabinete Parlamentar". À esquerda, um menu lateral contém opções como "Dashboard", "Cadastros" (com subitens para Instituição, Liderança e Cidade), "Conteúdo do Site" e "Sair". O menu "Cadastros" está expandido, e "Cadastrar Cidade" está selecionado. O formulário principal, intitulado "Cadastrar Cidade" com o subtítulo "UC02.5 - Adicione uma nova cidade ao sistema", contém os seguintes campos: "Nome da Cidade" (com o exemplo "Ex: Campinas"), "Região" (um seletor com o texto "Selecione a região") e "População (estimada)" (com o exemplo "Ex: 12000000"). Abaixo dos campos, há uma instrução "Informe a população aproximada do município". No rodapé do formulário, há dois botões: "Cadastrar Cidade" e "Ver Todas".

Figura 77 – Tela de Cadastro de Cidades

Na Figura 78, a interface de consulta permite buscar cidades por nome, exibindo em tabela as informações de nome, região, população e ações disponíveis.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Campo de busca textual, tabela informativa, indicadores populacionais.

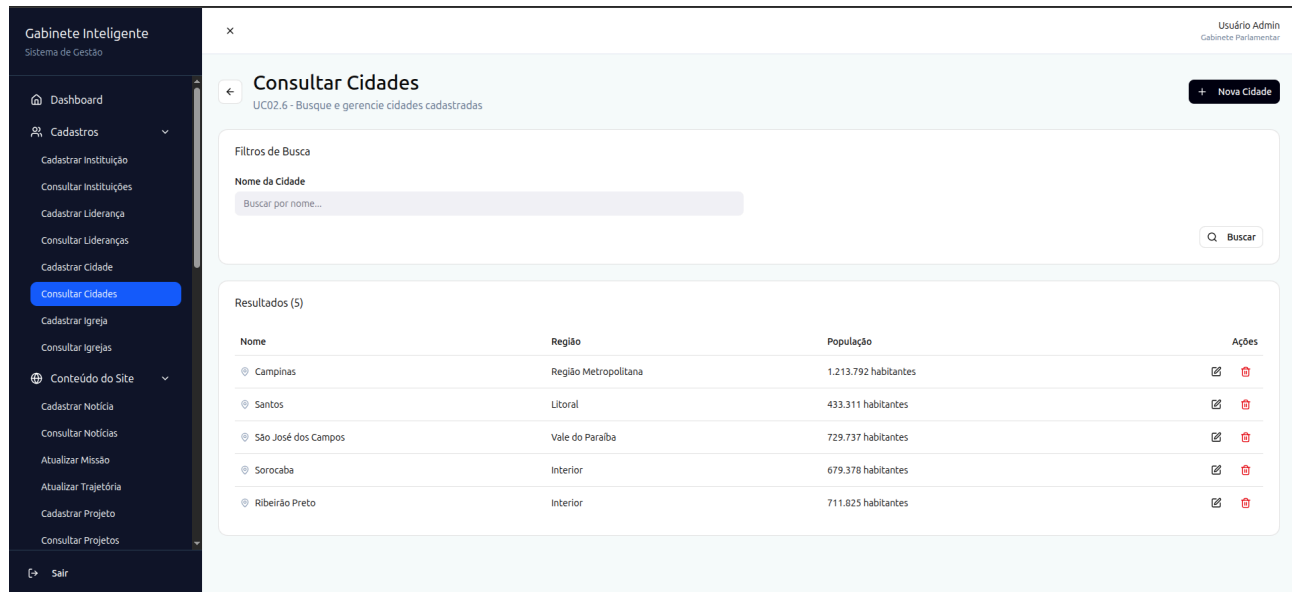


Figura 78 – Tela de Consulta de Cidades

A Figura 79 apresenta o editor para publicação de notícias no portal institucional, contendo campos para título, conteúdo (com formatação rica), data de publicação e autor. O sistema registra automaticamente o usuário responsável pela publicação.

Perfis com acesso: Chefe de Gabinete, Funcionários com permissão editorial.

Elementos principais: Editor de texto rico, campos de metadados, botão de publicação.

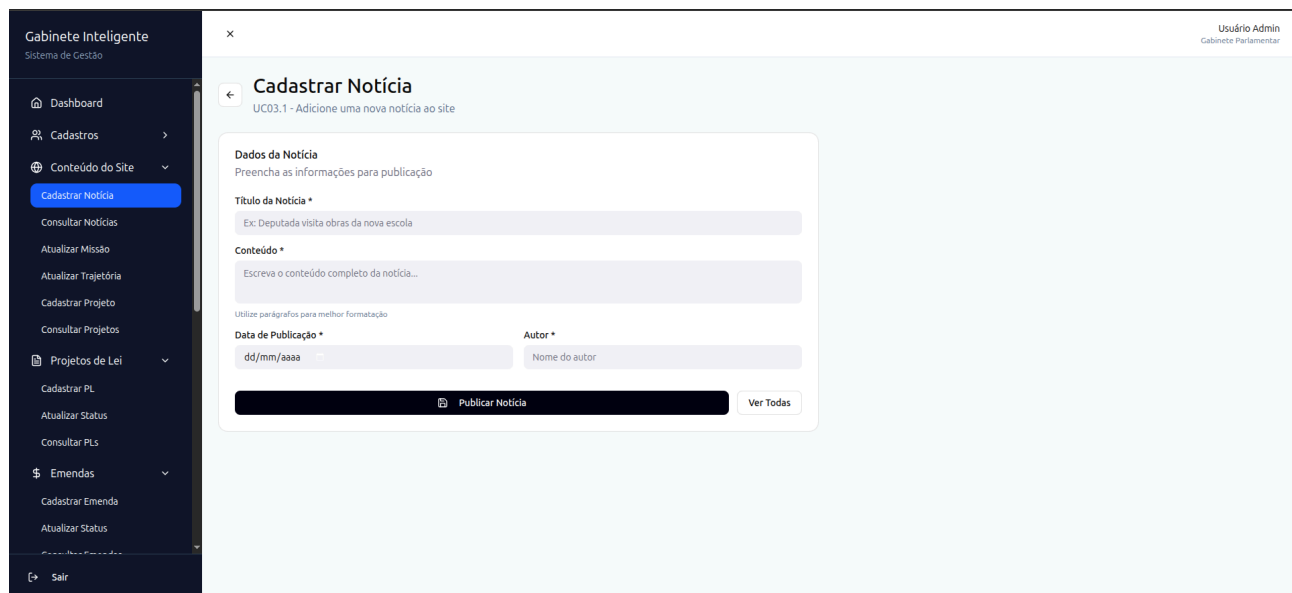
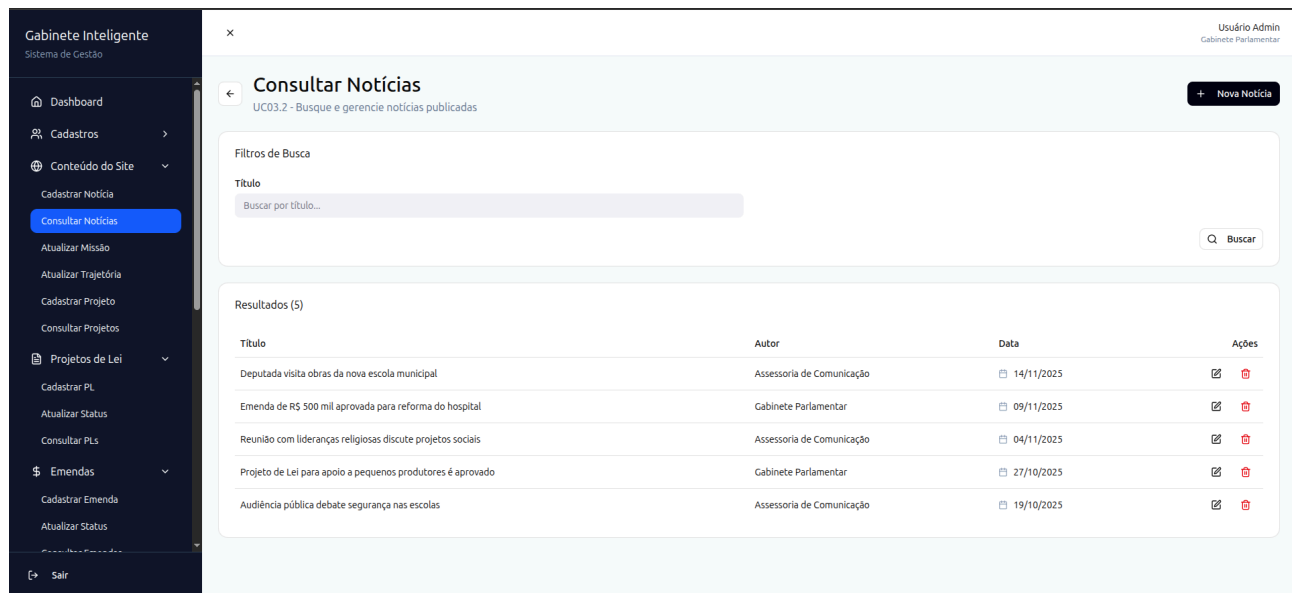


Figura 79 – Tela de Cadastro de Notícias

Na Figura 80, a interface permite buscar notícias por título, exibindo resultados com título, autor, data e ações de edição/exclusão.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtro por título, listagem cronológica, botão de nova notícia.



A Figura 81 demonstra o editor para atualização da seção institucional "Missão" do *site*. A interface apresenta um campo de texto rico com preview em tempo real, permitindo visualizar como o conteúdo será exibido publicamente.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Editor de texto, área de preview, botão de salvar alterações.

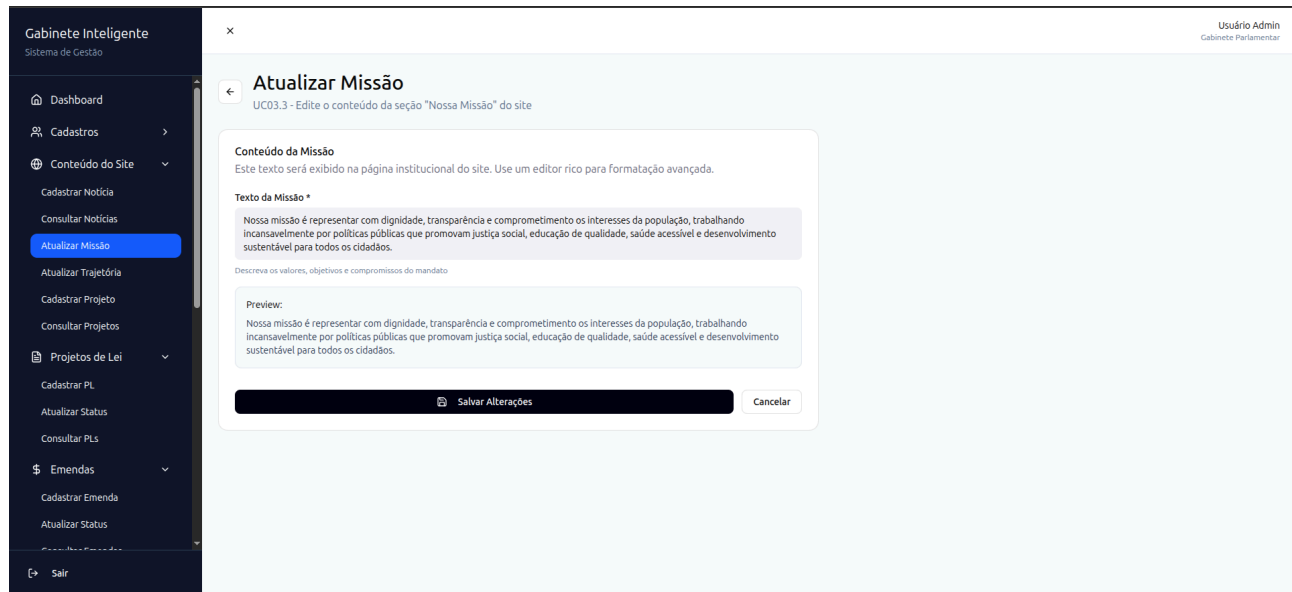


Figura 81 – Tela de Atualização de Missão

Como mostra a Figura 82, similar à tela anterior, esta interface permite editar a seção "Trajetória" do portal, com editor rico e preview.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Editor de texto longo, preview contextual, botão de confirmação.

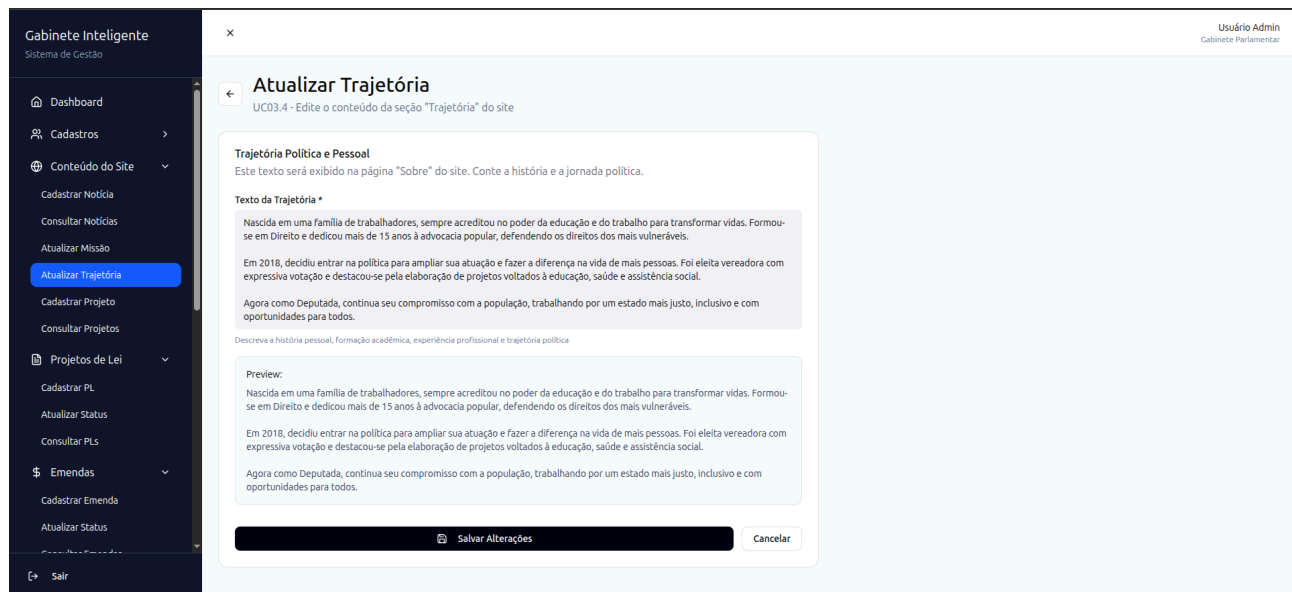


Figura 82 – Tela de Atualização de Trajetória

A Figura 83 apresenta o formulário para cadastro de projetos institucionais a serem exibidos no portal público, com campos para nome do projeto, descrição, município beneficiado e status.

Perfis com acesso: Chefe de Gabinete, Funcionários.

Elementos principais: Campos descritivos, seletor de município e status, botão de cadastro.

Gabinete Inteligente
Sistema de Gestão

Usuário Admin
Gabinete Parlamentar

Cadastrar Projeto do Site
UC03.5 - Adicione um novo projeto para ser exibido no site

Dados do Projeto
Preencha as informações do projeto para publicação no site

Nome do Projeto *
Ex: Reforma da Praça Central

Descrição *
Descreva o projeto, seus objetivos e benefícios para a população...

Município * **Status ***
Selecione o município Seleccione o status

Cadastrar Projeto **Ver Todos**

Figura 83 – Tela de Cadastro de Projetos do Site

Na Figura 84, a consulta permite filtrar projetos por nome, exibindo tabela com nome do projeto, município, status e ações disponíveis.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtro de busca, tabela de projetos, indicadores de status coloridos.

Gabinete Inteligente
Sistema de Gestão

Usuário Admin
Gabinete Parlamentar

Consultar Projetos do Site
UC03.6 - Busque e gerencie projetos exibidos no site

Filtros de Busca
Nome do Projeto
Buscar por nome... **Buscar**

Resultados (5)

Nome do Projeto	Município	Status	Ações
Reforma da Praça Central	Campinas	Em Execução	
Construção de Centro Comunitário	Santos	Em Planejamento	
Revitalização da Escola Municipal	São José dos Campos	Concluído	
Ampliação do Posto de Saúde	Sorocaba	Em Execução	
Iluminação LED nas ruas	Ribeirão Preto	Concluído	

Figura 84 – Tela de Consulta de Projetos do Site

A Figura 85 demonstra o formulário de registro de PLs, contendo campos para número do projeto, descrição ou ementa, status inicial e data de protocolo.

Perfis com acesso: Deputada, Chefe de Gabinete, Funcionários.

Elementos principais: Campo de número, editor de ementa, seletor de status, campo de data.

A imagem mostra a interface de usuário do sistema 'Gabinete Inteligente'. No topo, há uma barra de navegação com o nome do sistema e o perfil do usuário 'Usuário Admin - Gabinete Parlamentar'. À esquerda, um menu lateral contém opções como 'Dashboard', 'Cadastros', 'Conteúdo do Site', 'Projetos de Lei' (com subopções 'Cadastrar PL', 'Atualizar Status' e 'Consultar PLS'), 'Emendas' (com subopções 'Cadastrar Emenda', 'Atualizar Status' e 'Consultar Emendas'), 'Demandas' (com subopções 'Abrir Demanda', 'Atribuir Responsável', 'Registrar Pessoa Atendida' e 'Definir Prazo') e 'Sair'. O painel principal exibe a tela 'Cadastrar Projeto de Lei' com o subtítulo 'UC04.1 - Adicione um novo PL ao sistema'. O formulário contém campos para 'Número do PL' (com exemplo 'Ex: PL 234/2025'), 'Descrição / Ementa' (com instrução 'Descreva o objetivo e conteúdo do projeto de lei...'), 'Status' (menu suspenso com 'Selecione o status') e 'Data de Protocolo' (formato 'dd/mm/aaaa'). No rodapé do formulário, há dois botões: 'Cadastrar PL' e 'Ver Todos'.

Figura 85 – Tela de Cadastro de Projetos de Lei

Como mostra a Figura 86, esta interface permite atualizar o andamento de projetos de lei, selecionando o PL desejado, definindo novo status e adicionando observações sobre a mudança.

Perfis com acesso: Deputada, Chefe de Gabinete, Funcionários.

Elementos principais: Seletor de PL, seletor de status, campo de observações, botão de atualização.

Atualizar Status do PL
UC04.2 - Atualize o status de um Projeto de Lei

Atualização de Status
Selecione o PL e informe o novo status

Projeto de Lei *
Selecione o PL

Novo Status *
Selecione o novo status

Observações
Adicione observações sobre a mudança de status...

Atualizar Status **Ver Todos PLs**

Figura 86 – Tela de Atualização de Status do Projeto de Lei

A Figura 87 apresenta a interface de consulta com filtro por número do PL, exibindo tabela com número, descrição, status e data, além de ações de visualização e edição.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtro numérico, tabela de PLs, badges de status, ações por linha.

Consultar Projetos de Lei
UC04.3 - Busque e gerencie Projetos de Lei

Filtros de Busca

Número do PL
Buscar por número...

Buscar

Resultados (4)

Número	Descrição	Status	Data	Ações
PL 234/2025	Incentivo à agricultura familiar	Em Comissão	14/03/2025	
PL 189/2025	Programa de capacitação profissional	Em Votação	19/02/2025	
PL 156/2024	Melhorias no transporte público	Aprovado	09/11/2024	
PL 098/2024	Política de assistência social	Sancionado	04/09/2024	

Figura 87 – Tela de Consulta de Projetos de Lei

Na Figura 88, o formulário permite registrar emendas parlamentares com número, valor em reais, finalidade, município beneficiado e status inicial.

Perfis com acesso: Deputada, Chefe de Gabinete, Funcionários.

Elementos principais: Campos numéricos, campo monetário, área de texto, seletores.

Figura 88 – Tela de Cadastro de Emenda

A Figura 89 demonstra a interface para mudança de status de emendas, com seletor da emenda, novo status e campo de observações.

Perfis com acesso: Deputada, Chefe de Gabinete, Funcionários.

Elementos principais: Seletor de emenda, seletor de status, campo de observações.

Figura 89 – Tela de Atualização de Status de Emenda

Como mostra a Figura 90, a consulta permite buscar por número da emenda, exibindo tabela com número, finalidade, valor, município, status e ações.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtro de busca, tabela com valores monetários, badges coloridos de status.

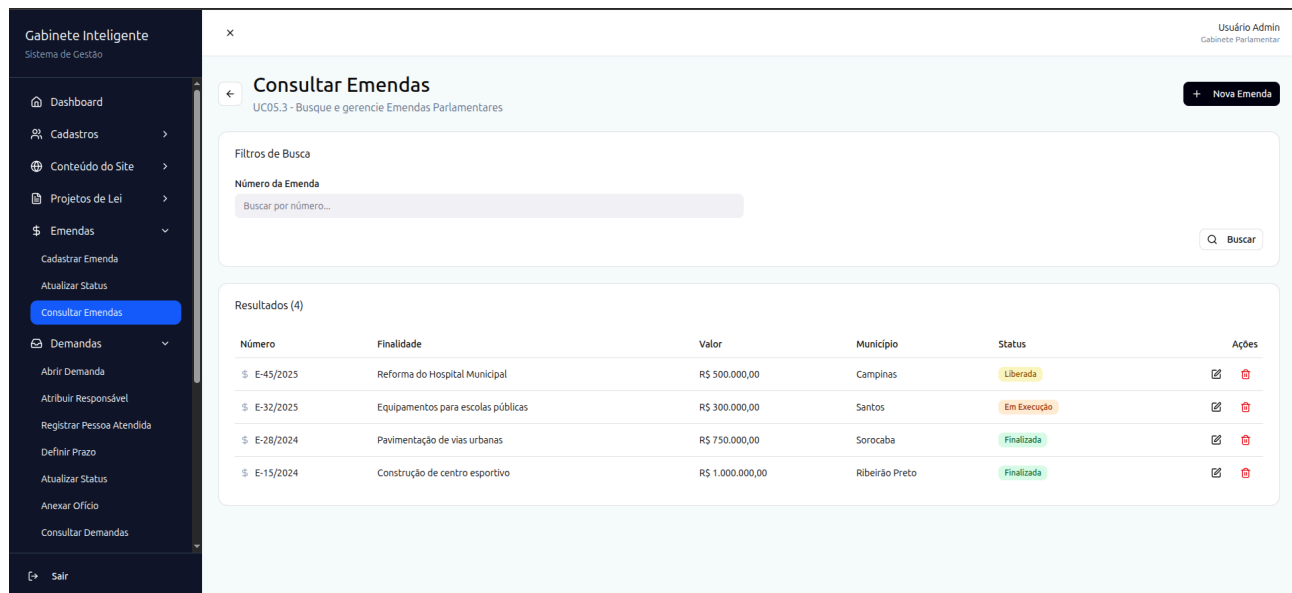


Figura 90 – Tela de Consulta de Emendas

A Figura 91 apresenta o formulário completo para registro de novas demandas, com campos para título, descrição detalhada, município, instituição solicitante e prioridade (Alta, Média, Baixa).

Perfis com acesso: Chefe de Gabinete, Gestora de Demandas, Equipe de Atendimento.

Elementos principais: Campos de texto, seletores múltiplos, indicador de prioridade, botão de abertura.

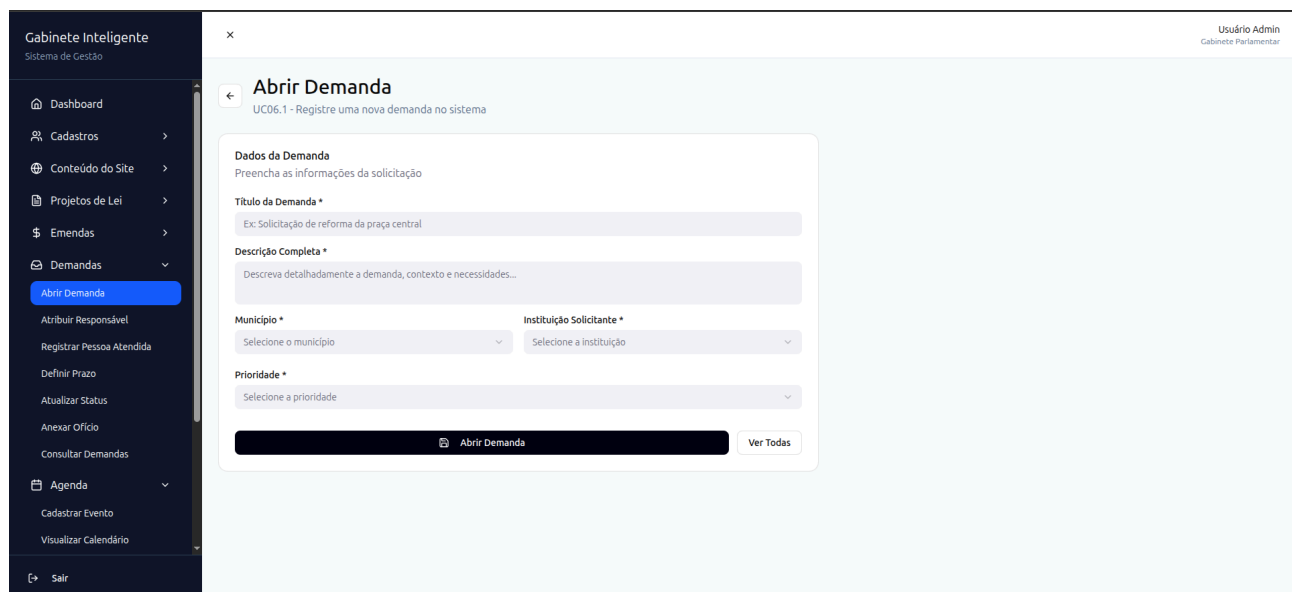


Figura 91 – Tela de Abertura de Demanda

Na Figura 92, a interface permite selecionar uma demanda e atribuir um usuário responsável, com notificação automática ao designado.

Perfis com acesso: Chefe de Gabinete, Gestora de Demandas.

Elementos principais: Seletor de demanda, seletor de usuário responsável, botão de atribuição, aviso de notificação.

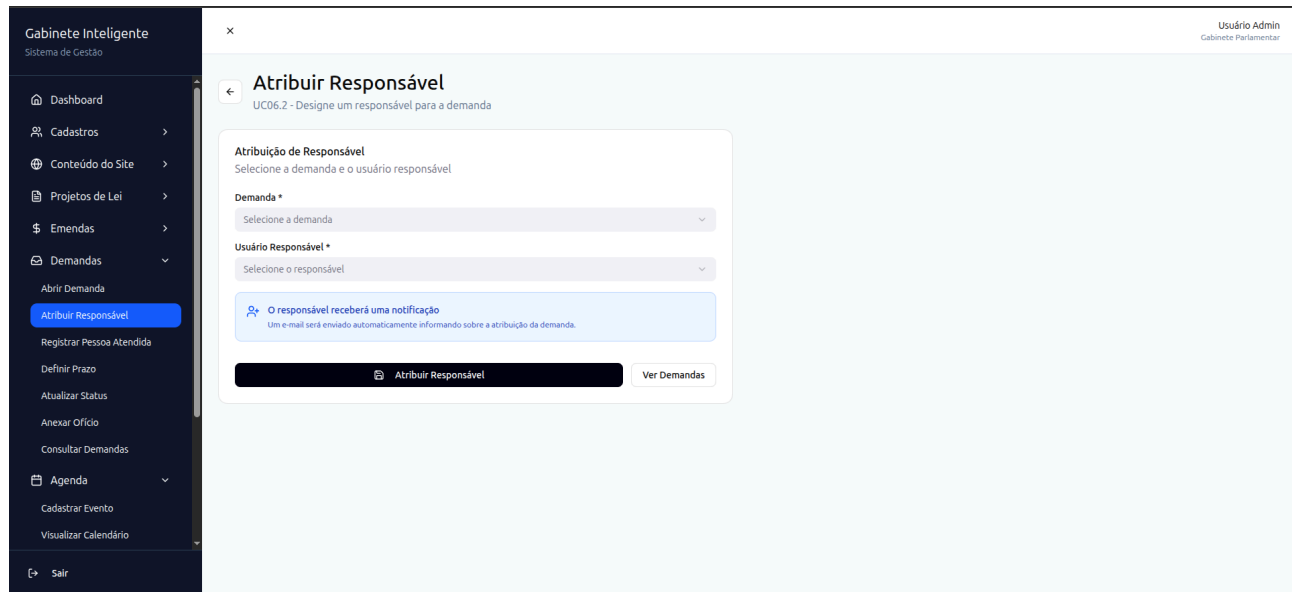


Figura 92 – Tela de Atribuição de Responsável

A Figura 93 demonstra o formulário para vincular dados do cidadão atendido à demanda, com campos para demanda relacionada, nome completo, CPF e telefone.

Perfis com acesso: Gestora de Demandas, Equipe de Atendimento.

Elementos principais: Seletor de demanda, campos de identificação pessoal, botão de registro.

Gabinete Inteligente
Sistema de Gestão

Usuário Admin
Gabinete Parlamentar

Registrar Pessoa Atendida

UC06.3 - Vincule uma pessoa a uma demanda

Dados da Pessoa Atendida
Registre os dados de quem será beneficiado pela demanda

Demanda Relacionada *
Selecione a demanda

Nome Completo *
Ex: Maria da Silva

CPF * 000.000.000-00 **Telefone *** (00) 00000-0000

Registrar Pessoa Ver Demandas

Figura 93 – Tela de Registro de Atendimento

Como mostra a Figura 94, esta tela permite estabelecer data limite para conclusão da demanda, com seletor de demanda, campo de data e aviso sobre lembretes automáticos próximos ao vencimento.

Perfis com acesso: Chefe de Gabinete, Gestora de Demandas.

Elementos principais: Seletor de demanda, campo de data, informativo de notificações, botão de definição.

Gabinete Inteligente
Sistema de Gestão

Usuário Admin
Gabinete Parlamentar

Definir Prazo

UC06.4 - Estabeleça um prazo limite para a demanda

Definição de Prazo
Estabeleça a data limite para conclusão da demanda

Demanda *
Selecione a demanda

Data Limite *
dd/mm/aaaa

Selecione a data limite para resolução da demanda

Lembretes automáticos
O sistema enviará notificações próximo ao vencimento do prazo.

Definir Prazo Ver Demandas

Figura 94 – Tela de Definição de Prazo

A Figura 95 apresenta o formulário para anexar documentos oficiais às demandas, com campos para identificação da demanda, tipo de documento, número do ofício, data de emissão, órgão de destino, assunto, observações e upload do arquivo digital.

Perfis com acesso: Gestora de Demandas, Equipe de Atendimento.

Elementos principais: Seletor de demanda, campos descritivos do documento, área de upload de arquivo.

A imagem mostra a interface de usuário do sistema 'Gabinete Inteligente'. No topo, há uma barra de navegação com o nome do sistema e o perfil do usuário 'Usuário Admin - Gabinete Parlamentar'. À esquerda, um menu lateral contém opções como 'Demandas', 'Agenda' e 'Relatórios'. O formulário principal, intitulado 'Anexar Ofício', está dividido em seções: 'Identificação da Demanda' com campos para 'Demanda', 'Tipo de Documento', 'Número do Ofício', 'Data de Emissão' e 'Órgão de Destino'; 'Conteúdo do Documento' com campos para 'Assunto' e 'Observações'; e 'Upload do Arquivo' com o campo 'Arquivo Digital'.

Figura 95 – Tela de Anexo de Ofício

Na Figura 96, a interface permite consultar as demandas registradas no sistema, com filtros por texto, status, prioridade, responsável e prazo, além de ações para visualização e atualização do registro.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Filtros combinados, tabela detalhada, badges de status e prioridade, ações de visualização e atualização.

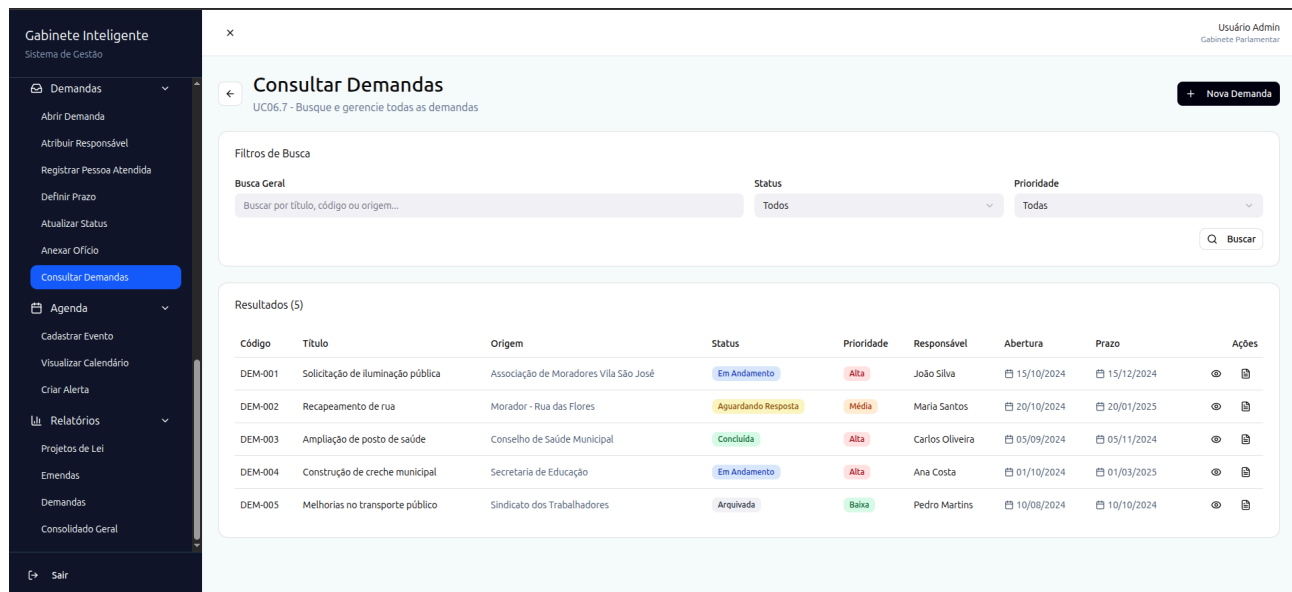


Figura 96 – Tela de Consulta de Demandas

A Figura 97 demonstra o formulário para registro de compromissos na agenda, com campos para título do evento, tipo, local, período, descrição, participantes e cidade relacionada.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Campos de identificação do evento, período, descrição, participantes e ações de salvamento.

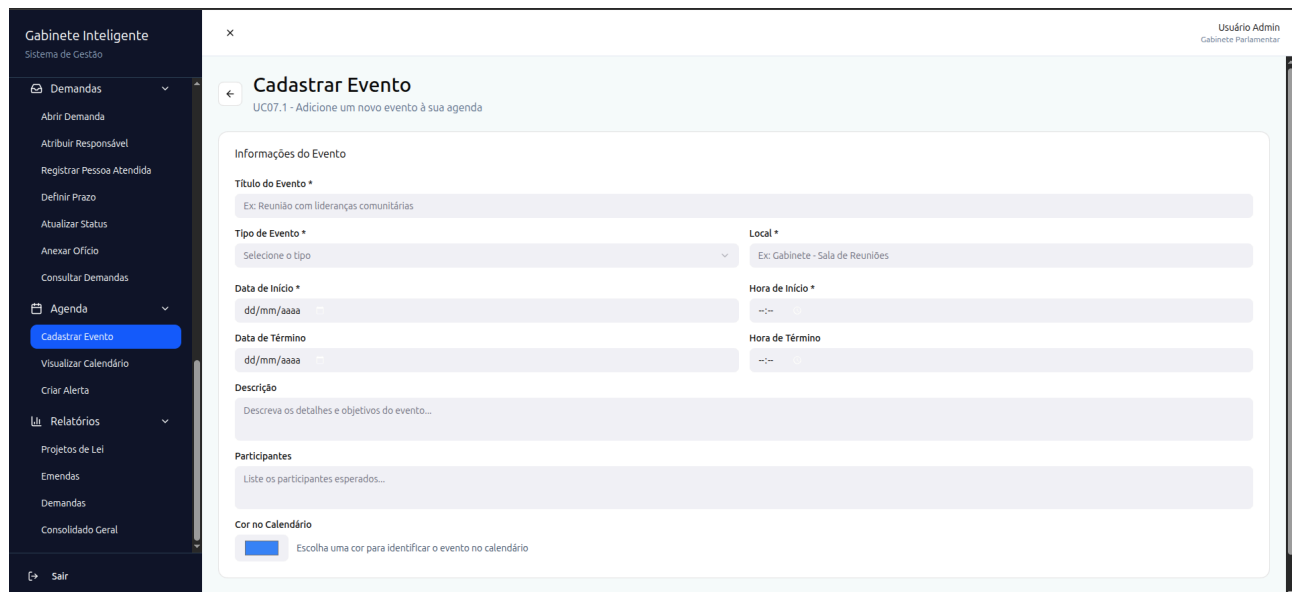


Figura 97 – Tela de Cadastro de Evento

A Figura 98 apresenta a visualização do calendário do gabinete, exibindo os eventos agendados por período e permitindo acompanhar rapidamente os próximos compromissos institucionais.

Perfis com acesso: Todos os perfis internos.

Elementos principais: Calendário mensal interativo, painel de próximos eventos, navegação temporal e listagem de compromissos.

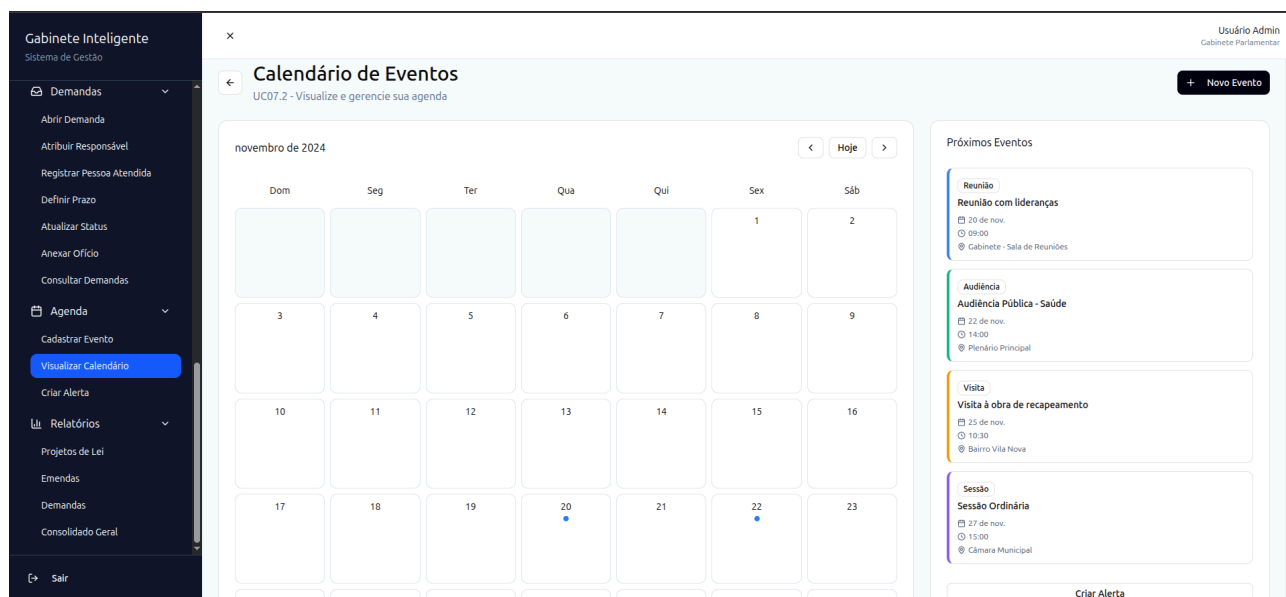
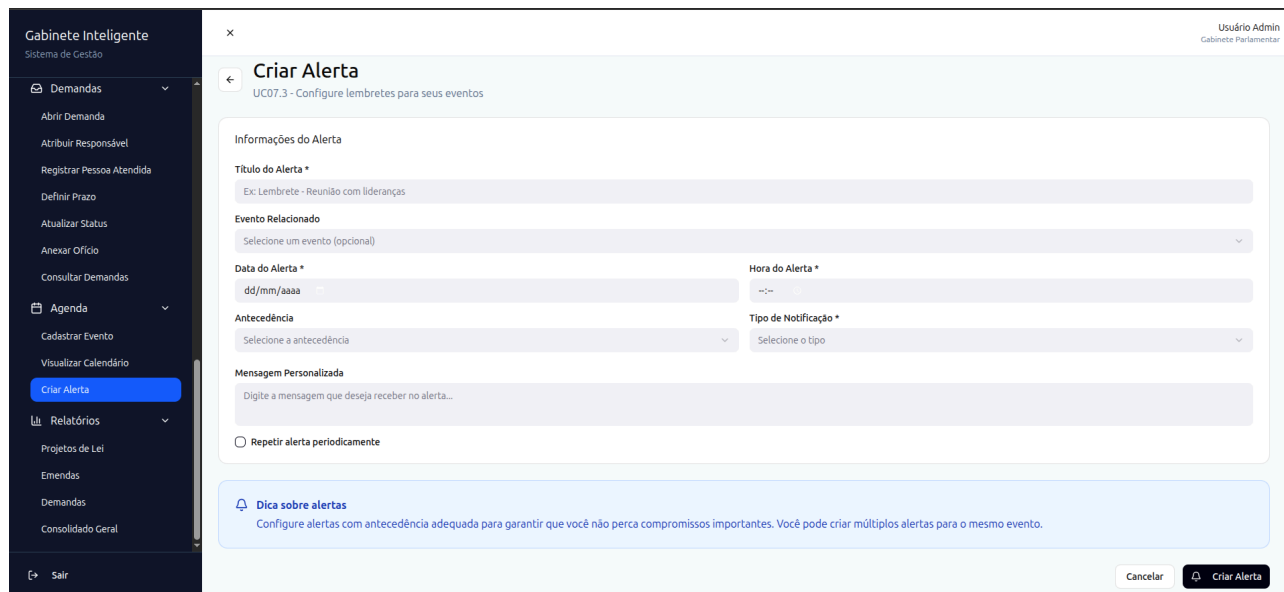


Figura 98 – Tela de Visualização do Calendário

Como mostra a Figura 99, a interface permite configurar alertas e lembretes automáticos associados à agenda, com definição de antecedência, canal de notificação e mensagem de apoio ao usuário.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Configuração de antecedência, canal de notificação, mensagem de lembrete e vínculo com evento quando aplicável.



A interface de criação de alertas apresenta um formulário com campos para: Título do Alerta (exemplo: 'Lembrete - Reunião com lideranças'), Evento Relacionado (seleção de evento), Data do Alerta (formato dd/mm/aaaa), Hora do Alerta (formato --:--), Antecedência (seleção de antecedência), Tipo de Notificação (seleção de tipo), Mensagem Personalizada (campo de texto) e uma opção para repetir o alerta periodicamente. Abaixo do formulário, há uma dica sobre alertas. No topo, o título é 'Criar Alerta' com o código 'UC07.3 - Configure lembretes para seus eventos'. No rodapé, há botões para 'Cancelar' e 'Criar Alerta'.

Informações do Alerta

Título do Alerta *

Ex: Lembrete - Reunião com lideranças

Evento Relacionado

Selecione um evento (opcional)

Data do Alerta *

dd/mm/aaaa

Hora do Alerta *

--:--

Antecedência

Selecione a antecedência

Tipo de Notificação *

Selecione o tipo

Mensagem Personalizada

Digite a mensagem que deseja receber no alerta...

☐ Repetir alerta periodicamente

Dica sobre alertas

Configure alertas com antecedência adequada para garantir que você não perca compromissos importantes. Você pode criar múltiplos alertas para o mesmo evento.

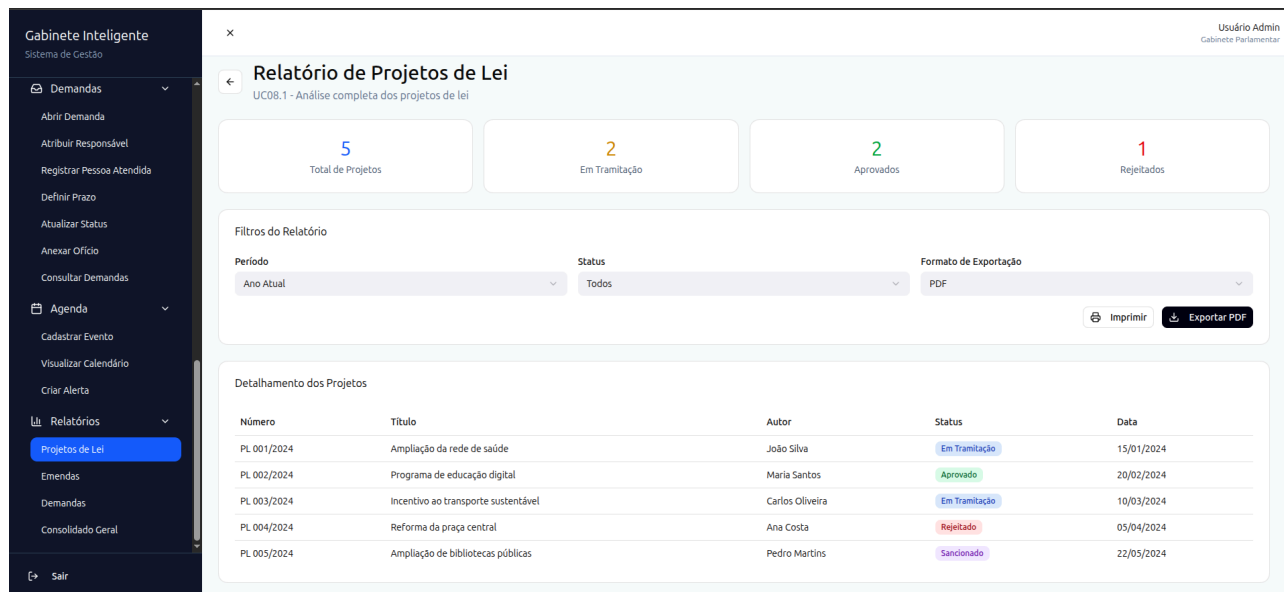
Cancelar Criar Alerta

Figura 99 – Tela de Criação de Alerta

A Figura 100 demonstra o relatório de projetos de lei, com indicadores consolidados, filtros por período e status e tabela contendo os principais dados legislativos acompanhados pelo gabinete.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Indicadores legislativos, filtros por período e status, tabela de projetos e ações de exportação.



A interface de relatório de projetos de lei apresenta indicadores consolidados, filtros e uma tabela de detalhes. No topo, o título é 'Relatório de Projetos de Lei' com o código 'UC08.1 - Análise completa dos projetos de lei'. Os indicadores mostram: 5 Total de Projetos, 2 Em Tramitação, 2 Aprovados e 1 Rejeitados. Abaixo, há filtros para Período (Ano Atual), Status (Todos) e Formato de Exportação (PDF). No rodapé, há botões para 'Imprimir' e 'Exportar PDF'.

Relatório de Projetos de Lei

UC08.1 - Análise completa dos projetos de lei

5 Total de Projetos

2 Em Tramitação

2 Aprovados

1 Rejeitados

Filtros do Relatório

Período: Ano Atual

Status: Todos

Formato de Exportação: PDF

Imprimir Exportar PDF

Detalhamento dos Projetos

Número	Título	Autor	Status	Data
PL 001/2024	Ampliação da rede de saúde	João Silva	Em Tramitação	15/01/2024
PL 002/2024	Programa de educação digital	Maria Santos	Aprovado	20/02/2024
PL 003/2024	Incentivo ao transporte sustentável	Carlos Oliveira	Em Tramitação	10/03/2024
PL 004/2024	Reforma da praça central	Ana Costa	Rejeitado	05/04/2024
PL 005/2024	Ampliação de bibliotecas públicas	Pedro Martins	Sancionado	22/05/2024

Figura 100 – Tela de Relatório de Projetos de Lei

Na Figura 101, o relatório de emendas apresenta total de emendas (5), valor total (R\$ 2.500.000,00), distribuição por status (2 Aprovadas, 1 Executada) e filtros por período, tipo de emenda e status. A tabela detalha número, tipo, valor, município e status de cada emenda.

Perfis com acesso: Deputada, Chefe de Gabinete.

Elementos principais: Indicadores financeiros, filtros combinados, tabela com valores monetários, opções de exportação.

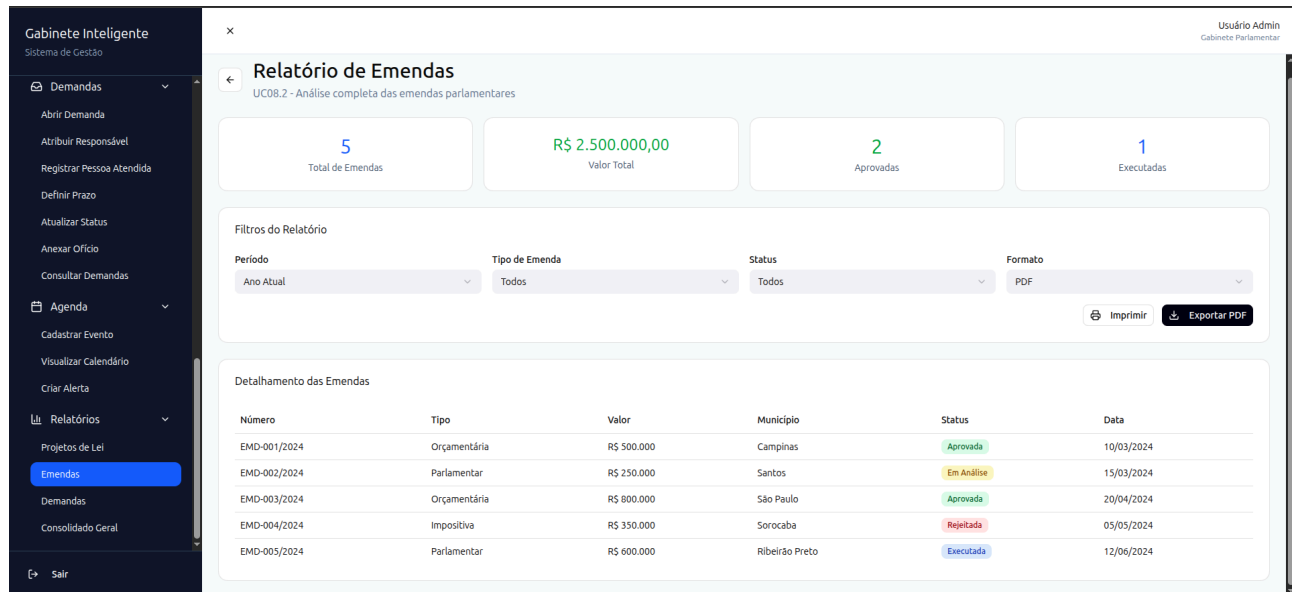


Figura 101 – Tela de Relatório de Emendas

A Figura 102 demonstra o relatório operacional com total de demandas (5), distribuição por status (2 Em Andamento, 1 Concluída, 1 Aguardando) e filtros por período, prioridade, status e formato. A tabela lista código, título, origem, prioridade, status e data de cada demanda.

Perfis com acesso: Deputada, Gestora de Demandas, Equipe de Atendimento.

Elementos principais: Cards quantitativos, filtros múltiplos, tabela operacional, botões de geração.

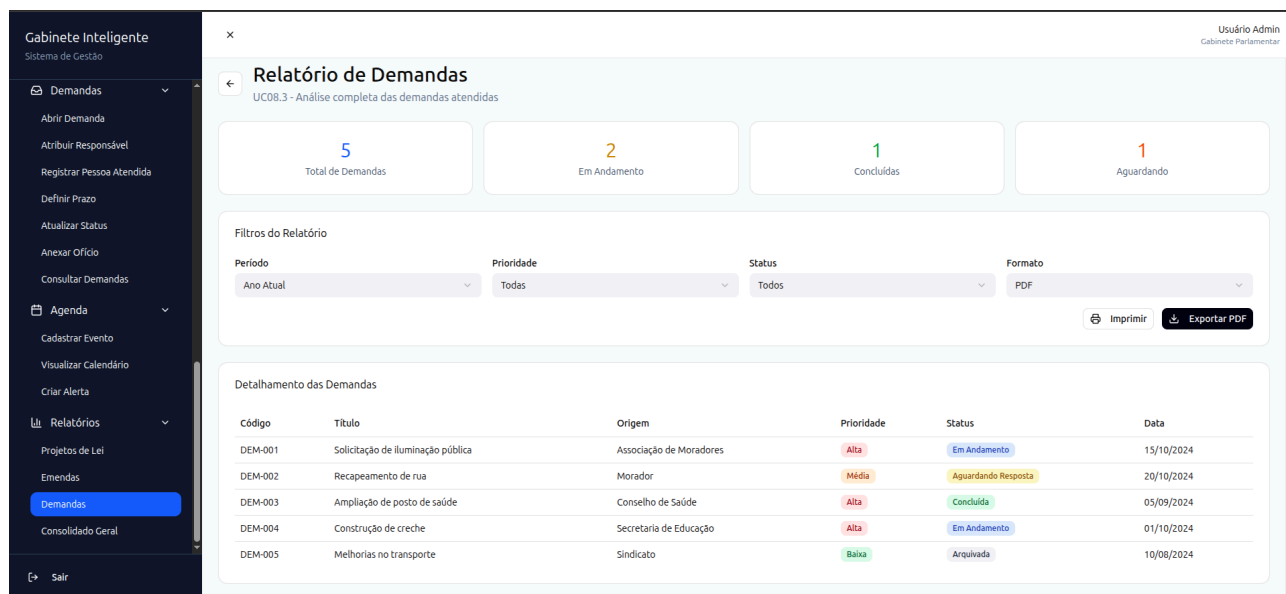


Figura 102 – Tela de Relatório de Demandas

Como mostra a Figura 103, o relatório estratégico exclusivo da Deputada consolida todas as atividades do gabinete em uma única visualização. A seção "Configurações do Relatório" permite selecionar período e formato de exportação (PDF Completo). O relatório é dividido em quatro blocos principais: Projetos de Lei (12 totais, com distribuição: 5 Aprovados, 6 Em Tramitação, 1 Rejeitado, taxa de aprovação 42%), Emendas Parlamentares (18 totais, R\$ 4.500.000,00 destinados, 10 Aprovadas, 5 Executadas), Demandas Atendidas (35 totais, com distribuição não detalhada na imagem) e Agenda e Eventos (42 totais).

Perfis com acesso: Deputada (exclusivo).

Elementos principais: Configurações de período, múltiplos painéis de indicadores, dados legislativos e financeiros consolidados, botões de impressão e exportação.

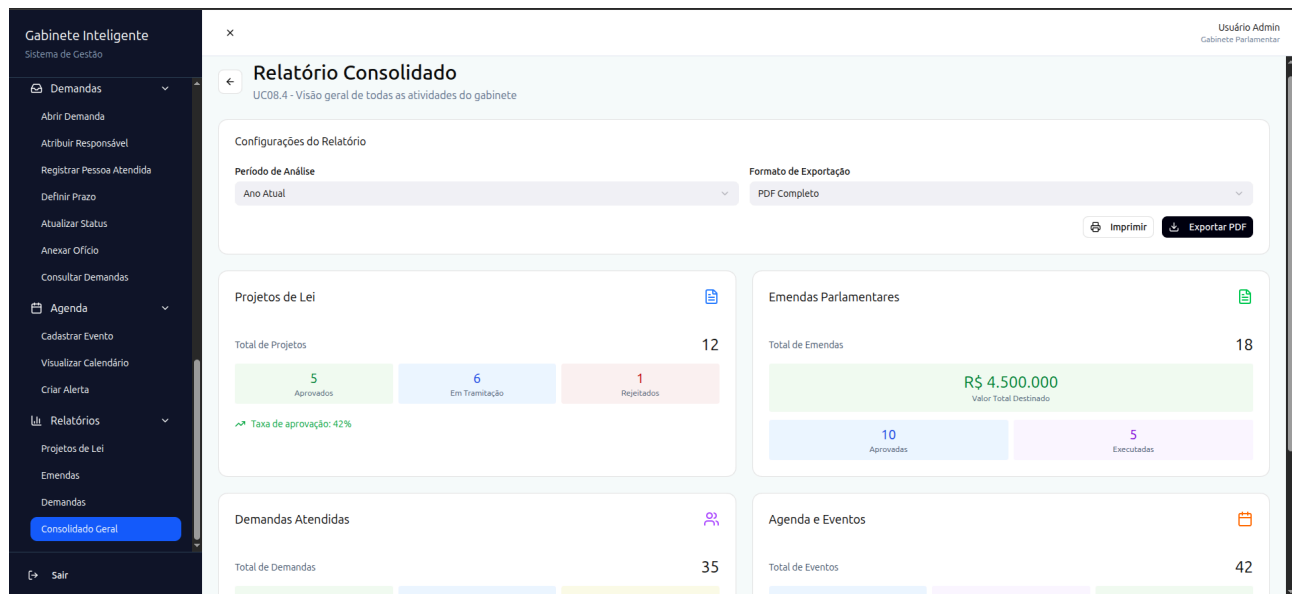


Figura 103 – Tela de Relatório Consolidado

5. Glossário e Modelos de Dados

O modelo de dados foi desenvolvido com base nas principais entidades identificadas nos casos de uso e histórias de usuário. Ele reflete o relacionamento entre usuários internos, demandas, emendas, eventos, cidades, instituições, cidadãos, telefones, sessões de *chatbot*, alertas e demais objetos centrais do domínio. A Figura 104 representa o Diagrama de Entidade e Relacionamento atualizado do sistema, contemplando também as estruturas necessárias para o atendimento automatizado e para o envio de notificações.

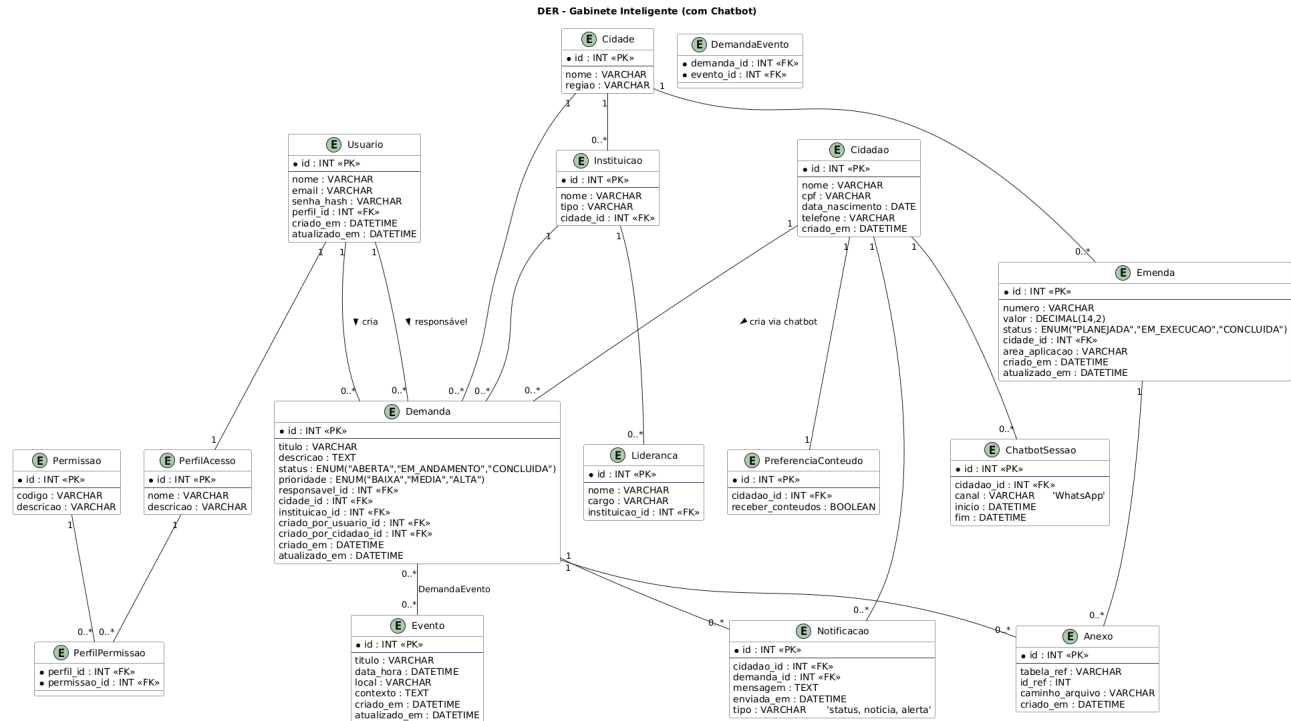


Figura 104 – Diagrama de Entidade e Relacionamento

As Tabelas 27 a 42 apresentam um glossário das principais tabelas físicas persistidas pelo sistema, com foco nas estruturas centrais do domínio e nas entidades mais relevantes para os fluxos administrativos, parlamentares, de atendimento e de notificações. Para manter a leitura objetiva, tabelas estritamente técnicas ou auxiliares, como *cache*, *jobs*, *tokens* e estruturas internas de suporte, não são detalhadas individualmente nesta seção.

Cidade		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da cidade.
name	VARCHAR	Nome do município.
region	VARCHAR	Região associada ao município.

Tabela 27 - Glossário para Cidade

PerfilAcesso		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do perfil de acesso.
name	VARCHAR	Nome do perfil.

PerfilAcesso		
description	VARCHAR	Descrição funcional do perfil.

Tabela 28 - Glossário para PerfilAcesso

Permissao		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da permissão.
code	VARCHAR	Código usado nas regras de autorização.
description	VARCHAR	Descrição textual da permissão.

Tabela 29 - Glossário para Permissao

PerfilPermissao		
Atributo	Tipo	Descrição
access_profile_id	BIGINT (FK)	Perfil de acesso associado.
permission_id	BIGINT (FK)	Permissão vinculada ao perfil.

Tabela 30 - Glossário para PerfilPermissao

Usuario		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do usuário interno.
name	VARCHAR	Nome completo do usuário.
email	VARCHAR UNIQUE	E-mail utilizado para autenticação.
password	VARCHAR	Senha criptografada.
access_profile_id	BIGINT (FK)	Perfil de acesso associado ao usuário.
created_at	TIMESTAMP	Data de criação do registro.
updated_at	TIMESTAMP	Data da última atualização.

Tabela 31 - Glossário para Usuario

Instituicao		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da instituição.
name	VARCHAR	Nome da instituição.
type	VARCHAR	Tipo ou categoria da instituição.

Instituicao		
city_id	BIGINT (FK)	Cidade à qual a instituição pertence.

Tabela 32 - Glossário para Instituicao

Lideranca		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da liderança.
name	VARCHAR	Nome da liderança cadastrada.
position	VARCHAR	Cargo ou função exercida.
institution_id	BIGINT (FK)	Instituição à qual a liderança está vinculada.

Tabela 33 - Glossário para Lideranca

Emenda		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da emenda.
number	VARCHAR	Número oficial da emenda.
amount	DECIMAL(14,2)	Valor financeiro previsto.
status	ENUM	Situação atual da emenda.
application_area	VARCHAR	Área de aplicação vinculada à emenda.
city_id	BIGINT (FK)	Município beneficiado.
created_at	TIMESTAMP	Data de criação do registro.
updated_at	TIMESTAMP	Data da última atualização.

Tabela 34 - Glossário para Emenda

ProjetoLei		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do projeto de lei.
number	VARCHAR UNIQUE	Número oficial do projeto.
description	TEXT	Descrição ou ementa do projeto.
status	ENUM	Situação legislativa atual.
protocol_date	DATE	Data de protocolo do projeto.
created_at	TIMESTAMP	Data de criação do registro.
updated_at	TIMESTAMP	Data da última atualização.

Tabela 35 - Glossário para ProjetoLei

Demanda		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador da demanda.
title	VARCHAR	Título resumido da solicitação.
description	TEXT	Descrição detalhada da demanda.
status	ENUM	Estado atual da demanda, incluindo descarte.
discard_message	TEXT	Mensagem explicando o motivo do descarte automático, quando aplicável.
service_area	VARCHAR	Área de atendimento responsável pelo caso.
priority	ENUM	Prioridade operacional da demanda.
responsible_user_id	BIGINT (FK)	Usuário interno responsável pela condução.
city_id	BIGINT (FK)	Cidade relacionada à demanda.
institution_id	BIGINT (FK)	Instituição relacionada, quando informada.
created_by_citizen_id	BIGINT (FK)	Cidadão que originou a demanda via <i>chatbot</i> , quando houver.
oficio_path	VARCHAR	Caminho do ofício ou documento vinculado.

Tabela 36 - Glossário para Demanda

Evento		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do evento.
title	VARCHAR	Título do compromisso.
type	VARCHAR	Tipo de evento institucional.
starts_at	DATETIME	Data e hora de início.
ends_at	DATETIME	Data e hora de término.
location	VARCHAR	Local de realização.
description	TEXT	Contexto ou descrição do evento.
city_id	BIGINT (FK)	Cidade associada ao evento, quando houver.

Tabela 37 - Glossário para Evento

HistoricoDemanda		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do histórico.
demand_id	BIGINT (FK)	Demanda à qual o histórico pertence.
user_id	BIGINT (FK)	Usuário que realizou a ação, quando interno.
action	VARCHAR	Tipo de ação executada.
description	TEXT	Descrição textual da alteração registrada.
metadata	JSON	Dados complementares da alteração.
created_at	TIMESTAMP	Momento em que a alteração foi registrada.

Tabela 38 - Glossário para HistoricoDemanda

Cidadao		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do cidadão.
name	VARCHAR	Nome informado pelo cidadão.
cpf	VARCHAR	CPF, quando fornecido.
birth_date	DATE	Data de nascimento, quando informada.
phone	VARCHAR	Telefone principal associado ao cadastro.
receive_demand_updates	BOOLEAN	Indica se o cidadão deseja receber atualizações da demanda.

Tabela 39 - Glossário para Cidadão

ChatbotSessao		
Atributo	Tipo	Descrição
id	INT (FK)	Demanda associada.
cidadao_id	INT (FK)	Cidadão da sessão
canal	VARCHAR	Canal de comunicação (whatsapp)
inicio	DATETIME	Início da sessão
fim	DATETIME	Fim da sessão

Tabelas Auxiliares do <i>Chatbot</i>		
Tabela	Campos-chave	Função
citizen_phones	citizen_id, phone, normalized_phone	Armazena múltiplos telefones e a versão normalizada usada na identificação.
chatbot_sessions	citizen_id, channel, started_at, ended_at	Registra o contexto temporal das conversas atendidas pelo <i>chatbot</i> .
content_preferences	citizen_id, receive_content	Persiste a preferência de recebimento de conteúdo institucional.

Tabela 40 - Glossário para Tabelas Auxiliares do *Chatbot*

PreferenciaConteudo		
Atributo	Tipo	Descrição
id	INT (FK)	Demanda associada.
cidadao_id	INT (FK)	Cidadão
receber_conteudos	BOOLEAN	Se deseja ou não receber conteúdos

AlertaDemanda		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do alerta de demanda.
demand_id	BIGINT (FK)	Demanda que originou o alerta.
user_id / citizen_id	BIGINT (FK)	Destinatário interno ou cidadão vinculado ao alerta.
channel	VARCHAR	Canal de envio, como system ou <i>chatbot</i> .
status	VARCHAR	Estado do envio do alerta.
title	VARCHAR	Título exibido ao destinatário.
message	TEXT	Mensagem principal do alerta.
read_at / sent_at	TIMESTAMP	Controle de leitura e confirmação de envio.

Tabela 41 - Glossário para AlertaDemanda

Notificacao		
Atributo	Tipo	Descrição
id	INT (FK)	Demanda associada.
cidadao_id	INT (FK)	Cidadão
demanda_id	INT (FK)	Demanda relacionada (quando aplicável)
mensagem	TEXT	Conteúdo da notificação enviada
tipo	VARCHAR	Tipo da notificação (<i>status</i> , notícia, alerta)
enviada_em	DATETIME	Data e hora do envio

AlertaEvento		
Atributo	Tipo	Descrição
id	BIGINT (PK)	Identificador do lembrete de agenda.
event_id	BIGINT (FK)	Evento ao qual o lembrete pertence.
user_id	BIGINT (FK)	Usuário que receberá o lembrete.
alert_at	DATETIME	Momento planejado para disparo do alerta.
lead_time_minutes	INTEGER	Antecedência em minutos em relação ao evento.
channel	ENUM	Canal configurado para o alerta.
status	VARCHAR	Situação de processamento do lembrete.
is_automatic	BOOLEAN	Indica se o lembrete foi criado automaticamente pelo sistema.

Tabela 42 - Glossário para AlertaEvento

6. Casos de Teste

Esta seção foi refatorada com base nos testes efetivamente implementados no *backend*

Laravel e no

chatbot

em Python, complementados por validações de integração e de sistema realizadas sobre o *frontend*

React em ambiente conectado aos serviços reais. Em vez de descrever cenários hipotéticos, a seção resume a cobertura que já foi exercitada no código e os fluxos ponta a ponta verificados durante o desenvolvimento.

6.1 Testes Automatizados de *Backend* e *Chatbot*

A cobertura automatizada foi concentrada principalmente em testes de feature e API no *backend*, além de testes de serviço, validação e notificações internas no *chatbot*. Essa estratégia foi adequada ao projeto porque as regras críticas dependem diretamente da interação entre autenticação, banco de dados, filas, integrações HTTP e notificações em tempo real.

6.1.1 *Backend*: autenticação, permissões e módulos administrativos

Caso de Teste 1 — Autenticação, autorização e usuários

Item	Descrição
Base automatizada	AuthApiTest, AuthorizationApiTest, UserApiTest e AccessProfileApiTest.
Escopo validado	Registro, login, logout, consulta de perfil, proteção de rotas e gestão de usuários, perfis e permissões.
Resultado esperado	A autenticação gera <i>tokens</i> válidos e as regras de acesso impedem operações sem permissão adequada.
Evidência	Suítes de feature do <i>backend</i> Laravel executadas sobre <i>endpoints</i> reais da API.

Caso de Teste 2 — Demandas internas e regras de permissão

Item	Descrição
Base automatizada	DemandApiTest.
Escopo validado	Criação, edição, exclusão, histórico, filtro por responsável, área de atendimento, ofício e instituição opcional.
Resultado esperado	Usuários com manage controlam todas as demandas; usuários sem manage ficam restritos às próprias demandas e filtros permitidos.
Evidência	Validação de download e atualização de ofício, histórico persistido e ocultação padrão de demandas descartadas.

Caso de Teste 3 — Módulos legislativos, CMS e conteúdo público

Item	Descrição
Base automatizada	AmendmentApiTest, ProjectLawApiTest, CmsApiTest, DashboardApiTest e PublicApiTest.
Escopo validado	Gestão de emendas, projetos de lei, seções CMS, notícias, projetos do <i>site</i> , visão do <i>dashboard</i> e <i>endpoints</i> públicos.
Resultado esperado	Os módulos legislativos e institucionais respondem corretamente às operações de consulta e administração autorizada.
Evidência	Os testes validam <i>endpoints</i> protegidos, dados consolidados e publicação das informações públicas consumidas pelo <i>site</i> .

6.1.2 Backend: agenda, alertas e processamento assíncrono

Caso de Teste 4 — Agenda e lembretes automáticos

Item	Descrição
Base automatizada	AgendaApiTest e AgendaReminderApiTest.
Escopo validado	Criação e edição de eventos, detecção de conflitos, listagem padrão por data, geração de lembretes automáticos e leitura por usuário.
Resultado esperado	Eventos em conflito são recusados e lembretes são gerados, filtrados e processados conforme o responsável autenticado.
Evidência	A suíte confirma criação automática de lembretes, fila de processamento e marcação de leitura pelo próprio usuário.

Caso de Teste 5 — Alertas de demanda e *jobs* assíncronos

Item	Descrição
Base automatizada	DemandAlertApiTest, ProcessDemandAlertJobTest e ProcessAgendaReminderJobTest.
Escopo validado	Geração de alertas após atualização de demanda, publicação para <i>WebSocket</i> e envio de mensagens internas ao <i>chatbot</i> .
Resultado esperado	Alertas persistidos são enfileirados, enviados ao canal correto e podem ser marcados como lidos pelo destinatário.
Evidência	Os testes simulam chamadas HTTP internas e validam a mudança de status dos alertas para sent.

Caso de Teste 6 — API interna do *chatbot* e vínculo com cidadão

Item	Descrição
Base automatizada	ChatbotDemandApiTest e ChatbotDemandStatusApiTest.
Escopo validado	Lookup de cidadão por telefone, cadastro, listagem de cidades e instituições, abertura de demanda, consulta de status e proteção por <i>token</i> interno.
Resultado esperado	O <i>backend</i> interno do <i>chatbot</i> identifica corretamente o cidadão, cria demandas vinculadas e rejeita acessos sem <i>token</i> configurado.
Evidência	A suíte valida inclusive criação de demanda sem instituição e busca de demandas recentes para similaridade.

6.1.3 Chatbot

Caso de Teste 7 — Validações conversacionais e descarte automatizado

Item	Descrição
Base automatizada	test_chatbot_service.py e test_demand_validation.py.
Escopo validado	Idioma inválido, conteúdo ofensivo, tom agressivo, discurso de ódio, similaridade com demandas abertas e instituição em branco.
Resultado esperado	Mensagens inválidas geram descarte automatizado com contexto de validação; mensagens válidas prosseguem normalmente no fluxo conversacional.

Evidência Os testes exercitam validadores isolados e o serviço conversacional completo, incluindo o salto da etapa de nome quando o telefone já é conhecido.

Caso de Teste 8 — Serviço do *chatbot* e verificações auxiliares

Item	Descrição
Base automatizada	test_health.py, test_config.py, test_demo_chat.py e test_internal_notifications.py.
Escopo validado	<i>Healthcheck</i> , carregamento de configuração, fluxo demonstrativo e <i>endpoints</i> internos de mensagem e publicação de alerta em tempo real.
Resultado esperado	O serviço permanece operacional, lê corretamente sua configuração e responde adequadamente às chamadas internas do <i>backend</i> .
Evidência	A suíte verifica envio interno de mensagem para WhatsApp e publicação de payloads de demanda e agenda para o broker de alertas.

6.1.4 Observações sobre cobertura

Item	Descrição
Entrada	Filtro “Saúde”

Saída Esperada Listagem de emendas na categoria
“Saúde”

Além dos cenários resumidos nas tabelas anteriores, o *backend* também possui cobertura complementar para módulos institucionais e legislativos por meio de suítes como *AmendmentApiTest*, *ProjectLawApiTest*, *CmsApiTest*, *DashboardApiTest*, *PublicApiTest* e *AccessProfileApiTest*. Essas suítes ampliam a validação dos fluxos administrativos, públicos e de autorização da aplicação.

No *chatbot*, existem ainda testes auxiliares para *healthcheck*, carregamento de configuração, fluxo demonstrativo e rotas internas de notificação, garantindo que o serviço responda corretamente tanto para verificações operacionais quanto para chamadas internas do *backend*.

Item	Descrição
Entrada	Nome, tipo (prefeitura), cidade
Saída Esperada	Instituição cadastrada

No *frontend*, a validação foi conduzida principalmente em nível de integração e de sistema. Os fluxos reais da interface foram exercitados contra o *backend* e os serviços de alerta, cobrindo autenticação, filtros, criação e edição por modais, atualização reativa das listagens, relatórios, telas públicas e exibição de notificações em tempo real.

Item	Descrição
Entrada	Liderança: “Carlos”, Instituição: “Secretaria de Obras”
Saída Esperada	Vínculo apresentado na página de detalhes

Essa escolha foi coerente com a natureza do projeto, pois grande parte do comportamento esperado no *frontend* depende diretamente das permissões devolvidas pela API, do estado persistido no banco e da chegada de eventos assíncronos, o que torna mais relevante a validação integrada do que uma suíte isolada de componentes.

Também foram exercitados cenários de borda, como ausência de instituição na demanda, ocultação padrão de demandas descartadas, filtragem restrita para usuários sem permissão de gestão e rejeição de mensagens consideradas inválidas pelo fluxo do chatbot.

Item	Descrição
Entrada	Título, data, cidade, descrição
Saída Esperada	Evento registrado e visível no calendário

Com isso, a cobertura adotada priorizou os fluxos mais críticos do domínio, validando tanto o caminho de sucesso quanto regras de acesso, descarte automatizado, conflitos de agenda e entrega assíncrona de notificações.

Item	Descrição
Entrada	Registrar evento no mesmo horário
Saída Esperada	Mensagem de conflito exibida

6.2 Testes de Integração e de Sistema

Item	Descrição
Entrada	Mês = Dezembro
Saída Esperada	Exibe compromissos cadastrados

6.2.1 Visão Geral

Os testes de integração e de sistema foram conduzidos considerando a aplicação como um conjunto formado por *frontend* React, *backend* Laravel, *chatbot* e mecanismos de alerta em tempo real. Nesse nível, o objetivo não foi validar métodos isolados, mas confirmar que os componentes trocam dados corretamente e preservam o comportamento esperado em fluxos completos.

Foram verificados fluxos integrados entre *backend* e *chatbot*, como identificação do cidadão por telefone, consulta de cidades e instituições, criação de demandas normais ou descartadas e consulta de status por canal interno. Da mesma forma, foram testadas integrações entre *backend*, filas e mecanismos de publicação de alertas, assegurando que mudanças de demanda e lembretes de agenda gerem notificações persistidas e encaminhadas ao destino correto.

No *frontend*, os testes de sistema focaram a jornada do usuário com os serviços reais: autenticação, consulta com filtros, criação e edição por formulários modais, consumo de relatórios, telas públicas e recepção de alertas em tempo real. Essa abordagem permitiu validar o comportamento observável da solução como produto integrado, e não apenas como soma de componentes isolados.

6.2.2 Interfaces de Integração Validadas

Integração	Interface	Descrição
<i>Frontend</i> React ↔ <i>Backend</i> Laravel	HTTPS/JSON + JWT	Autenticação, filtros, formulários modais, relatórios e consumo dos dados públicos e administrativos.
<i>Chatbot</i> API ↔ <i>Backend</i> Laravel	HTTP interno + <i>token</i>	Lookup de cidadão por telefone, listagem de opções, criação de demandas e consulta de status.
<i>Backend</i> Laravel ↔ <i>Chatbot</i> Interno	<i>Jobs</i> + HTTP interno	Envio de mensagens ao cidadão e publicação de alertas para o serviço de <i>chatbot</i> .
<i>Backend</i> Laravel ↔ <i>Websocket</i> /Broker	<i>Jobs</i> + publicação interna	Distribuição de <i>pop-ups</i> de demanda e lembretes de agenda em tempo real para o <i>frontend</i> .

Backend Laravel ↔ Disco público + queue/jobs
Storage/Fila

Persistência de ofícios e
processamento assíncrono de
lembretes e alertas.

6.2.3 Estratégia Adotada

A estratégia adotada combinou testes automatizados de API e serviço com validações integradas em ambiente funcional. No *backend* e no *chatbot*, os testes exercitam *endpoints* e fluxos internos com banco efêmero, armazenamento simulado, filas falsas e clientes HTTP interceptados, permitindo verificar tanto as regras de domínio quanto o comportamento das integrações externas.

1. Para os fluxos assíncronos, os *jobs* de alerta e lembrete foram testados separadamente, confirmando a criação dos registros persistidos e a emissão correta para os canais de *chatbot* e *WebSocket*.
2. No *frontend*, a validação foi realizada em nível de integração e sistema, utilizando a aplicação React conectada ao *backend* e aos serviços de alerta. Assim, foi possível verificar permissões, dados persistidos, atualização reativa das telas e comportamento de notificações em um cenário próximo ao uso real.
3. Também foram considerados cenários de borda e erro, como mensagens rejeitadas pelo *chatbot*, demandas sem instituição, restrições de filtragem para usuários sem permissão de gestão e ocultação padrão de demandas descartadas na listagem principal.
 - Essa estratégia privilegiou a confiabilidade ponta a ponta dos fluxos críticos do domínio, cobrindo não apenas o sucesso das operações, mas também restrições de acesso, descarte automatizado, conflitos de agenda e entrega assíncrona de notificações.
4. 6.2.4 Exemplos de Cenários Integrados Validados
5. Um primeiro cenário integrado validado foi o envio de demanda pelo *chatbot*: o cidadão é identificado pelo telefone, escolhe a cidade, seleciona ou omite a instituição, passa pelas validações automáticas e, ao final, a demanda é criada no *backend* com o contexto correto, inclusive como descartada quando necessário.
 - Outro cenário integrado relevante foi a atualização de uma demanda por um usuário interno, seguida da geração de histórico, criação de alertas persistidos, publicação

para o canal em tempo real do *frontend* e envio de mensagem ao cidadão quando ele optou por receber atualizações.

Esses cenários representam bem a lógica adotada no projeto: *backend*, *chatbot* e *frontend* foram validados como partes de um mesmo ecossistema, com ênfase nos fluxos reais de uso e nas consequências assíncronas geradas pelas operações principais.

Entrada	Cadastro de nova demanda vinculada a Instituição ID 15
Processo	API valida existência da instituição
Saída Esperada	Demanda Criada: instituição exibida no detalhe
Falha Esperada	Erro 404 caso instituição inexistente

Entrada	Criar evento com cidade “Curvelo”
Saída	Evento salvo: renderização mostra nome e contexto

7. Plano de Desenvolvimento

O processo de implementação do sistema Gabinete Virtual será conduzido de forma incremental e iterativa, inspirado em práticas de metodologias ágeis, garantindo flexibilidade, adaptação contínua às necessidades reais do gabinete parlamentar e evolução progressiva do *software*. Essa abordagem considera o sistema como um ecossistema integrado, composto pelo *frontend web*, *backend API* e *chatbot*, permitindo que diferentes canais de interação evoluam de maneira coordenada e coerente. A estratégia adotada privilegia ciclos curtos de desenvolvimento, organizados em sprints quinzenais, nos quais funcionalidades são projetadas, implementadas, testadas e avaliadas de forma incremental. Durante esses ciclos, haverá validações frequentes junto aos usuários-chave, como a Deputada, a Chefe de Gabinete, a Gestora de Demandas e a Equipe de Atendimento, possibilitando que o

sistema seja continuamente refinado com base em *feedback* real. O *chatbot* será validado em conjunto com esses usuários, especialmente nos fluxos de envio de demandas, consulta de *status* e recebimento de notificações, garantindo aderência ao contexto institucional.

A gestão do projeto será realizada por meio de ferramentas de apoio, como Jira ou Trello, utilizadas para organizar o *backlog*, planejar *sprints*, priorizar tarefas e acompanhar o progresso das atividades. Cada *sprint* contempla etapas de análise de requisitos, implementação de módulos isolados ou integrados, desenvolvimento de funcionalidades do *chatbot*, produção de testes automatizados e validações intermediárias. O uso de quadros de tarefas, definição clara de prioridades e revisões frequentes contribuirá para a visibilidade, rastreabilidade e controle do andamento do desenvolvimento.

O GitHub Classroom será utilizado como ambiente oficial de versionamento, atendendo às diretrizes institucionais e permitindo histórico completo das alterações realizadas ao longo do projeto. A estratégia de *branches* seguirá um fluxo estruturado, com separação entre *branches* de desenvolvimento, funcionalidades e correções, garantindo isolamento adequado de mudanças, facilidade de revisão e integração contínua entre *backend*, *frontend* e *chatbot*.

Para assegurar a qualidade da implementação, o projeto adotará práticas de Integração Contínua (CI) e Entrega Contínua (CD), utilizando GitHub Actions para automatizar processos essenciais, como execução de testes, análise estática de código, *build* do *frontend*, validação do *backend* e implantação em ambiente de homologação. Essas rotinas automatizadas reduzem erros humanos, promovem consistência entre os ambientes e aceleram a disponibilização de versões estáveis para avaliação, incluindo as funcionalidades do *chatbot* integradas ao *backend*.

A garantia de qualidade será reforçada por meio da adoção abrangente de testes automatizados e validações integradas. No *backend*, desenvolvido em Laravel, serão aplicados testes unitários, de integração, de API e de processamento assíncrono com PHPUnit, contemplando tanto os módulos tradicionais quanto os *endpoints* responsáveis pela integração com o *chatbot*. No *frontend*, construído em React, a validação será conduzida principalmente por testes de integração e de sistema dos fluxos completos, incluindo os cenários que envolvem criação e acompanhamento de demandas. Esses fluxos serão complementados por testes no *chatbot*, desenvolvido em Python com FastAPI e validado com pytest para regras de conversa, validações de demandas e notificações internas.

Além da automação de testes, o processo prevê revisões sistemáticas de código por meio de *pull requests*, validação contínua dos requisitos junto aos usuários reais e auditoria periódica das funcionalidades mais sensíveis, como autenticação, cadastros, fluxo de demandas, notificações e integração do *chatbot*. A participação ativa do gabinete nas avaliações incrementais assegura que as entregas estejam alinhadas ao contexto político-administrativo e às necessidades reais do mandato.

Ao longo do desenvolvimento, diversos artefatos de Engenharia de Software serão produzidos ou atualizados, incluindo diagramas UML (casos de uso, sequência, classes, componentes e implantação), modelos de dados, contratos de operação, relatórios de testes, protótipos de interface, fluxos conversacionais do *chatbot* e documentação técnica. Esses artefatos sustentam o processo de implementação, facilitam a manutenção futura e garantem consistência com o Documento de Visão e com a arquitetura proposta.

Dessa forma, o plano de desenvolvimento do Gabinete Virtual, incluindo o *chatbot* como canal adicional de interação, combina rigor técnico, práticas modernas de engenharia de *software* e participação efetiva dos usuários. Essa abordagem assegura que o sistema evolua com qualidade, previsibilidade e alinhamento ao domínio parlamentar, resultando em um produto robusto, confiável e adequado às demandas institucionais do gabinete.

8. Cronograma e Processo de Implementação

O cronograma do TCC II foi estruturado em quinzenas, cobrindo o período entre a primeira quinzena de fevereiro de 2026 e a segunda quinzena de junho de 2026. Esse planejamento distribui de forma progressiva e organizada todas as etapas necessárias para a implementação, testes, integração do *chatbot*, documentação e preparação da defesa do Sistema Gabinete Virtual.

O cronograma, apresentado na Tabela 43, inicia-se com atividades de alinhamento, revisão dos artefatos produzidos durante o TCC I e preparação do ambiente técnico de desenvolvimento. Essa etapa inicial também contempla a definição da arquitetura final, incluindo a estratégia de integração do *chatbot* como canal adicional de interação com o sistema.

Na sequência, o planejamento avança para as fases de implementação dos principais módulos do sistema, iniciando pelos componentes centrais (autenticação, perfis de acesso, cadastros básicos e

gestão de demandas) que servem de base tanto para o *frontend web* quanto para o *chatbot*. A partir dessas fundações, são implementados os módulos legislativos, a gestão de agenda, o CMS e os serviços de geração de relatórios.

Paralelamente à evolução do *backend*, o cronograma contempla o desenvolvimento e integração do *chatbot*, incluindo a implementação do *Webhook*, tratamento de mensagens, cadastro do cidadão, envio de demandas, consulta de status e recebimento de notificações. Essa abordagem garante que o *chatbot* reutilize integralmente os serviços já existentes, mantendo a coerência arquitetural do sistema.

A partir do mês de maio, o foco se desloca para testes integrados entre *frontend*, *backend* e *chatbot*, refinamento das funcionalidades, otimização das rotinas e ajustes de usabilidade. Essa fase assegura que o sistema esteja estável, aderente aos requisitos e alinhado às especificações técnicas propostas.

Por fim, no mês de junho, concentram-se as atividades de finalização do *frontend* e do *chatbot*, revisão detalhada da documentação, consolidação dos artefatos técnicos, elaboração dos slides e preparação para a apresentação perante a banca avaliadora. A última quinzena é dedicada à entrega final e à defesa do trabalho.

Esse planejamento escalonado permite que o desenvolvimento avance de forma contínua e controlada, reduzindo riscos, garantindo tempo adequado para validações e possibilitando que cada módulo, incluindo o *chatbot*, seja implementado, integrado e validado com qualidade antes da entrega final.

Período	Atividades Previstas
1ª Quinzena – Fevereiro/2026	Início oficial do TCC II; Revisão final dos artefatos; Alinhamento com orientador; Ajustes na documentação; Definição do escopo da implementação e da integração do <i>chatbot</i> .
2ª Quinzena – Fevereiro/2026	Preparação do ambiente; Estruturação inicial do <i>backend</i> ; Definição das entidades; Configuração do banco de dados. Definição do fluxo de integração do <i>chatbot</i> (<i>Webhook</i>).
1ª Quinzena – Março/2026	Implementação da autenticação e perfis; Cadastros básicos;

	Criação das primeiras rotas; Estrutura inicial do <i>Chatbot</i> API. Testes iniciais.
2ª Quinzena – Março/2026	Implementação da gestão de demandas; Histórico; Integração inicial com <i>frontend</i> ; Integração do <i>chatbot</i> para envio de demandas; Testes integrados.
1ª Quinzena – Abril/2026	Implementação da gestão de emendas e PLs; Validação de <i>status</i> ; Criação de serviços auxiliares. Integração do <i>chatbot</i> para consulta de status.
2ª Quinzena – Abril/2026	Implementação da agenda; CMS; Integração com <i>frontend</i> ; Integração do <i>chatbot</i> para recebimento de conteúdos e notificações; Testes dos módulos.
1ª Quinzena – Maio/2026	Desenvolvimento dos relatórios; Criação de componentes visuais; Ajustes de usabilidade.
2ª Quinzena – Maio/2026	Testes completos; Correções; otimização de consultas; Revisão dos módulos.
1ª Quinzena – Junho/2026	Conclusão do <i>frontend</i> e <i>chatbot</i> ; Revisão da documentação; Preparação da apresentação.
2ª Quinzena – Junho/2026	Entrega final; Defesa; Consolidação dos artefatos no GitHub Classroom.

Tabela 43 - Cronograma do Sistema

9. Post-Mortem

Esta seção apresenta uma reflexão final sobre o processo de desenvolvimento do software produzido neste trabalho, com foco nas decisões técnicas, nos desafios encontrados e nos aprendizados obtidos ao longo da implementação da solução. O objetivo do post-mortem é registrar uma análise crítica da experiência de construção do sistema, considerando os aspectos práticos envolvidos na concepção, integração, evolução e validação das funcionalidades desenvolvidas.

Experiências Positivas

O desenvolvimento do sistema proporcionou uma experiência positiva principalmente pela oportunidade de construir uma solução completa, envolvendo *frontend*, *backend*, banco de dados, documentação e integrações externas. Isso permitiu uma visão ampla do ciclo de desenvolvimento de *software*, desde o levantamento das necessidades até a implementação de funcionalidades utilizáveis em um cenário próximo ao real.

Outro ponto positivo foi o aprendizado obtido com a integração da API do WhatsApp e com a construção do *chatbot*. Essa parte do projeto exigiu o entendimento de fluxos assíncronos, tratamento de *webhooks*, gerenciamento de contexto de conversa e envio automatizado de mensagens, ampliando significativamente o domínio sobre integrações com serviços externos e automação de atendimento.

Também foi positiva a evolução da arquitetura do sistema ao longo do trabalho. A separação entre *frontend*, *backend*, *chatbot* e serviço de notificações em tempo real permitiu organizar melhor as responsabilidades de cada parte da aplicação, facilitando a manutenção, a escalabilidade e a clareza do projeto como um todo.

Experiências Negativas

A principal dificuldade do projeto esteve relacionada à complexidade de desenvolver sozinho uma solução com múltiplos módulos e integrações. A ausência de divisão de responsabilidades aumentou a carga de planejamento, implementação, testes e correções, exigindo alternância constante entre diferentes contextos técnicos.

Houve também desafios importantes na integração entre os componentes do sistema. Recursos como notificações em tempo real, filas assíncronas, envio de mensagens pelo *chatbot* e sincronização entre *frontend* e *backend* exigiram diversas etapas de ajuste, depuração e refinamento até atingirem um comportamento consistente.

Além disso, algumas funcionalidades demandaram mais tempo do que o inicialmente previsto, especialmente aquelas que envolviam serviços externos, regras de validação mais elaboradas e

tratamento de fluxos alternativos. Isso evidenciou que integrações reais e requisitos incrementais aumentam significativamente o esforço de desenvolvimento e validação.

Lições Aprendidas

Uma das principais lições aprendidas foi a importância de definir uma arquitetura clara desde cedo, mas mantendo flexibilidade para refatorações ao longo do projeto. À medida que novas necessidades surgiram, ficou evidente que componentes bem separados facilitam a evolução do sistema e reduzem o impacto de mudanças.

Também foi reforçada a importância dos testes e da validação de fluxos completos. Em funcionalidades que envolvem múltiplos serviços, como atualização de demandas com disparo de alertas ou registro de demandas via *chatbot*, validar apenas partes isoladas não é suficiente. Foi necessário pensar no comportamento ponta a ponta para garantir confiabilidade.

Outra lição relevante foi perceber que o desenvolvimento individual de um projeto completo exige disciplina maior na organização técnica. Documentação, padronização, controle de alterações e clareza nas decisões arquiteturais tornam-se ainda mais importantes quando todas as etapas dependem de uma única pessoa. Nesse sentido, o trabalho contribuiu para o amadurecimento na condução de um projeto *full stack* de forma mais estruturada.

Por fim, o projeto trouxe aprendizado prático sobre integração com APIs externas, especialmente a API do WhatsApp, e sobre a construção de um *chatbot* orientado a regras e validações. Esse conhecimento mostrou a importância de tratar cuidadosamente autenticação, eventos assíncronos, persistência de contexto e respostas automatizadas em sistemas que interagem diretamente com usuários.

Repositório do Trabalho

O código-fonte desenvolvido no trabalho encontra-se versionado em repositório GitHub, no seguinte endereço:

<https://github.com/ICEI-PUC-Minas-PPLES-TI/plf-es-2025-2-tcci-0393100-dev-bernardo-biondini>